



- Introduzione
- Numeri infiniti
- Non determinismo
 - E-closure
 - Operazioni tra linguaggi
 - Pumping Lemma
 - Teorema di Myhill-Nerode
 - 1) $\rightarrow$ 2)
 - 2) $\rightarrow$ 3)
 - 3) $\rightarrow$ 1)
 - Proprietà di chiusura
- Linguaggi CF
 - Albero di Derivazione (ParseTree)
 - Pumping Lemma per i linguaggi CF
 - Riassuntino:
 - Unione di linguaggi CF
 - Decidibilità
 - Requisiti per essere un algoritmo:
- Gerarchia di Chomsky
 - Tipo 1
 - Tipo 0
 - Macchine di Turing
 - Kleene-Robinson
 - Ackermann
 - ricapitolando
 - Pairing function
 - Teorema di equivalenza delle funzioni calcolabili
 - Tesi di Church-Turing:
 - Teorema di indecidibilità di Halting
 - Teorema S-M-N
 - Classificazione delle funzioni calcolabili
 - TEOREMA
 - 1) $\Rightarrow$ 2)
 - 3) $\Rightarrow$ 1)
- Tipologia di insiemi ricorsivi
 - Una terminazione in un punto
 - Due terminazioni
- I Teorema di Ricorsione
 - Il teorema di ricorsione
 - Teorema di Rice
 - Proprietà di Programmi
 - RICE-SHAPIRO
 - Teorema di Myhill
 - Insiemi semplici
 - Teorema di Selberg
 - Complessità computazionale
 - Decimali vs Minimizzazione
 - Complessità computazionale: Macchina di Turing



Introduzione

sito del Prof: <https://users.dimi.uniud.it/~agostino.dovier/>

orari:

- LUN 8.30-12.30 (C2)
- MER 8.30-12.30 (C2)
- GIO 8.30-12.30 (C2)

ricevimento:

- **A2 126:** LUN 16.30-18.30

esame: Scritto e Orale (entrambi obbligatori e lo stesso giorno, uno la mattina e uno il pomeriggio)

Programma:

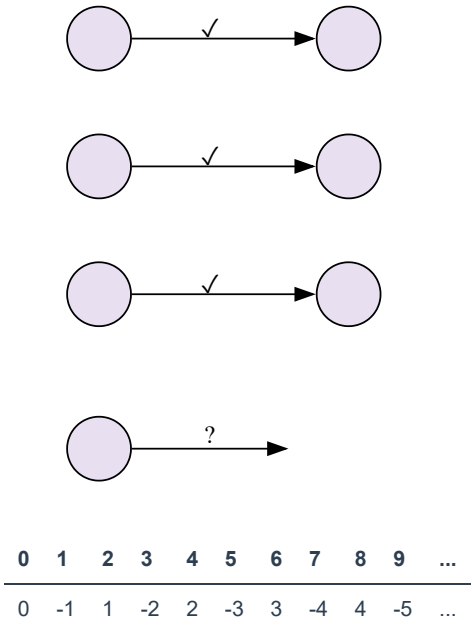
- Linguaggi formali
- Computabilità
- Complessità



Numeri infiniti

I programmi per definizione hanno una lunghezza **finita**, ma un singolo programma può avere un **numero infinito** di modi per essere scritto (ad esempio aggiungendo linee di codice **inutili**)

- Abbiamo degli insiemi di numeri $\mathbb{N}, \mathbb{Z}, \mathbb{Q}, \mathbb{R}$
- Sappiamo che $\mathbb{N} \subset \mathbb{Z}$ (è incluso strettamente)
- Due insiemi A e B hanno la stessa cardinalità se esiste una funzione biettiva $g : A \rightarrow B$



Se volessimo per i pari e dispari, come positivi e negativi:

$$g(x) = \begin{cases} x/2 & x \text{ è pari} \\ -(x+1)/2 & x \text{ è dispari} \end{cases}$$

$$g(x) = \frac{x}{2}(1 - x \bmod 2) - \frac{x+1}{2}(x \bmod 2)$$

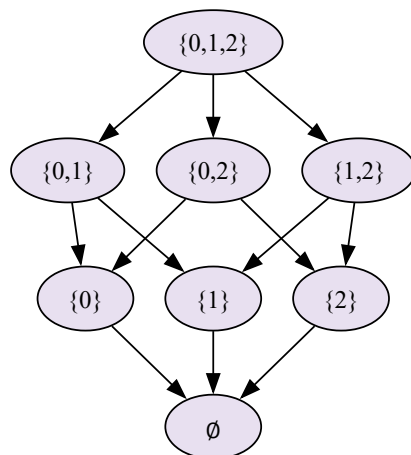
$\mathbb{N} = \mathbb{Z}$

	1	2	3	4	5	6	7	8	9	10	...
1	1/1	2/1	3/1	4/1	5/1	6/1	7/1	8/1	9/1	...	
2	1/2	1/1	3/2	2/1	5/2	3/1	7/2	4/1	9/2	...	
3	1/3	2/3	1/1	4/3	5/3	2/1	7/3	8/3	3/1	...	
4	1/4	1/2	3/4	1/1	5/4	3/2	7/4	2/1	9/4	...	
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮

Come le righe, le colonne sono infinite

- $|\mathbb{Q}| = |\mathbb{N}|$

Qualore invece si prendessere in condiserazione i sottoinsiemi di A e considerano



Idea di Cantor (anche al finito):

- Consideriamo l'insieme $\{0,1,2,3\}$ e immaginiamo una (a caso) funzione iniettiva g da $\{0,1,2,3\}$ a $P(\{0,1,2,3\})$

	0	1	2	3	
$g(0)$	1	0	1	1	ovvero $\{0,2,3\}$
$g(1)$	1	0	1	0	ovvero $\{0,2\}$
$g(2)$	0	0	0	0	ovvero $\{\emptyset\}$
$g(3)$	0	1	1	1	ovvero $\{1,2,3\}$

- Guardando la diagonale, la nego (inverto) ottengo $\{1,2\}$ che è diverso da tutti gli altri insiemi
- Diverso dal primo su 0, del secondo su 1, dal terzo su 2, dal quarto su 3
- Vedo come cantor porta questo ragionamento all'infinito.
- Supponiamo quindi **per assurdo** che esista $g: \mathbb{N} \rightarrow P(\mathbb{N})$ biettiva
- Dunque per ogni $g(i)$ è un sottinsieme (finito o infinito) di $\mathbb{N}$
- Lo possiamo rappresentare con dei 0 (elemento non nell'insieme) e 1 nella seguente tabella: (vedi tabella da slides)
- L'insieme (continua da slides)

Quindi ci chiediamo:

- quante sono le stringhe $\{0,1\}$ di 1 elemento?
- Quante sono le stringhe $\{0,1\}$ di 2 elementi?
- Quante sono le stringhe $\{0,1\}$ di n elementi?

ε	0	1	00	01	10	11	000	001	010	011	100	101	110	...
0	1	2	3	4	5	6	7	8	9	10	...			

- notare che riuscirei a metter ein ordine e in corrispondenza con i numeri naturali (Assumendo che ci sia anche la stringa di zero elementi, ovvero ε)
- Dunque le stringhe binarie (continua con le slides) (...)

Sia f una funzione che accetta di input un numero naturale e risponde 0 o 1 (a seconda di una qualunque proprietà del numero)

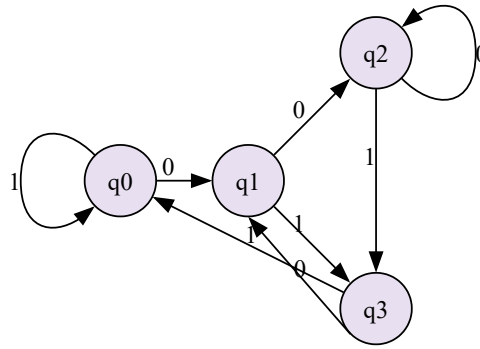
Siamo in grado di calcolarle **tutte** le combinazioni?

NO

L'insieme è infinito e **Non** numerabile, e quindi non calcolabili da nessun calcolatore e quindi programma.

Lezioni da rivedere

Fine lezioni da rivedere



(vedi foto telefono per rifare tabella)

$$\hat{S}(q, x \cdot y) = \hat{S}(\hat{S}(q, x), y)$$

$$u \in L \implies \hat{S}(q_0, u) \in F$$

con questo automa dobbiamo prima ragionare una parte per induzione su due stringhe su y :

induzione su y , sulla lunghezza di $|y| \leq 0$

BASE: $y = \epsilon' (|y| = 0)$

$$\hat{\delta}(q, x \cdot \epsilon) = \hat{\delta}(\hat{\delta}(q, x), \epsilon)$$

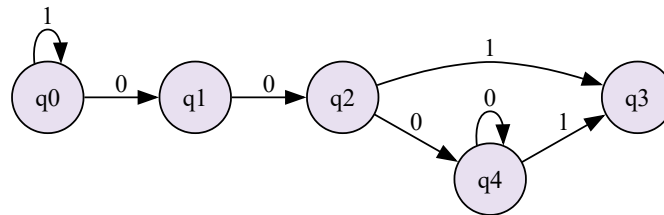
passo induttivo:

$$y = ua$$

$$|y| = u - 1$$

$$\hat{\delta}(q, x \cdot u \cdot a) = \delta(\hat{\delta}(q, u), a)$$

$$\{x0 \quad ab \quad x \in \{0, 1\}^x \quad a, b \in \{0, 1\}\}$$





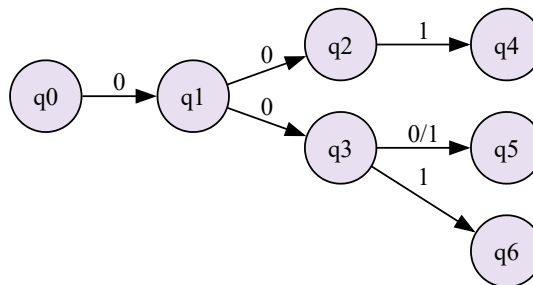
Non determinismo

(trovati un immagine che mostra un sistema di ricerca binaria vs un sistema di ricerca quantistica)

Con i principi non deterministici, si corre nel rischio di non concludere mai o comunque in elevato tempo, perchè tutti i singoli elementi di ricerca devono finire.

Implementare il non determinismo tramite parallelizzazione dei processi funziona, ma solo con piccoli problemi, perchè il numero di processori finisce rapidamente.

Quindi un automa potrebbe essere:



ci fermiamo al primo 1 trovato (abbiamo trovato un percorso che porta alla destinazione)

Con **001**: $\hat{\delta}(p_0, 001)$

q_4	yes
q_5	no
q_6	no

Con **00**: $\hat{\delta}(p_0, 00)$

q_2	no
q_3	no

quindi non abbiamo trovato un percorso che porta alla destinazione

Ci interessa trovare solo un percorso che porta alla destinazione, se vi sono più percorsi, non ci interessa, basta uno solo.

Un automa a stati finiti non deterministico (NFA) è una quintupla $M = (Q, \Sigma, \delta, q_0, F)$ dove:

- Q è un insieme finito di stati
- Σ è un alfabeto finito di simboli
- $\delta : Q \times \Sigma \rightarrow 2^Q$ è
- $q_0 \in Q$ è lo stato iniziale
- $F \subseteq Q$

$$\delta : Q \times \Sigma^* \rightarrow \beta(Q)$$

$$\delta(q_0, 0) = \{q_1\} \quad \delta(q_0, 1) = \emptyset$$

$$\delta(q_1, 0) = \{q_2, q_3\} \quad 00$$

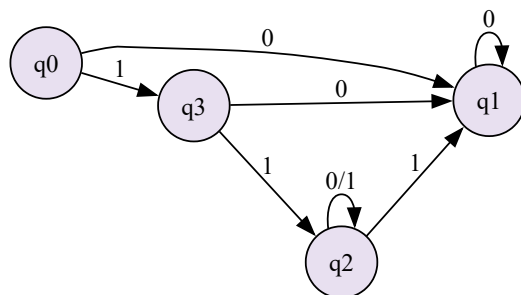
$$\delta(q_2, 1) = \{q_4\} \quad \hat{\delta}(q_0, 00)$$

quindi:

$$\hat{\delta} : Q \times \Sigma^* \rightarrow \beta(Q)$$

$$\begin{cases} \hat{\delta}(q, \epsilon) = \{q\} \\ \hat{\delta}(q, va) = \bigcup_{p \in \hat{\delta}(q, v)} \delta(p, a) \end{cases}$$

Un DFA è un NFA?



Se L è regolare $\implies \exists$ NFA che lo accetta

	0	1	2	3	4	5	6
$\hat{\delta}(q_0, \epsilon) = q_0$	1	0	0	0	0	0	0
0 ->	0	1	0	0	0	0	0
0 ->	0	0	1	1	0	0	0
1 ->	0	0	0	0	1	1	1

questo non determinismo sembra facilmente gestibile

DFA e NFA:

$$M = \langle Q, \Sigma, \delta, q_0, F \rangle$$

$$\delta : Q \times \Sigma \rightarrow Q \mid \delta : Q \times \Sigma \rightarrow \beta(Q)$$

Serie infinita di passaggi

$$\hat{\delta}(q, v_x) = \bigcup_{p \in \hat{\delta}(q, v)} \delta(p, a)$$

Con il **teorema di RadinScott**:

Se L è accettato da un NFA M , allora esiste un DFA M' tale che $L = L(M')$

DIM:

- Sia M NFA, $M = (Q, \Sigma, \delta, p_a, F)$
- Definisco $M' = (Q', \Sigma, \delta', q'_0, F')$ dove:
- $Q' \approx \beta(Q)$

Esempio

$Q' =$	$\{q'_0 \rightarrow \{q_0\}\}$
$q'_1 \rightarrow \{q_1\}$	
q'_2	
q'_3	
$q'_5 \rightarrow \{q_1, q_2\}$	
$q'_6 \rightarrow \{q_0, q_1, q_2\}$	

$$F' = \{A \in Q : A \cup F \neq \emptyset\}$$

$$\delta'(A, a) = \bigcup_{p \in A} \delta(p, a)$$

Continuando con la dimostrazione:

$$(\forall x \in \delta^*)(x \in L(M) \iff x \in L(M'))$$

$$(\forall x \in \delta^*)(\hat{\delta}(q_0, x) \cap F \neq \emptyset \iff \hat{\delta}(q'_0, x) \in F')$$

$$(\forall x \in \delta^*)(\hat{\delta}(p_a, x) \cap F \neq \emptyset \iff \hat{\delta}(q'_0, x) \in F' \neq \emptyset)$$

Mostro per induzione sulla lunghezza x che $\hat{\delta}(q, x) = \hat{\delta}'(q\{ \}, x)$

Base: $x = \epsilon$



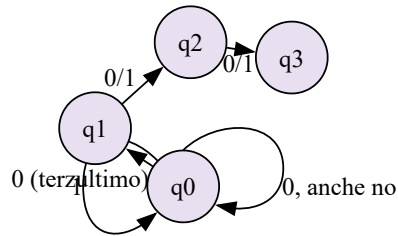
$$\hat{\delta}(q, \epsilon) =^* \{q\} \quad \hat{\delta}'(\{q\}, \epsilon) =^{*2} \{q\}$$

passo:

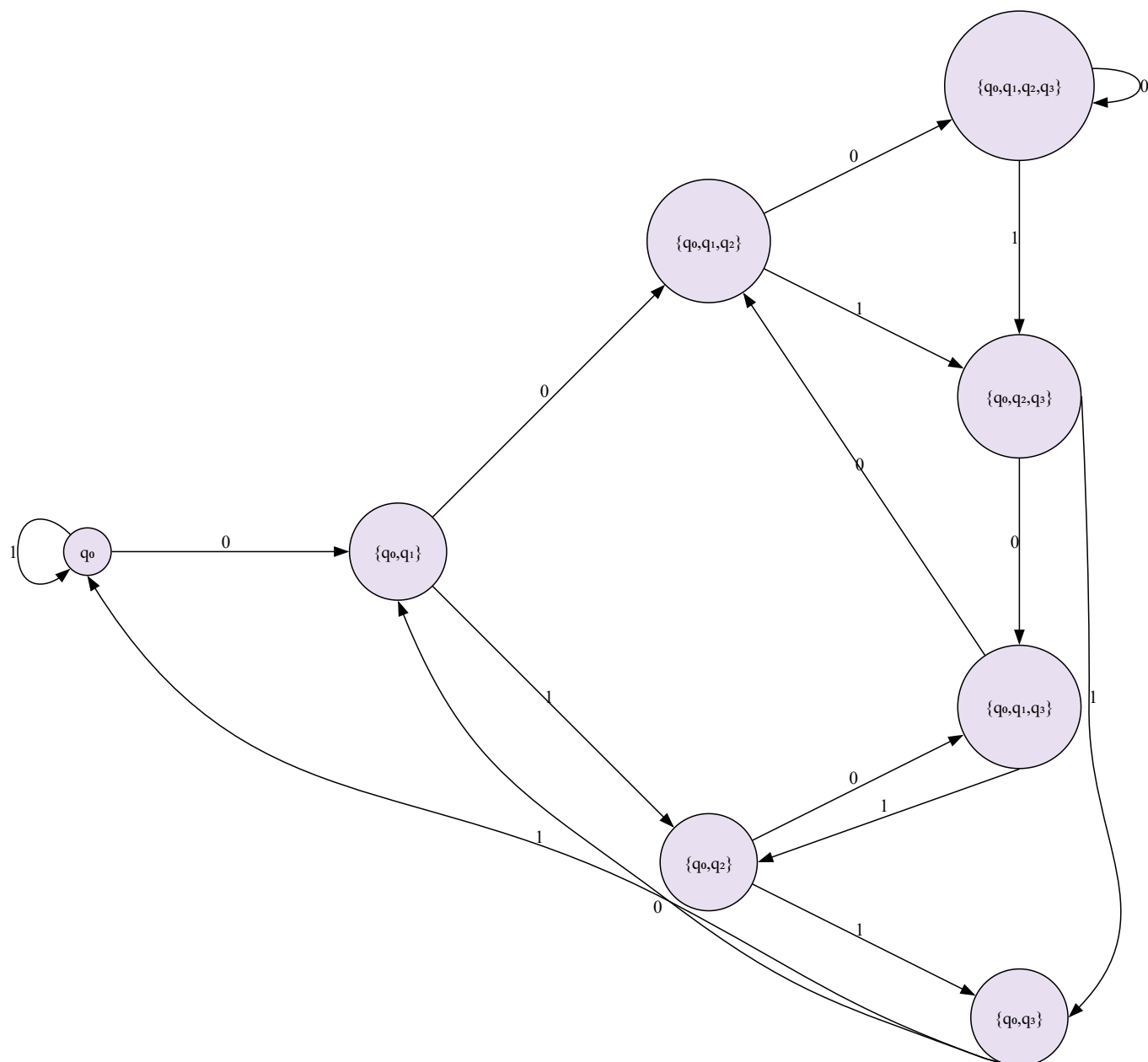
$$\hat{\delta}(q, x) = \hat{\delta}(q, Va) =^{*3} \bigcup_{p \in \hat{\delta}(q, V)} \delta(p, a)$$

$$\hat{\delta}'(\hat{\sigma}'(\{q\}, V), q) = \bigcup_{p \in \hat{\delta}'(\{q\}, V)} \delta'(p, a)$$

Quindi per ipotesi induttiva **sono uguali**.



	0	1
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_0, q_1\}$	$\{q_0, q_1, q_2\}$	$\{q_0, q_2\}$
$\{q_0, q_2\}$	$\{q_0 q_1, q_3\}$	$\{q_0, q_3\}$
$\{q_0, q_3\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_0, q_1, 1_3\}$	$\{q_0, q_1, q_2\}$	$\{q_0, q_2\}$
$\{q_0, q_1, q_2\}$	$\{q_0, q_1, q_2, q_3\}$	$\{q_0, q_2\}$
$\{q_0, q_1, q_2, q_3\}$	$\{q_0, q_1, q_2, q_3\}$	$\{q_0, q_2, q_3\}$
$\{q_0, q_2, q_3\}$	$\{q_0, q_1, q_3\}$	$\{q_0, q_3\}$



Esempio: $A = \{0,001,1001\}$

(copiati l'albero alla lavagna)

$B = \Sigma^* \{11,000,1010\}$

Devo individuare gli n-cammini che accettano queste stringhe

Dimostrazione:

$L(M) = \epsilon^* / L(M)$ è regolare

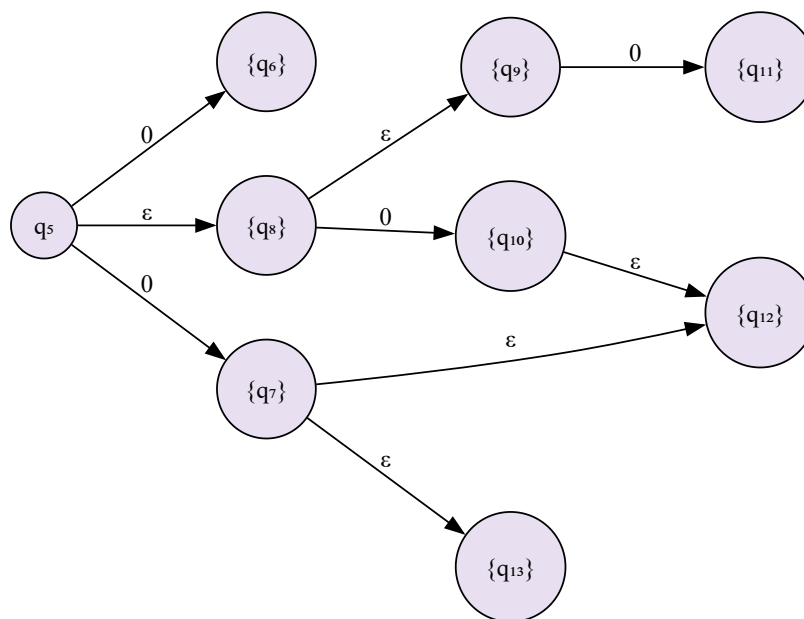
$L_i = \{0, 1\}$

$\bigcup_{i \leq 0} L_i = \{0^n, 1^n : n \in \mathbb{N}\}$

Questo vale solo per i finiti, per all'infinito va usato un altro modo.



E-closure



$\varepsilon\text{-closure}(q)$ è l'insieme di stati raggiunti da q da soli archi ε

Es:

$$\varepsilon\text{-closure}(q_{13}) = \{q_{13}\}$$

$$\varepsilon\text{-closure}(q_7) = \{q_7, q_{12}, q_{13}\}$$

$$\varepsilon\text{-closure}(q_5) = \{q_5, q_8, q_9\}$$

$$\hat{\delta}(q_9, 0) \stackrel{?}{=} \left\{ q_6, q_7, q_{12}, q_{13}, q_{10}, q_{11} \right\}_{\varepsilon\text{-closure}}$$

Un automa non deterministico con ε -transizioni (ε -NFA) è una quintupla $M = (Q, \Sigma, \delta, p_0, F)$ dove:

$$Q, \epsilon, \delta, p_0, F$$

$$\delta : Q \times (\Sigma \cup \{\epsilon\}) \rightarrow \mathcal{P}(Q)$$

$$\hat{\delta}(q, \epsilon) = \varepsilon\text{-closure}(q)$$

$$\hat{\delta}(q, Va) = \bigcup_{p \in \hat{\delta}(q, V)} \varepsilon\text{-closure}(\delta(p, a))$$

$$\varepsilon\text{-closure}(P) = \bigcup_{p \in P} \varepsilon\text{-closure}(p)$$

dimostrando quindi:

$$\delta : Q \times (\Sigma \cup \{\epsilon\}) \rightarrow \mathcal{P}(Q) \quad L(M) = \{x \in \Sigma^* : \hat{\delta}(p_0, x) \cap F \neq \emptyset\}$$

Teorema:

Se L è accettabile ε -NFA M , allora esiste un NFA M' tale che $L = L(M')$

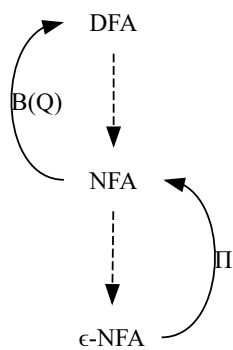
(Fatti il grafico, di nuovo)

Caso strano:

$\varepsilon\text{-NFA}$	NFL
grafico 1	grafico 2
accetta ε	non accetta ε

In questo caso, se $\varepsilon\text{-closure}(q_0) \cap F \neq \emptyset$:

$$e : q_a \notin F$$



Riassumendo:

$$\hat{\delta}(q, a) = \varepsilon - closure \left(\bigcap_{p \in \varepsilon - closure(a)} \delta(p, p) \right)$$



Operazioni tra linguaggi

abbiamo:

$$\Sigma L, L_1, L_2 \varepsilon \Sigma^*$$

contenente: $L = \Sigma^* \setminus L$

Unione: $L_1 \cup L_2 = \{x \in \Sigma^* : x \in L_1 \text{ o } x \in L_2\}$

Inclusione: $L_1 \subseteq L_2 = \{x \in \Sigma^* : x \in L_1 \text{ e } x \in L_2\}$

Concatenazione: $L_1 \cdot L_2 = \{uv : u \in L_1, \text{ e } v \in L_2\}$

$$g(x) = \begin{cases} L^0 = \{\varepsilon\} \\ L^n = L^{n-1} \cdot L^1 \end{cases}$$

Esempio:

$$L = \{0, 10\}$$

$$L^0 = \{\varepsilon\}$$

$$L^1 = \{0, 10\}$$

$$L^2 = \{00, 010, 100, 1010\}$$

⋮

Una abbreviazione:

$$(L^+ = \bigcup_{i>0} L^i = L^* \setminus \{\varepsilon\})$$

Espressioni regolari:

- ε è un'espressione regolare che denota il linguaggio $\{\varepsilon\}$
- $a \in \Sigma$ è un'espressione regolare che denota il linguaggio $\{a\}$
- $\emptyset$ è un'espressione regolare che denota il linguaggio vuoto

Se e_1 e e_2 sono espressioni regolari, allora:

- $e_1 \cdot e_2$ è un'espressione regolare che denota il linguaggio $L(e_1) \cdot L(e_2)$
- $e_1 + e_2$ è un'espressione regolare che denota il linguaggio $L(e_1) \cup L(e_2)$

Se e è un'espressione regolare, allora e^* è un'espressione regolare che denota il linguaggio $L(e)^*$

Esempio:

$$(0 + 1) \rightsquigarrow L(0) \cup L(1) \rightsquigarrow \{0, 1\} \cup \{1\} = \{0, 1\}$$

$$((0 + 1) \cdot (0 \cdot (0 + 1))) \rightsquigarrow \{0, 1\} \cdot \{00, 01\} = \{000, 001, 100, 101\}$$

Se usassimo star:

$$(0 + 1)^* = \{0, 1\}^* = \{\varepsilon, 0, 1, 00, 01, 10, 11, \dots\}$$

$$(0 + 1)^* \cdot 0 \cdot (0 + 1) \cdot (0 + 1) \rightsquigarrow \text{determina l'insieme di stringhe con terzultimo carattere 0}$$

Un esempio di dimostrazione:

somma:

$$a + b = b + a \text{ ? } \rightarrow L(a) \cup L(b) = L(b) \cup L(a)$$

E quindi sono uguali perchè l'unione è commutativa. *Prodotto:*

$$l \cdot r = r \cdot l \text{ ?}$$

$$l=0, r=1$$

$$\begin{array}{cc} 0 \cdot 1 & 1 \cdot 0 \\ \hline \{01\} & \{10\} \end{array}$$

Quindi **NON** sono uguali, perchè la concatenazione **NON** è commutativa.

$$\varepsilon + r \cdot r^* = r^*$$

$$L(\varepsilon + r \cdot r^*) = \{\varepsilon\} \cup L(r) \cdot L(r^*) = L(r^*) = L(r)^*$$

- se $i = 0 \implies x = \varepsilon \rightarrow x \in \{\varepsilon\} \cup (\text{robassopra})$
- se $i > 0 \implies x = uV$
 - $u \in L(M)$
 - $V \in L(M^{i-1})$



$$\blacksquare uV \in L(M) \cdot L(M)^*$$

$$r \cdot (n + t) = (rs) + (rt):$$

$$L(r \cdot (s + t)) = L(rs + rt)$$

$$x \in L(r \cdot (n + t)) \rightarrow x \in L(n + t)$$

$$1. u \in L(r) \text{ o } V \in L(n) \rightarrow uv \in L(rs)$$

$$2. u \in L(M) \text{ o } V \in L(t) \rightarrow uv \in L(rt)$$

(sarò sincero, mi sono perso qualche passaggio:D)

$$\text{quindi: } x \in L(r(s + t))$$

Ma con numeri?

$$2 + (3 \cdot 4)(2 + 3) \cdot (2 + 4) = 62$$

Lo "star" lo possiamo vedere come un esponente, concatenazione come moltiplicazione, e l'unione come addizione.

Teorema:

Se l è regolare, allora $L(l)$ è regolare

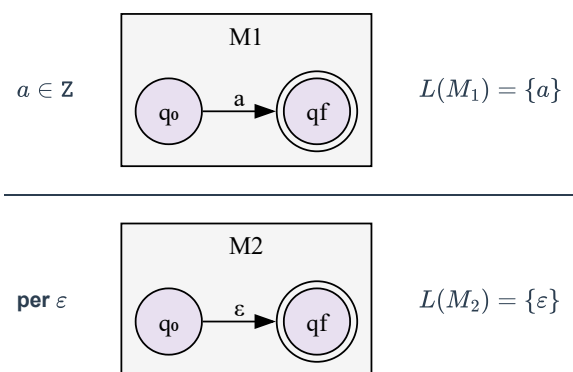
Dimostrazione:

Per induzione sulla struttura di l :

esiste un ϵ -NFA M tale che:

$L(M) = L(l)$; e tale che M ha un unico stato finale distinti tra di loro.

Base:



Passo:

- $l \rightsquigarrow L(l)$ è accettato da (grafico: $q_0 \xrightarrow{a} \dots \xrightarrow{a} q_f$)
- $M \rightsquigarrow L(l)$ è accettato da (grafico: $q_0 \xrightarrow{\epsilon} \dots \xrightarrow{\epsilon} q_f$)

$$\text{Vedo come: } pn \mid + r \implies L(l + r) = L(l) \cup L(r)$$

GRAFICO DI NUOVO

Visto che avere due stati finali crea problemi, aggiungiamo 2 epsilon, portando a un solo stato finale. (**grafico**)

$$L(l) \cup L(r)$$

$$\text{Vediamo come costruire automa per: } l \cdot r: \quad L(l \cdot r) = L(l) \cdot L(r)$$

$$\text{GRAFICO DI NUOVO } (Q_0 \xrightarrow{\epsilon} Q_0' \xrightarrow{\epsilon} Q_1 \xrightarrow{\epsilon} Q_1' \xrightarrow{\epsilon} Q_2 \xrightarrow{\epsilon} Q_2' \xrightarrow{\epsilon} \dots \xrightarrow{\epsilon} Q_n \xrightarrow{\epsilon} Q_n')$$

IPOTESI:

$$L \rightsquigarrow L(l) \text{ p accettato da (grafico: } q_0 \xrightarrow{\epsilon} \dots \xrightarrow{\epsilon} q_f)$$

Voglio definire un automa che accetta $L(l) \text{star}$ (che è uguale a: $\{\epsilon\} \cup L(l) \cup L(l) \cdot L(l) \cup L(l) \cdot L(l) \cdot L(l) \cup \dots$)

indovina che devi fare: (sì, è proprio un grafico)

Come ci aspettava le espressioni regolari si catturano le operazioni regolari.

Ma possiamo descriverle **tutte**?

$$\text{Grafico: } q_0 \xrightarrow{\epsilon} |0| q_1 q_0 \xrightarrow{\epsilon} |1| q_2 q_1 \xrightarrow{\epsilon} |0| q_3 q_1 \xrightarrow{\epsilon} |1| q_2 q_2 \xrightarrow{\epsilon} |0| q_3 q_2 \xrightarrow{\epsilon} |1| q_2 (q_3) \xrightarrow{\epsilon} |0| q_3 (q_3) \xrightarrow{\epsilon} |1| q_2$$



$$\begin{aligned}
a \in Q &\rightsquigarrow L(q) = \{x \in \Sigma^* : \hat{\delta}(p_0, x) = q\} \\
L(q_0) &= \{\varepsilon\} \text{ (in ogni DFA } \varepsilon \in L(q_0)) \\
L(q_1) &= \{0\} = L(q_0) \cdot 0 \quad L(q_2) = (L(q_0) \cdot 1 + L(q_1) \cdot 1 + L(q_3) \cdot 1)^* \\
L(q_3) &= (L(q_1) \cdot 0 + L(q_2) \cdot 0)0^*
\end{aligned}$$

Quindi:

$$L(q_0) \Rightarrow Q_0$$

$$L(q_1) \Rightarrow Q_1$$

$$L(q_2) \Rightarrow Q_2$$

$$L(q_3) \Rightarrow Q_3$$

$$\begin{cases} Q_0 = \varepsilon \\ Q_1 = Q_0 \cdot 0 \\ Q_2 = (Q_0 \cdot 1 + Q_1 \cdot 1 + Q_3 \cdot 1)^* \\ Q_3 = (Q_1 \cdot 0 + Q_2 \cdot 0)0^* \end{cases}$$

Per sostituzione:

$$\begin{aligned}
\begin{cases} Q_0 = \varepsilon \\ Q_1 = \varepsilon \cdot 0 = 0 \\ Q_2 = (\varepsilon \cdot 1 + 0 \cdot 1 + Q_3 \cdot 1)^* \\ Q_3 = (0 \cdot 0 + Q_2 \cdot 0)0^* \end{cases} &\Rightarrow \begin{cases} Q_0 = \varepsilon \\ Q_1 = 0 \\ Q_2 = (Q_3 \cdot 1 \cdot 1^* + (1 \cdot 1^* + 0 \cdot 1 \cdot 1^*))1^* \\ Q_3 = Q_2 \cdot 0 \cdot 0^* + 0 \cdot 0 \cdot 0^* = Q_2 \cdot 0^+ + 0 \cdot 0^* \end{cases} \\
\begin{cases} Q_0 = \varepsilon \\ Q_1 = 0 \\ Q_2 = Q_2 \cdot 0^+ \cdot 1^+ + 00^+1^+ + 1^+ + 01^+ \\ Q_3 = Q_2 \cdot 0^+ + 0 \cdot 0^+ \end{cases} &\Rightarrow \begin{cases} Q_0 = \varepsilon \\ Q_1 = 0 \\ Q_2 = Q_2 \cdot 0^+ \cdot 1^+ + 00^+1^+ + 1^+ + 01^+ \\ Q_3 = Q_2 \cdot 0^+ + 0 \cdot 0^+ \end{cases}
\end{aligned}$$

Noto che la stringa di Q_2 " $Q_2 \cdot 0^+ \cdot 1^+ + 00^+1^+ + 1^+ + 01^+$ " è il tragitto da q_0 a q_2

Esempio:

$$Q = Qr + s$$

$$Q = (Qr + s)r + s = Qr^2 + sr + s$$

$$Q = (Qr + sr)r^s + sr + s = Qr^3 + sr^2 + sr + s$$

quindi possiamo supporre che:

$$Q = sr^*$$

Ed è anche la più piccola possibile

Finendo la dimostrazione:

$$Q_2 = (00^+1 + 1^+ + 01^+) \cdot (0^+ \cdot 1^+)^*$$

sostituendo nel sistema precedente possiamo trovare anche q_3 :

$$Q_3 = (00^+1 + 1^+ + 01^+) \cdot (0^+ \cdot 1^+)^* \cdot 0^+ + 0 \cdot 0^*$$

Dimostrazione:

DFA $Q, = \{Q_0, -, Q_n\}$ indico $|Q|$ variabili $Q_0, \dots, Q_n$ scrivo una sistema di equazione

$$Q = (Q_{i1}a_{i1} + Q_{i2}a_{i2} + \dots + Q_{ii}a_{ii} + \varepsilon)(b_{i1} + b_{i2} + \dots + b_{im})^*$$

Se $i = 0$ sostuiamo ε con q_1

$$\text{Se capita che } Q = QM + s \Rightarrow Q = sM^*$$

Essendo **deterministico** allora con:

$$F = \{q_{a1} + q_{a2} + \dots + q_{al}\}$$

allora:

$$L(M) = Q_{a1} + Q_{a2} + \dots + Q_{al}$$



GRAFICOOOOO

$$\begin{cases} Q_0 = Q_3 1 + \varepsilon \\ Q_1 = Q_0 0 + Q_3 0 \\ Q_2 = (Q_1 0) 0^+ \\ Q_3 = Q_2 \cdot 1 Q_1 1 \end{cases}$$

Sostituendo:

$$\begin{cases} Q_0 = Q_1 0^* 1 1^+ 1^* \\ Q_1 = Q_0 0 + Q_1 0^* 1 0 \\ Q_2 = Q_1 0^* \\ Q_3 = Q_1 0^* 1 \end{cases}$$

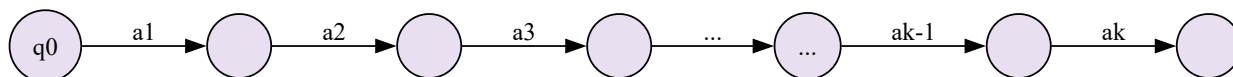
Usando $Q = Qr + n \implies sr^*$:

$$\begin{aligned} Q_1 &= 1^* 0 (0^* 1 1^* 0)^* \\ Q_2 &= 1^* 0 (0^* 1 1^* 0)^* 0^+ \\ Q_3 &= 1^* 0 (0^* 1 1^* 0)^* 0^+ 1 + 1^* 0 (0^* 1 1^* 0)^* \end{aligned}$$

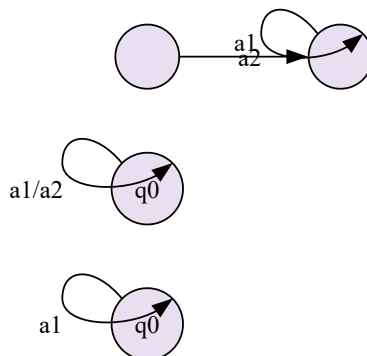


Pumping Lemma

Prendiamo un DFA e una stringa $x = a_1 a_2 \dots a_k$.



Se il DFA ha n stati e $k \geq n$, cosa posso dire con certezza?



Che almeno uno stato viene visitato più di una volta

(schema)

Dove $x = uvw$ $|uv| \leq n$

Cosa succederebbe alla stringa uw ?

Il DFA è **deterministico**.

Cosa succede ad $uv^i w$ ($i \geq 0$)?

Legandole per ogni $i \geq 0$

$uv^i w \in L(M)$

Se L è un linguaggio regolare, allora esiste $n \in \mathbb{N}$ tale che per ogni $x \in L$ con $|x| \geq n$, esistono Stringhe u, v, w tali che: $(uv) \leq n, |v| > 0, x = uvw$ e per ogni $i \in \mathbb{N}$ $uv^i w \in L(M)$

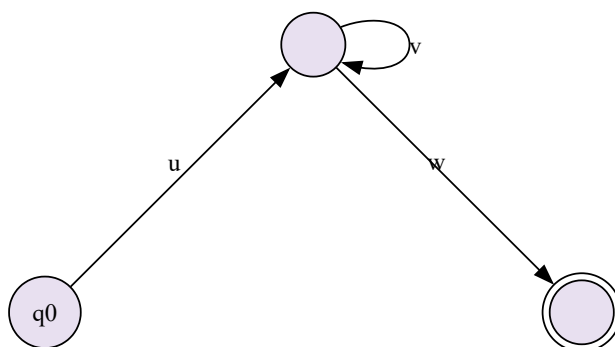
Dimostrazione:

sia $x \in L(M)$, con $|x| \geq n = |Q|$

Allora esiste u, v, w tali che $x = uvw$ $|uv| \leq n, |v| > 0$ e per ogni $i \in \mathbb{N}$

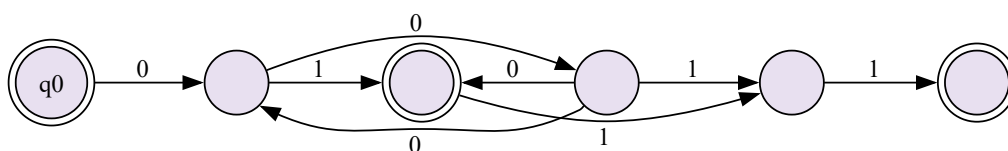
$uv^i w \in L(M)$

$V^0 = \varepsilon$ $V^2 = uv$ $V^3 = uvu$



Esempio:

Supponiamo che M abbia 6 stati



$A = \{0^m 1^m : m \geq 0\}$



Se per assurdo esiste M , DFA, che accetta A , sia n il numero di stati di M .

$n = 0^n 1^n$ e vedo che fin nella parte "0" avrò n stati. Dove (**vedi appunti di cudiz**)

$$(\forall L \in \Sigma^*) ("L \text{ è regolare}" \rightarrow (\exists n \in \mathbb{N})(\forall n)(x \in L \wedge |x| \geq n))((\exists u \in \Sigma^*)(\exists v \in v\Sigma^*)(\exists w \in \Sigma^*)(x = uvw \wedge |nv| \leq n(e)|v| > 0 \wedge (\forall i \in (\forall n \in \mathbb{N})(\exists x)(x \in L \wedge |x| \leq n) + c\forall u\forall v\forall w(x = uw \wedge |w| \leq n \wedge |u| > 0) \rightarrow (\exists u \in \mathbb{N})(uvw \notin L))$$

Considero L , assumendo che è regolare, devo dimostrare che:

Per ogni $n \in \mathbb{N}$ $n \leq$ avente n con $x \in L$, $|x| \geq n$ tale che per ogni modo di *parola strana* $x=uvw$, con $|uv| \leq n$ e $|v| > 0$ so che w è tale che $uv'w \notin L$

Esempio:

Dato n (arbitrario) scelgo $x = 0^n 1^n \in A \wedge |n| \geq n$

$x = 00 \dots 011 \dots 1$

caso 1:

$$u = 0^a \quad 0 \leq a < n$$

$$v = 0^b \quad a + b \leq n$$

$$w = 0^{n-a-b} 1^n$$

Per ogni modo di partizionarlo visto sopra, prendo $i = 0$, e quindi la stringa diventa:

$$uw = 0^a 0^{n-a-b} 1^n = 0^{n-b} 1^n \notin A \text{ essendo che } b > 0, n-b < n. \text{ (funziona anche al contrario, aumentando i)}$$

Ma se prendessimo una stringa più corta: $x = 0^{\frac{n}{2}} 1^{\frac{n}{2}} \in A \wedge |x| \leq n$

Con $i = 0$ non funziona, perchè appartiene ancora al linguaggio.

Con $i = 2$ funziona

Notiamo rapidamente che se prendessimo una stringa più corta, non potremmo dimostrare che non è regolare, perchè non abbiamo abbastanza stati per farlo.

Per casa:

$$C = \{0^p 1^q : p \text{ è un numero primo}\}$$

$$D = \{x \in \{0, 1\}^* : x \text{ è palindromo}\}$$

$$L(M) = \emptyset?$$

$$L(M) \text{ è finito/infinito ?}$$

$$\text{Poniamo che la più piccola } x \in L(M) |n| > n$$

Se per assurdo la stringa più corta trovata è più lunga di n , allora per il pumping lemma, possiamo trovare una stringa più corta riducendo una più piccola di n

Per il lemma superiore, se avessi un n infinito, e trovassi una stringa infinita, vuol dire che avrei una stringa lunga infinito.

Se trovassi una stringa $\geq n$ è maggiore di $2n$, se trovo qualcosa tra n e $2n$ vuol dire che ho un numero infinito di stringhe? (**ricontrolla**)



Teorema di Myhill-Nerode

Relazioni su S $R \in S \times S$

R si dice di equivalenza se è:

- $(\forall x \in S)(x, x) \in R, xRx$ (riflessività)
- $(\forall x \in S)(\forall y \in S)(xRy \rightarrow yRx)$ (simmetria)
- $(\forall x \in S)(\forall y \in S)(\forall z \in S)(xRy \wedge yRz \rightarrow xRz)$ (transitività)

Partizioni di S :

$S_1, S_2, \dots, S_k$ è una partizione di S se:

- $\forall i \quad S_i \neq \emptyset$
- $\forall i \forall j (i \neq j \rightarrow S_i \cap S_j = \emptyset)$
- $\bigcup_{i \geq 1} S_i = S$

1 (S_1)	1 (S_1)	1 (S_1)
2 (S_2)	2 (S_1)	2 (S_2)
3 (S_2)	3 (S_2)	2 (S_3)
4 (S_2)	4 (S_3)	4 (S_3)
5 (S_3)	5 (S_3)	5 (S_4)

Il primo e il terzo insieme sono molto simili, quindi forse sono in qualche legame.

Promemoria:

Se $a \subseteq S$, con $[a]$ spesso si indica le sua "classe" di appartenenza.

P_1	P_3
$[1]=\{1\}$	$[1]=\{1\}$
$[2]=\{2,3,4\}$	$[2]=\{2\}$
$[3]=[4]=\{2,3,4\}$	$[3]=\{3,4\}$
$[5]=\{5\}$	$[4]=\{3,4\}$
	$[5]=\{5\}$

Nota che sono tutti gli elementi di P_3 sono sottoinsiemi di quelli di P_1

P_3 è un raffinamento di P_1 , si può anche dire che p_1 è più grossolana di P_3

Per esempio se prendessi $\mathbb{N}$ posso partizionarlo in $P_1 = \{\text{pari, dispari}\}$

Se una partizione ha un numero finito di elementi, la partizione si dice di indice finito

$R \subseteq S \times S$ è di indice finito se una partizione con un numero finito di classi. Date:

$R_1, R_2 \subseteq S \times S$ equivalente si dice che R_1 è più fine di R_2 (o che R_2 è più grossolana di R_1)

se $(\forall x \in S)([x]_{R_1} \subseteq [x]_{R_2})$

$xRmy \iff \hat{\delta}(P_0, x) = \hat{\delta}(P_0, y)$ è di equivalenza

L'unione di tutti da Σ^*

$\forall x \in \Sigma^* \forall y \in \Sigma^* \forall z \in \Sigma^*$

$xRy \rightarrow xzRyz$

$xRy \rightarrow \hat{\delta}(P_0, x) = \hat{\delta}(P_0, y)$

Prendo z

$\hat{\delta}(P_0, xz) = \hat{\delta}(\hat{\delta}(P_0, x), z) \stackrel{?}{=} \hat{\delta}(P_0, yz) \stackrel{?}{=} \hat{\delta}(\hat{\delta}(P_0, y), z)$

$(\forall z \in \Sigma^*)(xz \in L \leftrightarrow yz \in L)$

Osservazione:

Se $nR_L y$ allora x e y sono entrambe in L o entrambe fuori da $L \rightarrow$ se $z = \varepsilon$ ($x\varepsilon = x \in L \leftrightarrow y\varepsilon = y \in L$)

R_L è di equivalenza



- $nz \in L \leftrightarrow yz \in L$
- $nR_L y \rightsquigarrow \forall z (nz \in L \leftrightarrow yz \in L) \rightarrow (yz \in L \leftrightarrow nz \in L) \rightarrow yR_L x$
- $nR_L y$ e $yR_L u \rightarrow \forall z (nz \in L \leftrightarrow yz \in L \leftrightarrow uz \in L) \rightarrow nR_L u$ R_L è invariante a destra?

Dimostrando per assurdo si può affermare che è vero

TEOREMA:

- L è un linguaggio regolare
- L è UNIONE di classi di equivalenze ridotte da due relazioni di equivalenza di indice finito e invariante a destra
- R_L è di indice finito

ordine di implicazioni: 1) $\rightarrow$ 2) 2) $\rightarrow$ 3) 3) $\rightarrow$ 1)

1) $\rightarrow$ 2)

Come dimostrato prima, L è un'unione di stati $L(q) = \{n \cdot \hat{\delta}(P_0, x) = q\}$

Quindi $L(M) = \bigcup_{q \in F} L(q)$

2) $\rightarrow$ 3)

Idea:

Dimostriamo per ogni $x \in \Sigma^*$:

$$[x]_R \subseteq [x]_{R_L}$$

Se dimostriamo che l'altra classe è più grossolana, allora R_L è più fine di R , e quindi R_L è di indice finito.

Sia $y \in [x]_R$, devo dimostrare che $y \in [x]_{R_L}$

xRy

x è invariante a destra $\rightarrow x \in L \rightarrow y \in L; x \notin L \rightarrow y \notin L$

$$nz \in L \leftrightarrow yz \in L$$

3) $\rightarrow$ 1)

Considero un DFA $M' = (Q', \Sigma, \delta'_0, P'_0, F')$ dove:

$$q'_0 \approx [\varepsilon]$$

$$Q' = \{q'_0, \dots, q'_{0_i}\}$$

Classi della partizione indotta da R_L

$$F' = \{[x] : x \in L\}$$

$$\delta'([x]_{R_L}, a) = [xa]_{R_L}$$

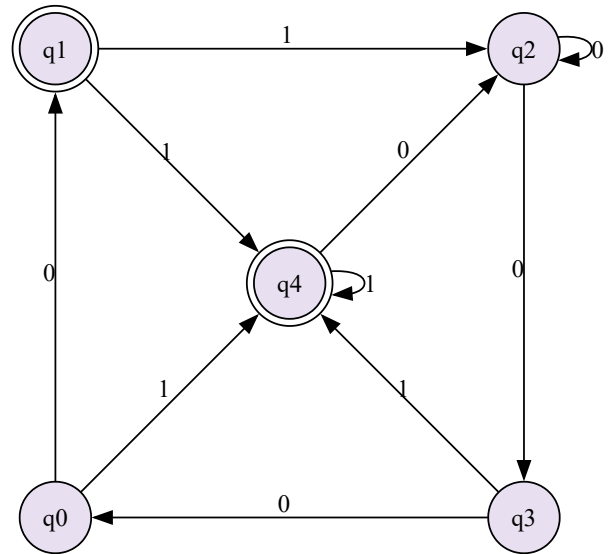
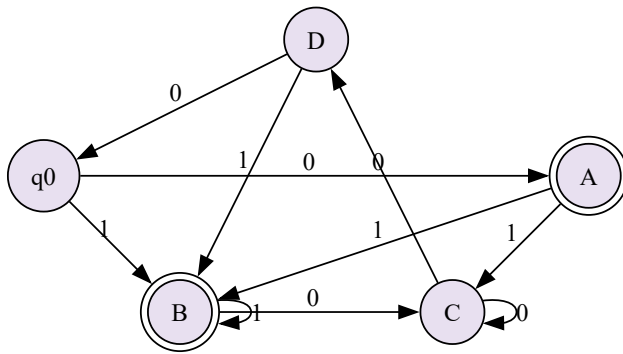
R_L è invariante a destra, quindi $[x_a]_{R_L} = [y_a]_{R_L}$

$$\hat{\delta}'(q'_0, x) \in F' \iff x \in L$$

Mi basta assumere che $\forall x$

$$\text{base: } n = \varepsilon \hat{\delta}'(\hat{\delta}'(P'_0, x), a) = \hat{\delta}'([x]_{R_L}, q) = [xa]_{R_L}$$

$F : Q \rightarrow Q'$ è un isomorfismo se $\forall q, \in Q \forall a \in \Sigma$



Abbiamo q! Modi di mostrare un automa

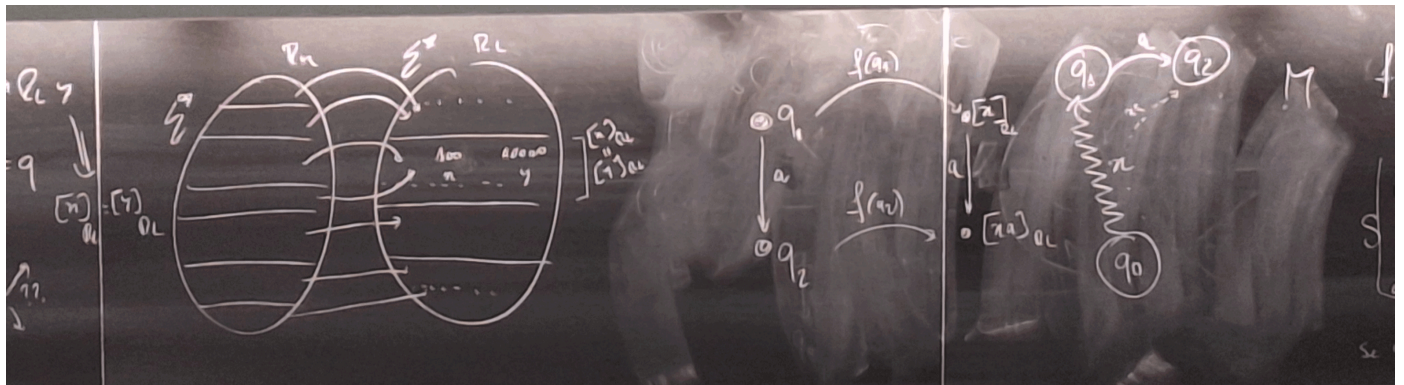
Sia M un DFA che accetta L e che $|Q|=|Q'|$

Voglio dimostrare che M è isomorfo a M'

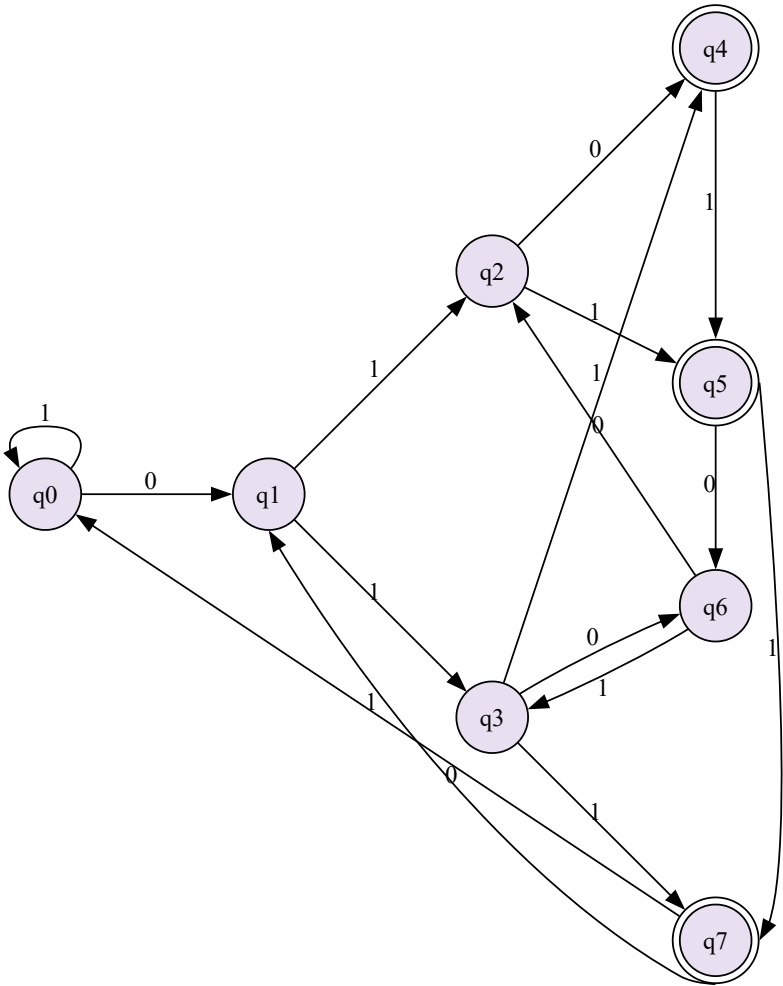
Idea:

Da M non sono niente però so che quando "parti" n in M
se io ho x e y (due stringhe), se:

$$f(q) = [x] \quad \hat{\delta}(P_0, x) = q \quad f(q) = [y] \quad \hat{\delta}(P_0, y) = q$$



M' è l'automata minimo ed è unico a meno di isomorfismo.



in **grossetto** gli split:

B_0	B_1	B_2	B_3	B_4	B_5	B_6	B_7
0,1,2,3	4,5,6,7						
**	,4,2,1						
0,0,0,1	1,1,0,0						
0,1	4,5	2,3	6,7				
1,2	4,6	4,6	2,1				
0,2	1,3	1,3	2,0				
0	4	2	6	1	5	3	7

con un automa meno utile:

(vedi se è opportuno copiare il grafico o no)ù

In ogni passaggio o splitto o no, nel caso peggiore, slitto sempre, avendo un algoritmo quadratico. Al massimo faccio q-1 passaggi

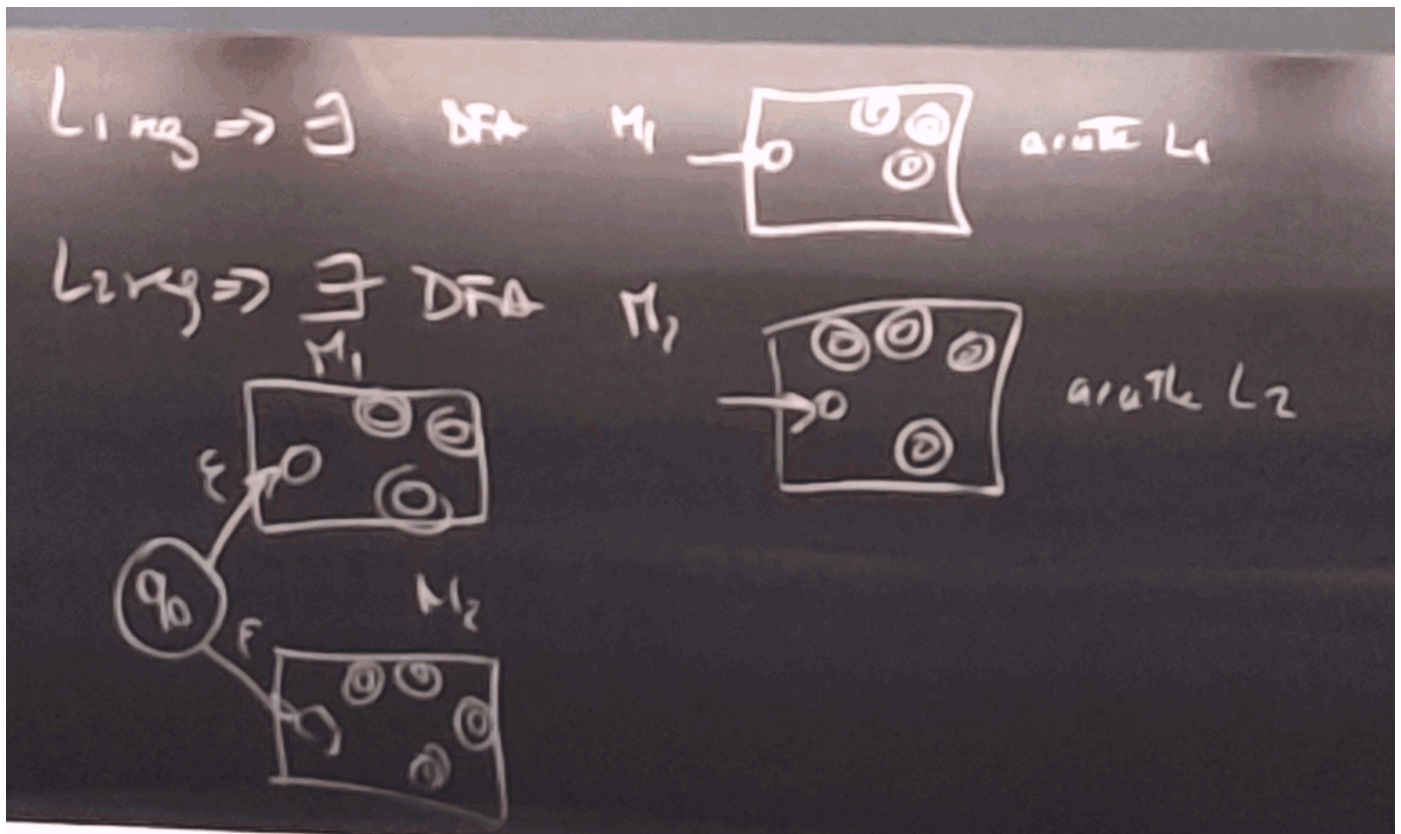


Proprietà di chiusura

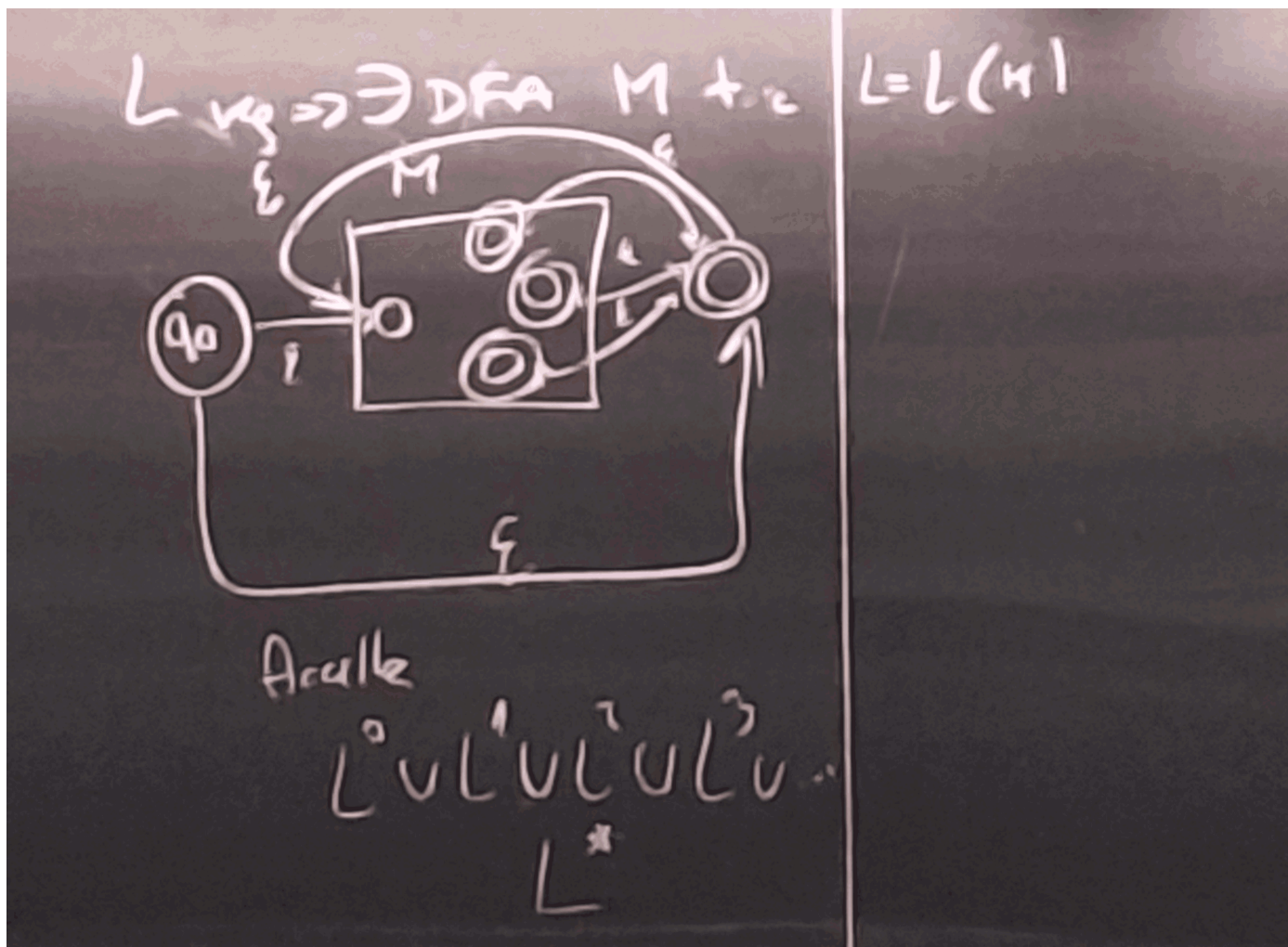
- Se L_1 e L_2 sono linguaggi regolari, allora anche $L_1 \cup L_2$ sono linguaggi regolari.
- L_1, L_2 sono regolari, allora anche $L_1 \cdot L_2$ è regolare
- L è regolare, allora anche L^* è regolare
- L è regolare, allora $\bar{L} \cdot \Sigma^*$ *Lè regolare* è regolare
- L_1 è regolare, allora $L_1 \cap L_2$ è regolare

L_1 è regolare $\Rightarrow \exists$ DFA $M_1 \rightarrow$ Accetta L_A

L_2 è regolare $\Rightarrow \exists$ DFA $M_2 \rightarrow$ Accetta L_B



$$L^0 \cup L^1 \cup L^2 \cup \dots \cup L^*$$



$$L_{1CG} \implies \exists \text{ DFA } M \text{ di } L = L(M)$$

$$A \cup B = \overline{\overline{A} \cap \overline{B}} = \overline{A} \cap \overline{B}$$

Definisco al variare di i gli insiemi:

$$A_i = 0^i 1^i$$

$$B_i = x \in 0, 1^* : |x| = i$$

Per: A_i :

graficone

$$x \in \overline{L} \iff \hat{\delta}(P_0, x) \notin F'$$

Se l'unione di un k linguaggi regolari è regolare, allora anche l'unione di un numero enorme finito di linguaggi regolari è regolare.

Se l'unione di un numero enorme finito di linguaggi regolari è regolare, allora anche l'unione di un numero infinito di linguaggi regolari **NON** è regolare.

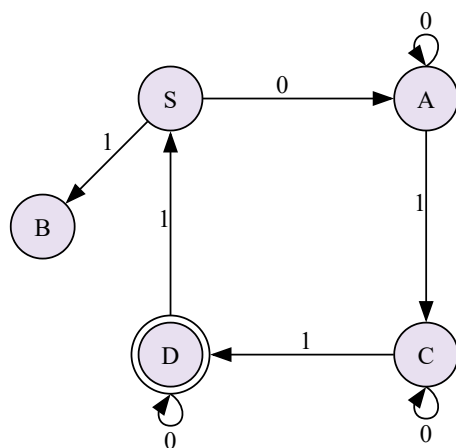
$$\bigcup_{i \geq 0} A_i = 0^n 1^n : n \geq 0 \text{ NON è regolare}$$

$$\bigcup_{i \geq 0} B_i = 0, 1^* \text{ è regolare}$$

Questo perchè se ho un numero infinito di linguaggi regolari, non posso costruire un automa che li accetti tutti, perchè avrei bisogno di un numero infinito di stati e non terminerebbe mai.

$$\begin{cases} S \rightarrow A|1B \\ A \rightarrow 0A|1C \\ C \rightarrow 0C|1D \\ D \rightarrow 0D|1S|\epsilon \end{cases}$$

Inizio con S , da S posso scrivere $S \rightarrow 0\underline{A} \rightarrow 01\underline{C} \rightarrow 010C \rightarrow 0|01D \rightarrow 0101S \rightarrow 0|0|0||B$
 $\rightarrow 0|011 \rightarrow 0|0||0A \rightarrow 0||01L \rightarrow 0|0||0||D \rightarrow 0|0|0||0||$ e quindi otteniamo una stringa.



$$S \xRightarrow{*} uQ \iff \text{nel DFA associato ho } \hat{\delta}(S, u) = Q$$

dato un DFA $\rightarrow \Delta(A, a) = B : A \rightarrow aA$

$$S \rightarrow 0S1|\varepsilon$$

$$S \xRightarrow{0} \varepsilon$$

$$S \xRightarrow{1} 0S1 \xRightarrow{1} 00S1 \xRightarrow{0} 000S1 \xRightarrow{\varepsilon} 000|||$$

Una grammatica libera dal contesto (CF, context free) è una quadrupla $G = (V, T, P, S)$ dove:

- V è un insieme finito di simboli, detti variabili o non terminali
- T è un insieme finito di simboli, detti terminali, con $V \cap T = \emptyset$
- P è un insieme finito di regole di riscrittura, ciascuna della forma $A \rightarrow \alpha$ con $A \in V$ e $\alpha \in (V \cup T)^*$
- $S \in V$ è il simbolo iniziale

copia vecchi appunti da ivona

$$G = (V, T, P, S)$$

$$A \rightarrow \alpha$$

$$L(G) = \{x \in T^* : S \xRightarrow{*} x\}$$

Esempio

$$x : \#(0, x) \neq (1, x)$$



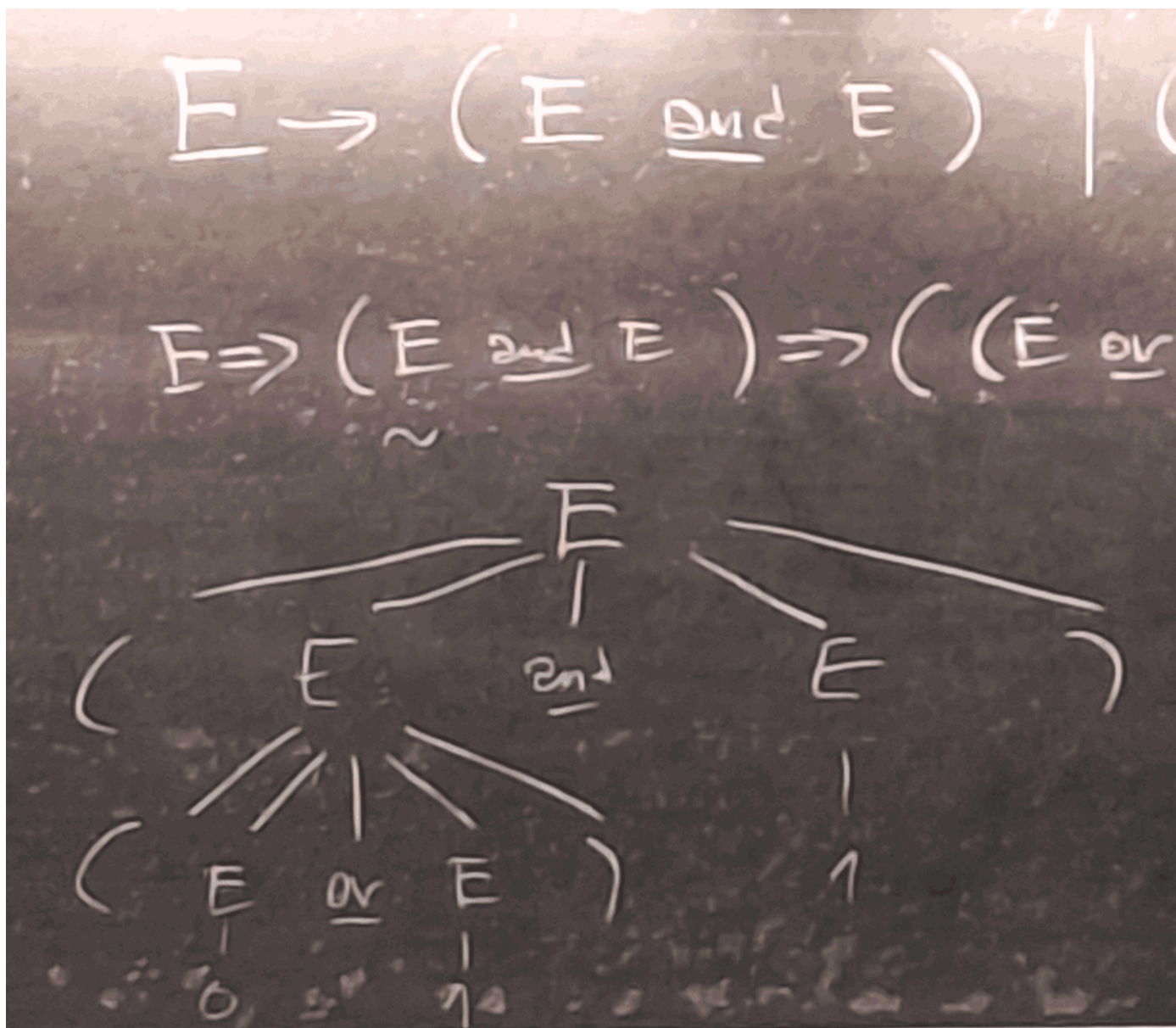
Linguaggi CF



Albero di Derivazione (ParseTree)

$$E \rightarrow (E \cdot E) | (E + E) | (\overline{E}) | 0 | 1$$

$$E \Rightarrow (E \cdot E) \Rightarrow ((E + E) \cdot E) \xrightarrow{3} ((0 + 1) \cdot 1)$$



E dopo aver semplificato e risolto questo albero specifico, si ottiene 1.

Se a ogni passo di derivazione espandiamo un albero, abbiamo una espansione della rappresentazione della derivativa.

Un albero di derivazione è un albero i cui nodi sono etichettati con $V \cup T \cup \varepsilon$

- Se un nodo è etichettato con T , allora è una foglia
- Se un nodo è etichettato con ε , allora è una foglia e **unico** figlio (è l'elemento vuoto, quindi aggiungerne altri non avrebbe senso)
- Se un nodo è etichettato con V (o da S), allora è la radice

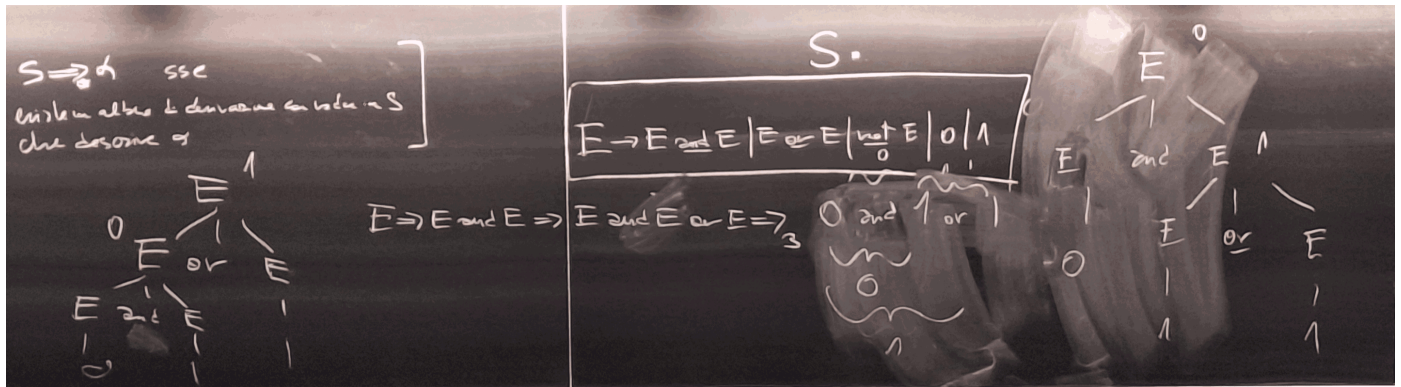
Se un nodo è etichettato con nk , allora esiste in P una produzione $A \in V$ (m è etichettato con A) $A \rightarrow X_1, X_2, \dots, X_n, X_i \in V \cup T \cup \varepsilon$

TEOREMA:

$S \Rightarrow \alpha \iff$ esiste un albero di derivazione con radice in S che descrive α

L'italiano è un linguaggio naturale molto ambiguo, in particolare semanticamente quando costruiamo l'albero di derivazione.

Se ad esempio prendiamo $0 \cdot 1 + 1$, se facessimo prima la moltiplicazione, avremmo un albero di derivazione diverso da quello in cui facciamo prima l'addizione.



Questi due alberi non sono isomorfici, perchè hanno una struttura diversa, e quindi la grammatica è ambigua.

L'Italiano è una lingua ambigua, se dato un testo, si possono costruire più alberi di derivazione, con significati diversi.

Data una grammatica $G = (V, T, P, S)$, vogliamo riscriverla come $G' = (V', T, P', S)$ in una certa forma "normale"

Elaborazione somme inutili $\implies$ Eliminazione delle ϵ produzioni $\implies$ Eliminazione delle produzioni unitarie $A \rightarrow B \rightarrow$ Eliminazione dei simboli inutili $\implies$ Forma normale di Chomsky

Le grammatiche in forma normale di Chomsky ha le regole nella forma:

- $A \rightarrow a$ con $a \in T$
- $A \rightarrow BC$ con $B, C \in V$ e A, B, C non sono simboli iniziali

Le grammatiche in forma normale di Greibach ha le regole nella forma:

- $A \rightarrow aB_1, \dots, B_n$ con $a \in T$ e $B_1, \dots, B_n \in V$

Dato $G, X \in V \cup T$, è utile se compare una denominazione che parte da una stringa di terminali, ove se esiste

$$S \implies \alpha \times \beta \xrightarrow{*} u \in T^*$$

$$\begin{cases} S \rightarrow A|01S|B \\ B \rightarrow 1C|0 \\ C \rightarrow 1CC|1C \\ A \rightarrow 0AD|1 \\ D \rightarrow 1D|1C \\ E \rightarrow 0E|\epsilon \end{cases}$$

$$T^0x \in V : X \rightarrow u \in P, u \in T^* = B, E, A$$

$$T^1x \in V : X \rightarrow \alpha \in P, \alpha \in (T \cup T^0)^* = A, B, E, S$$

$$T^2x \in V : X \rightarrow \alpha \in P, \alpha \in (T \cup T^1)^* = A, B, E, S$$

$$T^{n+1}x \in V : (X \rightarrow \alpha \in P), \alpha \in (T \cup T^n)^*$$

Questo algoritmo termina sempre, perchè o aggiungiamo qualcosa ad ogni passo, o alla prima ripetizione senza aggiunta, vuol dire che abbiamo finito e termina in un numero finito di passi, al massimo $|V|$ passi.

Notiamo che in questo caso **C** e **D** sono inutili, perchè non portano a nessuna stringa di terminali, quindi possiamo eliminarli, quindi:

$$T^0 = x \in V \cup T : (s \rightarrow \alpha \times \beta) \rightarrow P = 0, 1, A, S, B$$

$$T^1 = x \in V \cup T : (s \rightarrow \alpha \times \beta) \in P, A \in T^0 \cup US = 0, 1, A, S, B$$

Ripetizione, quindi mi fermo.

Un esempio di non raggiungibile:

$$\begin{cases} S \rightarrow 0A1 \\ A \rightarrow \epsilon|1A0|B|BC \\ B \rightarrow 1B|\epsilon \\ C \rightarrow 1C|\epsilon \end{cases}$$

analisi

nuovo automa

$$S \implies 0A1 \implies 01$$

$$A \implies 1A0 \implies 0A1 \implies 10$$

$$B \implies 1B \implies 1$$

$$\begin{cases} S \rightarrow 0A1|01 \\ A \rightarrow \epsilon|1A0|B|BC|C \\ B \rightarrow 1B|1 \\ C \rightarrow 1C|1 \end{cases}$$



$A \in V$ è annullabile se c'è $A \rightarrow \varepsilon$ oppure se c'è $A \rightarrow B_1 \dots B_n$ e sono tutti annullabili.

Tolgo tutte le produzioni $A \in \varepsilon$ e per ogni ver annullabile $A \ C \rightarrow \alpha A \beta$ allora $C \rightarrow \alpha \beta$

Supponendo di avere una situazione dove

$S \rightarrow ABCD$	A B C D
$B \rightarrow \varepsilon \dots$	AB AC AD
$C \rightarrow \varepsilon \dots$	BC BD CB
$A \rightarrow \varepsilon \dots$	ABC ACD
$D \rightarrow \varepsilon \dots$	ABD
	BCD

Riduzione unitarie:

Vado a identificare i seguenti di ogni elemento $A \in V$, successivamente taglio le produzioni unitarie e aggiungo tutte le produzioni seguenti. In questo modo elimino tutte le produzioni unitarie.

Le regole sono:

- $A \rightarrow a \in T$ (con a si intende un carattere)
- $A \rightarrow X_1 \dots X_n$ con $X_n, n \geq 2$

Se $n = 2$, ho 4 casi:

- $A \rightarrow BC$ OK
- $A \rightarrow bC$ introduco nuova variabile $N : A \rightarrow Nc \quad N \rightarrow b$
- $A \rightarrow Bc$ introduco nuova variabile $M : A \rightarrow BM \quad M \rightarrow c$
- $A \rightarrow bc$ introduco nuova variabile M e $N : A \rightarrow MN \quad M \rightarrow b \quad N \rightarrow c$



Se G' segue la forma normale di Chomsky, $A \rightarrow a \quad A \in V, a \in T$ e $A \rightarrow BC \quad A, B, C \in V$. Se $\varepsilon \in L(G)$ allora abbiamo $S' \rightarrow \varepsilon$ ma S' non compare a destra di nessuna produzione.

Come sono le denominazioni/alberi in FN di Chomsky?

$$\begin{cases} S' \Rightarrow \varepsilon \\ S' \Rightarrow a \\ S' \Rightarrow AB \Rightarrow aB \Rightarrow ab \\ A \Rightarrow CD \\ S' \Rightarrow AB \Rightarrow CDB \xrightarrow{3} cdb \\ S' \Rightarrow AB \Rightarrow ADB \xrightarrow{1} CDEF \xrightarrow{1} cdef \end{cases}$$

Quando un $A \rightarrow BC$ la shape dell'albero aumenta di 1. $A \rightarrow a \dots$

- 1 $\rightarrow$ 1
- 2 $\rightarrow$ 3
- 3 $\rightarrow$ 5
- 4 $\rightarrow$ 7
- $\vdots \rightarrow \vdots$



Una osservazione importante è che per generare una stringa di lunghezza $n \geq 1$, mi servono $2n - 1$ passi.

" S' $\rightarrow \varepsilon$

S' (1) $\rightarrow A, BA \rightarrow aB \rightarrow b$

S' (2) $\rightarrow A, BA \rightarrow C, DC \rightarrow cD \rightarrow dB \rightarrow b_0$

S' (3) $\rightarrow A, BA \rightarrow C, DC \rightarrow cD \rightarrow dB \rightarrow E, FE \rightarrow eF \rightarrow f$

S' (4) $\rightarrow A, BA \rightarrow C, DC \rightarrow E, FE \rightarrow eF \rightarrow fB \rightarrow b''$

Possiamo notare che S' (3) e S' (4) Sono due alberi diversi, il primo con altezza 3, il secondo con altezza 4.

Alberi di derivazione di h generano stringhe al massimo di numero di foglie $n = 2^{h-1}$. La stringa più corta di un albero di altezza h è una stringa di lunghezza h

Se prendiamo un albero di derivazione le cui foglie sono tutte etichettate in T , se l'albero ha altezza h , allora la stringa denotata ha lunghezza $h \leq n \leq 2^{h-1}$

Fatto:

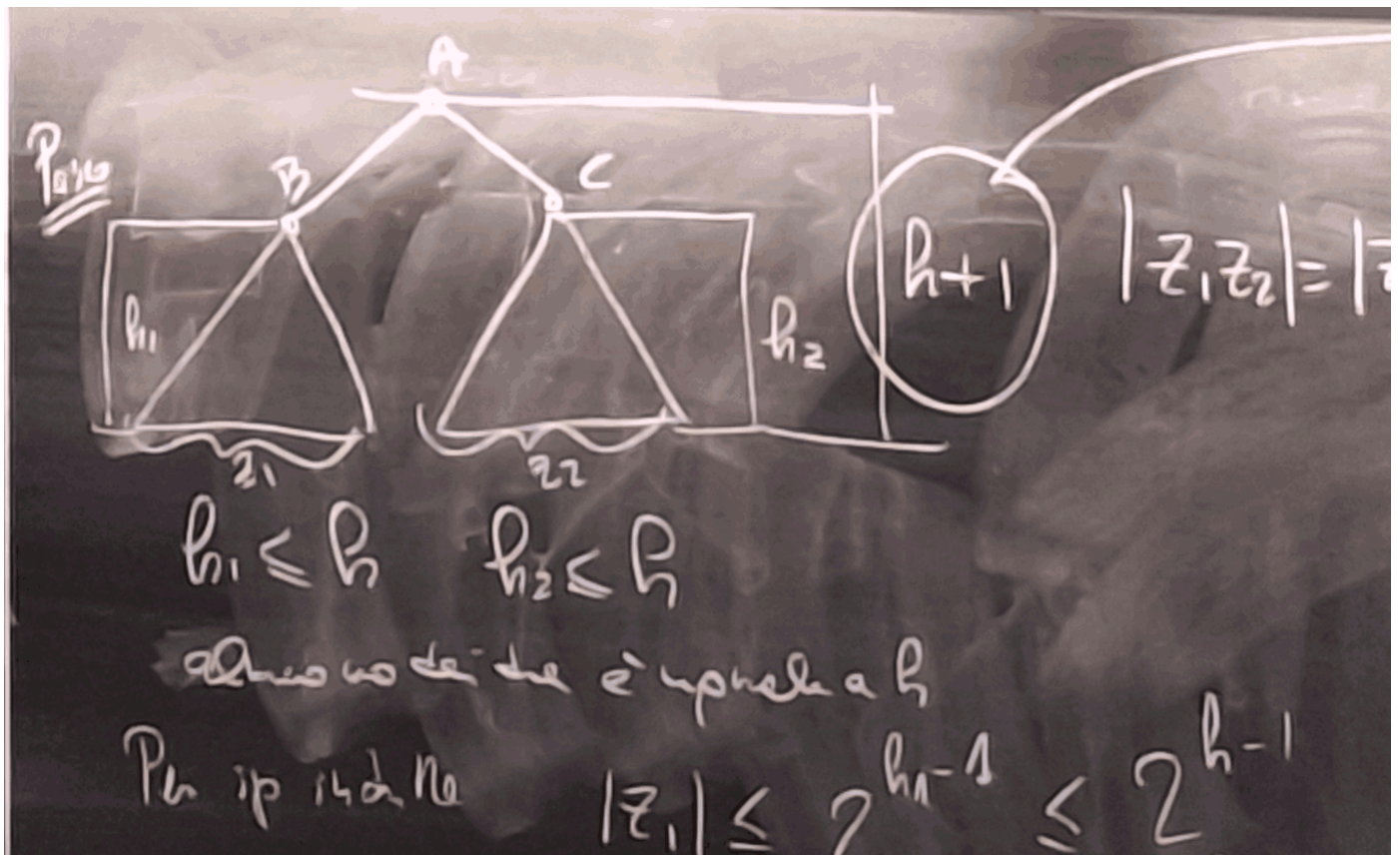
Se l'albero di derivazione, con foglie tutte etichettate in T , ha altezza h , allora la stringa denotata ha lunghezza $n \leq 2^{h-1}$

Dimostrazione:

induttiva su $h \geq 1$

base: $h = 1, |a| = 1 = 2^{1-1} = 2^0$

passo induttivo:



$h_1 \leq h \quad h_2 \leq h$, allora

Per ipotesi induttiva:

- $|z_1| \leq 2^{h_1-1} \leq 2^{h-1}$
- $|z_2| \leq 2^{h_2-1} \leq 2^{h-1}$

$$|z_1 z_2| = |z_1| + |z_2| \leq 2 \cdot 2^{h-1} = 2^h$$

$V = S, A, B, C$

Qualunque sia l'albero generato, sono sicuro che almeno una lettera sarà ripetuta.

Posso quindi modificare il mio albero per trovare altre stringhe, quindi posso avere infinite stringhe con un albero di generazione simile.



Albero iniziale	albero rigenerato
'''	
S -> A, B	
A -> a	
B -> C, A2	
A2 -> B2, C	
B2 -> b	
C -> c	
'''	'''
S -> A, B	
B -> C, A2	
A2 -> B2, C	
B2 -> C2, A3	
C -> c	
A3 -> B3, C2	
B3 -> b	
C2 -> c	
'''	



Pumping Lemma per i linguaggi CF

Sia L , CF, allora esiste un n che t.c. Se $z \in L$, $|z| \geq n$, allora esistono stringhe u, v, w, x, y tali che:

- $z = uvwxy$
- $|vwx| \leq n$
- $|vx| > 0$ tali che per ogni $i \geq 0$, $uv^iwx^iy \in L$

"" $S \rightarrow A, B B \rightarrow C, A^2 A^2 \rightarrow B^2, C B^2 \rightarrow C^2, A^3 C \rightarrow c A^3 \rightarrow B^3, C^2 B^3 \rightarrow b C^2 \rightarrow d$ "" **Proprietà:**

Se la stringa denotata ha lunghezza $n > 2^{h-1}$, allora l'albero di derivazione ha altezza h

Dimostrazione:

Sia L un CF $\implies \exists G = (V, T, P, S)$ in FN di Chomsky che lo genera

Sia $h = |V|$

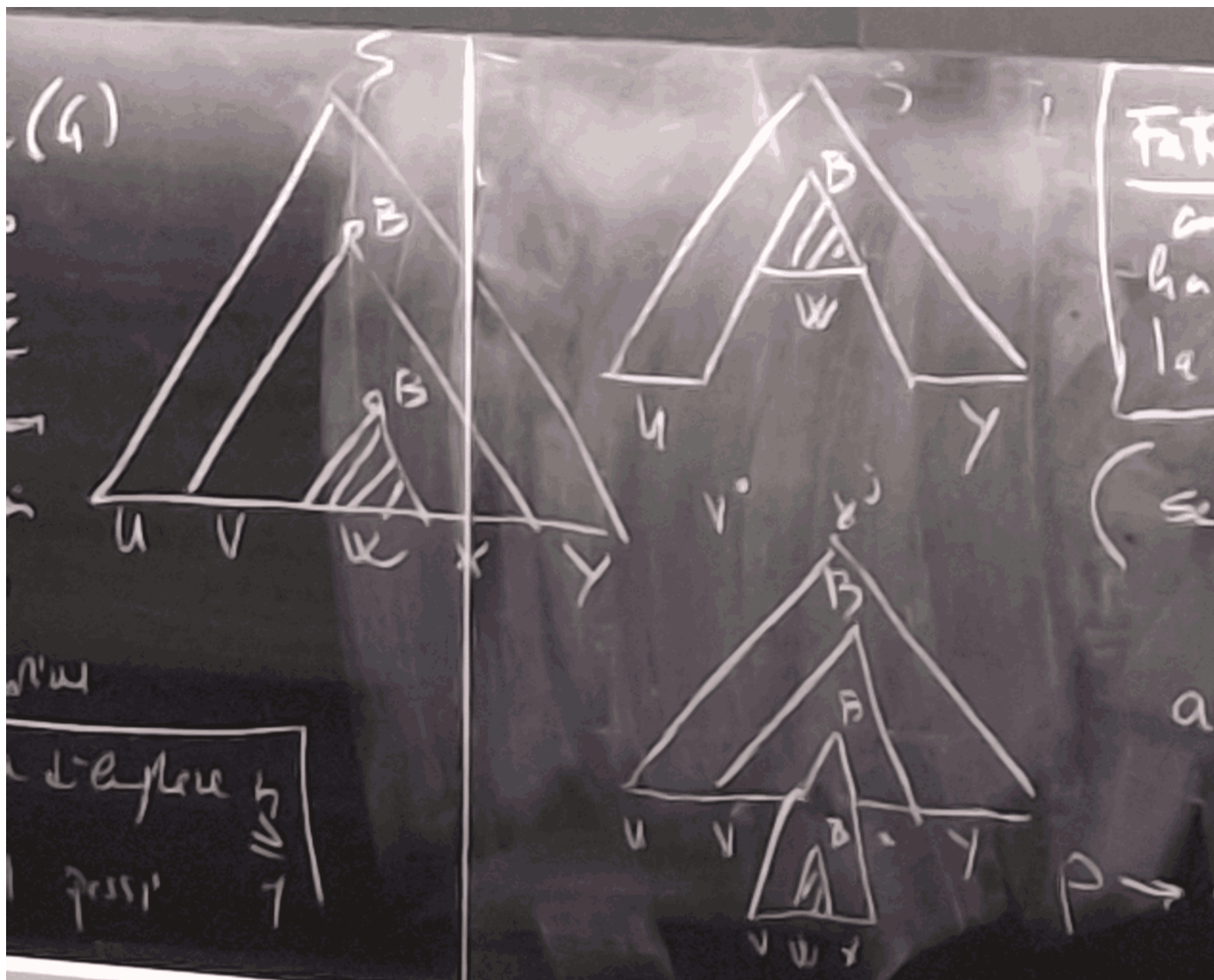
Sia $n = 2^h$

Sia $z \in L$ con $|z| \geq n$

Per la proprietà precedente, l'albero di derivazione per z ha altezza maggiore di h (almeno $h + 1$)

Albero

$$|vwx| \stackrel{?}{\leq} 2^h = n$$



Dato L , sia $n \in \mathbb{N}$ arbitrario, se so che $z \in L$ e $|z| \geq n$ tale che per ogni $z = uvwxy$, $|vwx| \leq n$, $|vx| > 0$ e trovo i per cui $uv^iwx^iy \notin L$, allora L non è CF.



$$L = 0^n 1^n 2^n : n \in \mathbb{N}$$

Dato $n \in \mathbb{N}$, sia $z = 0^n 1^n 2^n$

Ora partiziono arbitrariamente z in tutti i modi possibili $z = uvwxy$ rispettando $|vwx| \leq n$ e $|vx| > 0$

$$z = 00\dots 0011\dots 1122\dots 22 \quad u|vwx|y \quad u \quad |vwx|y \quad u \quad vwx|y \quad u|vwx|y \quad u|vwx|y$$

$$vwx: 0^a 0^b 0^c$$

$$vx = 0^{a+c} \text{ con } a+c > 0$$

$$uv^0wx^0y = 0^{b-a-c}1^n2^n \dots$$

Se io pompassi dei numeri, in questo caso cambierebbe proprio la struttura della stringa, quindi L **non** è CF.

Esercizio:

$$L = 0^n 1^n 2^{m+n} : n, m \in \mathbb{N}$$

Inizio parte dove non ho copiato bene e non ho capito assolutamente nulla

Partiziono z in tutti i modi possibili, e vedo che se pompo, non cambia la struttura della stringa, quindi L è CF:

$$\text{caso 1: } uv^0wx^0y = 0^{b-a-c}1^n2^{n+n} \implies n-a-c+n < n+n \implies \notin L$$

...

Caso 3: tolgo degli 1 e dei 2

Caso 4: tolgo degli 1 e dei 2, ma sostituisco i numeri con 0

fine parte dove non ho copiato bene e non ho capito assolutamente nulla

Riassuntino:

Regolari	C.F	Regolari	C.F
Chiusura	Chiusura	Decidibilità	Decidibilità
$\cap$ SI	$\cap$ NO	$X \in L(M)$ OK	$x \in L(G)$ OK
$\cup$ SI	$\cup$ SI	$L(M) \neq \emptyset$ OK	$L(G) \neq \emptyset$ OK
$\ddot{::}$ SI	$\ddot{::}$ NO	$L(M)$ è finito/infinito OK	$L(G)$ è finito/infinito OK
$\cdot$ SI	$\cdot$ SI	$L(M_1) = L(M_2)$ OK	$L(G_1) = L(G_2)$ NO
$\star$ SI	$\star$ SI		
$\bigcup_{i \geq 1} \bigcap_{i \geq 1}$ NO	$\bigcup_{i \geq 1} \bigcap_{i \geq 1}$ NO		



Unione di linguaggi CF

I CF sono chiusi per unione, ma non per intersezione.

$\cup$ binaria

$$G_1 = (V_1, T_1, P_1, S_1)$$

$$G_2 = (V_2, T_2, P_2, S_2)$$

Vogliamo dimostrare che $L = L(G_1) \cup L(G_2)$ è un linguaggio context-free.

Se Le Unisco	G_1	G_2
$S \rightarrow S0 1S1 \varepsilon$	$S \rightarrow 0S0 \varepsilon$	$S \rightarrow 1S1 \varepsilon$
$L(G) = xx^n : x \in 0, 1^* \quad L(G_1) = 0^n : n \geq 0 \quad L(G_2) = 1^n : n \geq 1$		
$S_2 \rightarrow 0S_20 \varepsilon$	$\neq L(G_1) \cup L(G_2)$	

Le due grammatiche o vanno prima in S_1 e poi fanno tutto il resto, o vanno prima in S_2 e poi fanno tutto il resto, quindi:

$$V_1 \cap V_2 = \emptyset$$

Per concatenazione:

$$L = L(G_1) \cdot L(G_2) = uv : u \in L(G_1), v \in L(G_2)$$

$$P_1 \cup P_2 \cup S \rightarrow S_1 \cdot S_2$$

Quindi sono chiusi per concatenazione

Per $*$:

$$G^* = (V_1, T_1, P_1, S)$$

$$G' = (V \cup S', T, P \cup S' \rightarrow \varepsilon | SS', S')$$

Per intersezione:

$$S \rightarrow 0S2|A$$

$$A \rightarrow 1A1|\varepsilon$$

$$L(G_1) = 0^m 1^n 2^m : m, n \geq 0$$

$$S \rightarrow AB$$

$$A \rightarrow 0A|\varepsilon$$

$$B \rightarrow 1B2|\varepsilon$$

$$L(G_2) = 0^m 1^n 2^m : n, m \geq 0$$

$$L(G) = L(G_1) \cap L(G_2) = 0^m 1^n 2^m : m, n \geq 0 \text{ (non è CF)}$$

Quindi possiamo dire che l'unione di due linguaggi CF è un linguaggio CF, ma l'intersezione di due linguaggi CF **NON** è un linguaggio CF.

Per De Morgan:

Se fossero chiusi per intersezione, allora avremmo che,

$$\begin{aligned} L &= L(G_1) \cap L(G_2) = \overline{\overline{L(G_1)} \cup \overline{L(G_2)}} \\ &= \overline{L(G_1)} \cup \overline{L(G_2)} \end{aligned}$$

Se prendessi un regolare e un CF e li unissi

Probabilmente l'intersezione è un CF

Decidibilità



Dato un DFA M è decidibile se stabile se $x \in L(M)$?

Calcolo $\hat{\delta}(p_0, x)$?

Date $G = (U, T, P, S)$ e $x \in T^*$, so stabilire se $x \in L(G)$?

$S \rightarrow \alpha_1 | \alpha_2 | \alpha_3 \quad S \rightarrow 0AB1 | \varepsilon$

$A \rightarrow 0A1 | 0B1 | \varepsilon$

$B \rightarrow 1S0 | 1$

Se mettessimo assieme un algoritmo di brute force per trovare la soluzione, la prima soluzione che trovo avrebbe 3 possibilità:

- $00A|B| \neq 00||$
- $00B|B| \neq 00||$
- $0B| \neq 00||$ Ma non sono valide, quindi continuo a cercare (conviene espandere in orizzontale la ricerca rispetto al verticale)

Se la trovo in un determinato punto, terminiamo, ma se non la trovassi, **quando mi fermo?**

Secondo Chomsky, $S \implies a_1 a_2 \dots a_n$ necessito per forza brutta con $2n$ (dove n è la lunghezza della stringa x) passi al massimo, se lo supero mi fermo con un "NO". Il numero di passi cresce esponenzialmente. Nonostante l'algoritmo non sia efficiente, è comunque decidibile se $x \in L(G)$

Nel corso vedremo algoritmi migliori di questo, ma per ora è sufficiente per la dimostrazione.

$L(M) \neq \emptyset$

$L(M) \neq \emptyset \iff \exists x \in T^* \quad |x| \leq |Q|$

Le proviamo tutte (anche se esponenziale) e lo determiniamo

CF

$L(G) \neq \emptyset \iff$ "S" è utile, c'è ne accorgiamo eliminando i simboli inutili

$L(M)$ è finito sse non c'è nessuna stringa in $L(M)$ di lunghezza che n è $2n$ tale che $n = |Q|$

$L(G)$ è finito o infinito?

Se io prendessi l'automa scritto un due dimostrazioni fa, se avessimo un ciclo, allora sarebbe infinito, altrimenti sarebbe finito.

(disegnati un esempio di linguaggio infinito e non finito)

Se io avessi:

DFA M_1 e DFA M_2

$L(M_1) = L(M_2)$

Abbiamo due opzioni:

- teorica:
 - $A = N$ sse $(A|B) \cup (B|A') = \emptyset$ e quindi costruisco DFA $n \ L(M_1)/L(M_2)$ e $L(M_2)/L(M_1)$ e poi faccio la loro unione e vedo se è vuota o no
- non teorica:
 - Abbiamo M_1 e M_2 che sono minimi, dobbiamo vedere se sono isomorfi

$\{0^m 1^n 2^n 3^m : m, n \geq 0\}$

$\{0^m 1^n 2^m 3^n : m, n \geq 0\}$

$\{0^m 1^n 2^{m \geq n} : m, n \geq 0\}$

$$M = n = \begin{cases} m - n & \text{se } m \geq n \\ 0 & \text{altrimenti} \end{cases}$$

Requisiti per essere un algoritmo:

1. deve avere lunghezza finita
2. Esiste un agente di calcolo che porta avanti il calcolo eseguendo le istruzioni dell'algoritmo
3. L'agente di calcolo ha a disposizione una memoria dove venono immagazzinati i risultati intermedi del calcolo
4. Il calcolo avviene per passi discreti
5. Il calcolo non è probabilistico (quindi deterministico)
6. Non deve esserci alcun limite finito alla lunghezza dei dati di ingresso
7. Non deve esserci alcun limite alla quantità di memoria disponibile
8. Deve esserci un limite finito al numero delle istruzioni eseguibili dal dispositivo
9. Deve esserci un limite finito alla complessità delle istruzioni eseguibili dal dispositivo



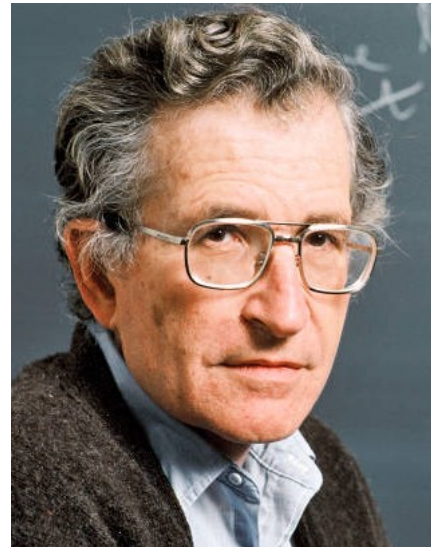
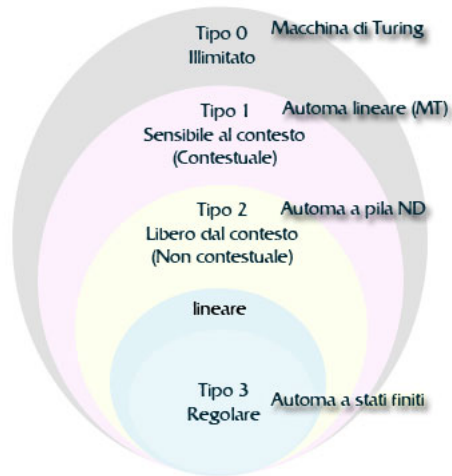
Ultima modifica: 2026-05-24 20:32:34

[Qui andrà il tuo Logo]

10. Sono ammesse esecuzioni con un numero di passi finito ma illimitato



Gerarchia di Chomsky



$$\left(\mathbb{P}(\Sigma^*) \left(\cdot 0^n 1^n 2^n : n \geq 0 \left(C.F. \cdot 0^n 1^n : n \geq 0 (icg) \right) \right) \right)$$

- **Tipo 1:** o monotona
 - $G = (V, T, P, S)$
 - V insieme finito di variabili
 - T insieme finito di simboli terminali
 - $S \in V$ simbolo iniziale
 - P insieme di produzioni del tipo $\alpha \rightarrow \beta, \alpha, \beta \in (V \cup T)^*$



Tipo 1

$$AB \rightarrow XC \implies ab \rightarrow XC$$

Osservazione:

Se $L \in C.F.$, allora L è generato anche da G di tipo 1.

$\exists G_1$ in FN di Chomsky che la genera (con al limite $S \rightarrow \varepsilon$, una S che non occorre α)

$$S \rightarrow 0S12|\varepsilon$$

Questo genererebbe:

$$S \rightarrow 0S12 \rightarrow 00S1212 \rightarrow 000S121212 \xrightarrow{S \rightarrow \varepsilon} 000121212$$

Provando con più S, ottengo per cancellare la ε :

$$\begin{aligned} S &\rightarrow \cancel{0S12}|\varepsilon \\ S' &\rightarrow \varepsilon|0S12 \\ S &\rightarrow 0S12|0|2 \end{aligned}$$

$$S' \rightarrow 0S12 \rightarrow 001212 \implies n = 2$$

$$S' \rightarrow 0S12 \rightarrow 00S1212 \rightarrow 00|2|2|2 \implies n = 3$$

Quindi:

$$A \rightarrow 1? B \rightarrow ?$$

$S' \rightarrow 0SAB \rightarrow 00SABAB \rightarrow 0000ABABAB$ in questa ultima iterazione, per la prima volta 0 è affiancato ad A, quindi $0A \rightarrow 01$

Posso anche notare che posso scambiare A con B, quindi $BA \rightarrow AB$

$$\begin{aligned} S' &\rightarrow 0SAB \rightarrow 00SABAAB \rightarrow 000BA BAB \\ &\rightarrow 000 AABBAB \rightarrow 000 AABABB \rightarrow 000 AAA BBB \\ &\rightarrow 0001AABBB \xrightarrow{2} 000||BBB \rightarrow 000||22B \xrightarrow{2} 000||222 \end{aligned}$$

Quindi:

$$1A \rightarrow 11$$

$$0A \rightarrow 01$$

$$BA \rightarrow AB$$

$$1B \rightarrow 12?$$

$$2B \rightarrow 22?$$

Se volessimo provare in un altro modo:

$$S' \rightarrow 0SAB \rightarrow 00SABAAB \rightarrow 000BA BAB \Rightarrow 001BABAB \rightarrow 00012A'BAB$$

Ma questo mi blocca la computazione, nonostante abbiamo già trovato in precedenza la stringa corretta (e quindi la grammatica corretta), quindi l'insieme di regole con 2A a sinistra bloccherebbe il completamento.

$$S \xRightarrow{2} O^m(AB)^m \xRightarrow{*} O^m A^m B^m \xRightarrow{2m} O^m 1^m 2^m$$

Se io devo generare da $S \rightarrow \alpha_1 \rightarrow \alpha_2 \rightarrow \dots \rightarrow \alpha_n = n$

$$x \in L(G) \iff \exists n, \in \alpha_1, \alpha_2, \dots, \alpha_n$$

Queste stringhe che mi portano ad n, sono di cardinalità strettamente crescente.

Dimostrando

Costruisco un grafo

nodici	multiple shape di VUT di lunghezza $\leq n$
archi	sono un numero esponenziale, la cui base è V. $n \in L(G) \iff$ c'è un cammino da S a n

Sia $f: \mathbb{N}^n \rightarrow \mathbb{N}$

f è definito da G se e solo se:

$$L(G) = 1^{x_1} 01^{x_2} \dots 01^{x_n} 0^{f(x_1-x_n)} : x_1, x_2, \dots, x_n \in \mathbb{N}$$



$$fN^2 \rightarrow N \approx (0, 0, 0), (0, 1, 1), (1, 0, 1), (1, 1, 2), \dots$$

zero: $\text{zero}(x) = 0 \forall x \in N$

qualsiasi sequenza di input che fornisco, l'output è sempre 0

Successore: $\text{succ}(x) = x + 1 \forall x \in N$

{01, 1011, 1011, 101011, 1010111, \dots} (nota dove quello prima del primo 0 è l'input, mentre quello dopo è l'output - e lo zero non lo conti)

questo linguaggio è regolare? **NO**

dimostro che se dato n arbitrario sia $z = 1^n 0 1^{m+1} \in$ successore mostro per ogni modo di partizionare z in uvw con $|uv| \leq n$ e $|v| > 0$, che $uv^i w \notin$ successore per qualche i e quindi non è regolare (è context-free)

$S \rightarrow 01|1S1$

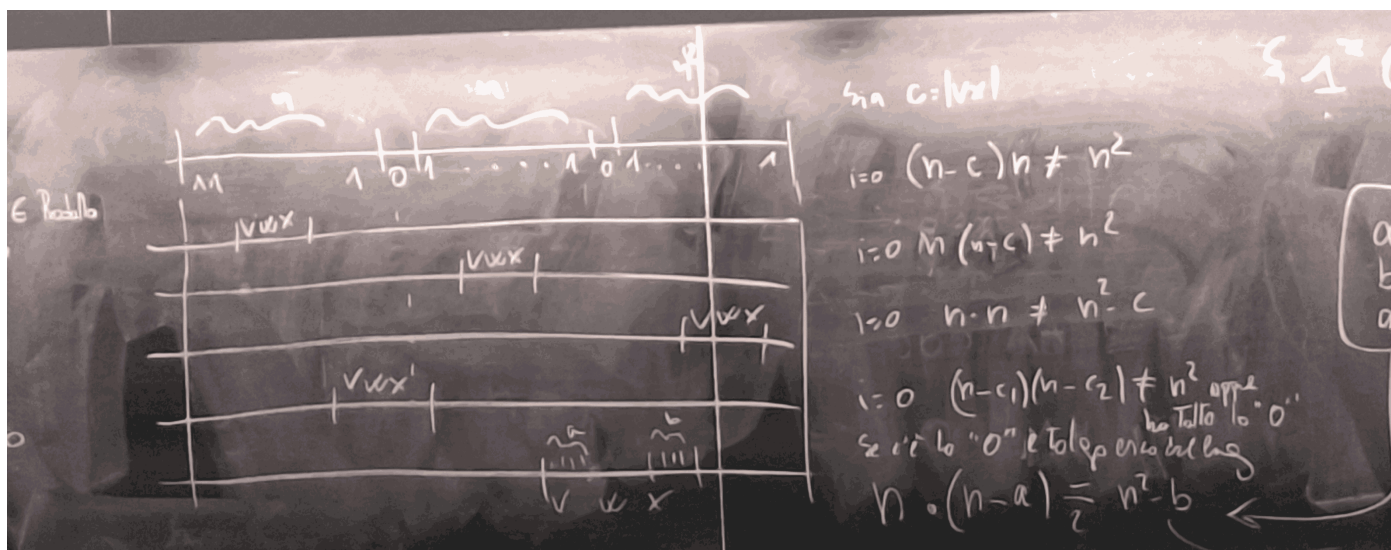
Che cosa abbiamo nei regolari?

Costante **k**

somma $\text{somma}(x, y) = x + y \quad 1^x 0 1^y 0 1^{x+y} : x, y \in N$

prodotto $\text{prodotto}(x, y) = x \cdot y \quad 1^x 0 1^y 0 1^{x \cdot y} : x, y \in N$

Dato n arbitrario, sia $z = 1^n 0 1^n 0 1^{n^2} \in$ prodotto mostro che per ogni modo di posizionare z in uvw con $|uv| \leq n$ e $|v| > 0$, che $uv^i w \notin$ prodotto per qualche i e quindi non è regolare (è context-free)



Quindi il prodotto è context-free, ma non è regolare.

$S \rightarrow X0Y0 \quad U0 \rightarrow 0V$

$Y \rightarrow 1Y|\epsilon \quad U1 \rightarrow 1U$

$X \rightarrow 1UX|\epsilon \quad V1 \rightarrow 12V$

$V0 \rightarrow 0$

Esempio:

$AB \rightarrow BA$ non è nella forma $\alpha_1 A \alpha_2 \rightarrow \alpha_1 B \alpha_2$, ma possiamo trasformarla con un algoritmo in 3 passi (con $m \leq n$):

1. se $n = 1$, allora siamo apposto, se $n > m$, possiamo "accorciare" il lato destro, introducendo una nuova variabile X sostituendo i valori in eccesso a destra con X
2. se $m = n > 2$, aggiungiamo delle variabili $X_1, X_2, \dots, X_{m-2}$:
 - $A_1 A_2 A_3 A_4 A_5 \rightarrow B_1 B_2 B_3 B_4 B_5$ diventa:
 - $A_1 A_2 \rightarrow B_1 X_1$
 - $X_1 A_3 \rightarrow B_2 X_2$
 - $X_2 A_4 \rightarrow B_3 X_3$
 - $X_3 A_5 \rightarrow B_4 B_5$
3. se $m = n = 2$, allora $A_1 A_2 \rightarrow B_1 B_2$ riscrivo le produzioni come:
 - $A_1 A_2 \rightarrow X_1 A_2$
 - $X_1 A_2 \rightarrow X_1 Y_1$
 - $X_1 Y_1 \rightarrow B_1 Y_1$
 - $B_1 Y_1 \rightarrow B_1 B_2$



Tipo 0

Sono grammatiche che non hanno alcun vincolo.

Un MdT è un dispositivo di calcolo rappresentabile con un nastro di lunghezza **illimitata** nel quale vengono immagazzinati i dati o sequenze di simboli di calcolo, uno di questi simboli è \$ che rappresenta l'assenza di simboli.

Il dispositivo ha una testina di lettura/scrittura che si muove lungo il nastro, e può leggere o scrivere simboli uno alla volta.

Al programma ci si accede con una testina, che può stare in un finito modo di stati.

Ma quando si ferma?

Macchine di Turing

È un DFA che ha una matrice di transizione che può essere parziale (per qualche coppia simbolo, potrebbe non essere definito, nella macchina di Turing è presente nella terminazione).

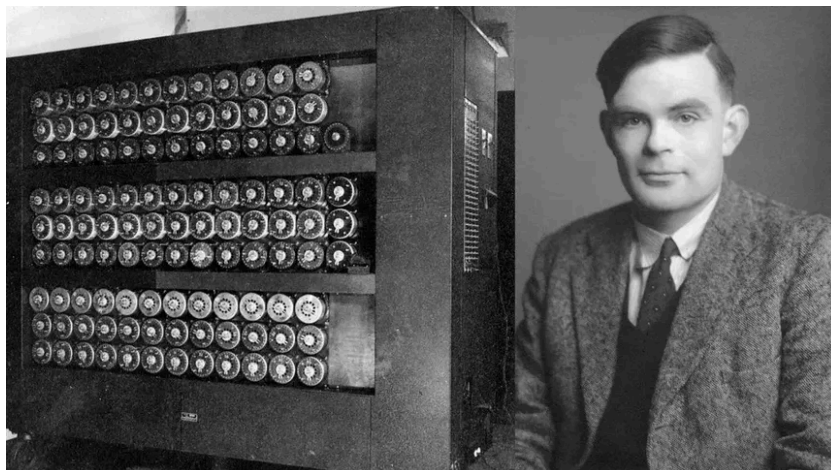
Ha solo 2 istruzioni (3 se contiamo la terminazione):

- scrivi un simbolo (sul nastro)
- spostati a sinistra o a destra (la testina)

Definizione formale:

Una macchina di Turing deterministica è una quadrupla MdT $M = (Q, \Sigma, P, q_0)$ dove:

- Q è un insieme **finito** di stati
- $q_0 \in Q$ è lo stato iniziale
- Σ è un alfabeto che contiene almeno due simboli
- P è un programma composto da quintuple (q, s, q', s', x)
 - $q, q' \in Q$
 - $s, s' \in \Sigma$
 - $x \in L, R$
 - $\implies \forall q, s$ esiste al più una quintupla che inizia con q, s



nota: nella macchina di Turing di solito si usa il simbolo \$ per rappresentare l'assenza di simboli, tuttavia a LaTeX (come lo ho configurato io) non gli piace per niente, quindi userò il simbolo £, e a chi da fastidio è liberissimo di rifarmi i miei appunti XD

Quindi è una funzione deterministica parziale S che $Q \times S \rightarrow Q \times S \times L, R$ e quindi ci possono essere casi per cui $\delta(q, s)$ non è definita

Nel caso volessimo una macchina di Turing che fa delle somme in binario:

$[\underline{\mathcal{L}}, 1, 0, 0, 1, 1, 1, \mathcal{L}] \rightarrow [\underline{\mathcal{L}}, 1, 0, 1, 0, 0, 0, \mathcal{L}]$ (gli elementi sottolineati rappresentano la posizione della testina)

Qualora avessimo un caso particolare (tutti i valori sono 1), verrebbe aggiunto un ulteriore elemento a sinistra, che per definizione dobbiamo assumere che esista e pronto alla scrittura.

Se invece volessimo verificare se una stringa è palindroma:

$[\underline{\mathcal{L}}, 1, 0, 0, 1, 0, 0, 1, \mathcal{L}] \rightarrow YES$

$[\underline{\mathcal{L}}, 1, 0, 0, \mathcal{L}] \rightarrow NO$

La macchina parte da q_0 per poi andare in q_1 il quale valore viene memorizzato nello stato, per poi procedere con q_2 , ricordandosi il valore di q_1 e così via, fino alla fine della stringa.

Computazione:

Punto base è la descrizione istanea ID

Ci interessa la parte prima dei *blank* (quindi prima del £)

La stringa tutta a sinistra vuota possiamo vederla come ε , la stringa di input come x , e la stringa a destra come ε

Se iniziasse la computazione, spegnersi la macchina, e la riaccendessi, anche se tenesse conto di qualche *blank* in più, non cambierebbe nulla.

$ID = (q, v, s, w)$ dove:

- $q \in Q$
- $v, w \in \Sigma^*$



- $s \in \Sigma$ Notiamo come u e w siano più interessanti, e spazino in un insieme infinito, e qui è dove si distingue da un DFA, che invece ha un insieme di stati finito.

Il successore di $ID \vdash \subseteq ID_s \times ID_s$	
1) $(q, s, q', s', L) \in P$	$(q, \epsilon, s, w) \vdash (q', \epsilon, \mathcal{L}, s'w)$ $(q, va, s, w) \vdash (q', v, a, \mathcal{L}'w)$
2) $(q, s, q', s', R) \in P$	$(q, v, s, \epsilon) \vdash (q', vs', \mathcal{L}, \epsilon)$ $(q, v, s, aw) \vdash (q', vs', a, w)$

Complessi è una specie $\alpha_0, \alpha_1, \dots, \alpha_n$ di ID tali che:

- $\alpha_0 = (q_0, v, s, w)$
- $\alpha_i \vdash \alpha_{i+1} \forall i$ Una computazione detta deterministica se $\alpha_n = (\tilde{q}, \tilde{v}, \tilde{s}, \tilde{w})$ e in P non c'è una quintupla che inizia con $\tilde{q}, \tilde{s}$ ($\delta(\tilde{p}, \tilde{s}) \uparrow$)

Il calcolatore (che noi usiamo) di fatto esegue solo somme tra naturali, ma non ha una memoria infinita e quello che terrà in memoria, nonostante quanto possa essere grande, sarà sempre un numero naturale finito.

Problema: studiare "calcolabilità" della funzione: $f : \mathbb{N}^n \rightarrow \mathbb{N}$ eventualmente localmente indefinite (o parziali)

Definizione di funzione Turing-calcolabile:

$f : \mathbb{N}^n \rightarrow \mathbb{N}$ (parziali o totali) è Turing-calcolabile se esiste MdT M tale che $\forall (x_1, \dots, x_n) \in \mathbb{N}^n$, ponendo

$[\mathcal{L}, x_1, x_2, \dots, x_n, \mathcal{L}]$

1. se $f(x_1, \dots, x_n)$ è definita, la MdT esegue computazione che termina
2. se $f(x_1, \dots, x_n)$ è indefinita, la MdT esegue una computazione che non termina

Rappresentazione unaria:

0 = 0

1 = 00

2 = 000

n = 0...n + 1...0

Inserire uno 0 indica la fine dell'input, motivo per cui abbiamo 0 per rappresentare lo 0.

zero $zero(n) = 0 \forall n \in \mathbb{N}$

$\mathcal{L}$	0000	$\mathcal{L}$
$\uparrow q_0$		

diventa:

$\mathcal{L}$	00	$\mathcal{L}$
$\uparrow q_0$		

Una macchina di turing intelligente neanche legge il nastro, e al primo $\mathcal{L}$ assume di averlo completamente pulito. E quindi il problema di sopra è un Turing-calcolabile.

$\mathcal{L}$	0
q_0	$q_1 \mathcal{L} R$
q_1	$q_2 \mathcal{L} R \quad q_1 0 R$
q_2	$q_3 0 R \quad q_3 0 R$
q_3	$q_4 \mathcal{L} L \quad q_4 \mathcal{L} L$
q_4	$\diagup \quad q_4 0 L$

Ma se avessimo un nastro "sporco", con dei valori già scritti sopra, e voglia seguire lo stesso un algoritmo, volendo mantenere i valori esistenti?

$[\mathcal{L}, 0, 0, 0, 0, \mathcal{L}] \rightarrow [\mathcal{L}, 0, 0, 0, 0, 0, \mathcal{L}]$

nel caso base (il nastro è pulito):



£	0
q_0	$q_1 0L$ $\nearrow$
q_1	$\nearrow$ $\nearrow$

nel caso "sporco":

£	0
q_0	$q_1 \mathcal{L} R$
q_1	$q_2 \mathcal{L} R$ $q_1 0R$
q_2	$q_3 0R$ $q_3 0R$
q_3	$q_4 \mathcal{L} L$ $q_4 \mathcal{L} L$
q_4	$\nearrow$ $q_4 0L$

proiezione:

$$\prod_i^n (x_i, -, x_n) = x_i$$

	x_1		x_2		x_3		x_4	
£	0000	£	00000	£	000	£	00000	£
$\uparrow q_0$								

$$\prod_3^4$$

e voglio che diventi:

	x_3	
£	000	£

£	0
q_0	$q_1 \mathcal{L} R$ $\nearrow$
q_1	$q_2 \mathcal{L} R$ $q_1 0R$
q_2	$\nearrow$ $q_2 0R$

Kleene-Robinson

Sia gli umano che i calcolatori, devono essere in grado di eseguire operazioni di base, come ad esempio:

- **zero:** $\mathbb{N} \rightarrow \mathbb{N}$ $zero(n) = 0 \forall n \in \mathbb{N}$
- **successore S:** $\mathbb{N} \rightarrow \mathbb{N}$ $S(x) = x + 1 \quad \forall x \in \mathbb{N}$
- **proiezione,** Dato $n \in \mathbb{N}$, Dati $i \in 1, \dots, n$ $\prod_i^n : \mathbb{N}^n \rightarrow \mathbb{N}$ $\prod_i^n (x_1, \dots, x_n) = x_i$

Ci sono anche delle regole, che devono saper essere applicate:

- **Composizione:** Dati $S_1, S_2, \dots, S_k : \mathbb{N}^n \rightarrow \mathbb{N}$ e tale $h : \mathbb{N}^k \rightarrow \mathbb{N}$ allora $f : \mathbb{N}^n \rightarrow \mathbb{N}$ definita con $f(x_1, \dots, x_n) = h(g_1(x_1, \dots, x_n), m, g_k(x_1, \dots, x_n))$ si dice definita per composizione (da $h, g_1, \dots, g_k$).
 - Esempio: $q(x_1, x_2) = S(\prod_1^2(x_1, x_2)) \implies x_1 + 1$
 - Esempio: $f(x) = S(S(x)) \quad f(x) = x + 2 \quad \forall x \in \mathbb{N}$
- **Ricorsione primitiva:** Dati $g : \mathbb{N}^{n-1} \rightarrow \mathbb{N}$ e $h : \mathbb{N}^{n+1} \rightarrow \mathbb{N}$, $f : \mathbb{N}^n \rightarrow \mathbb{N}$ si dice definita per ricorsione primitiva (da g e h) se:
 - $\begin{cases} f(x_1, \dots, x_{n-1}, 0) = g(x_1, \dots, x_{n-1}) \\ f(x_1, \dots, x_{n-1}, y + 1) = h(x_1, \dots, x_{n-1}, y, f(x_1, \dots, x_{n-1}, y)) \end{cases}$
 - $f(x_1, \dots, x_n, y) \begin{cases} g(x_1, \dots, x_{n-1}) & \text{se } y = 0 \\ h(x_1, \dots, x_n, y - 1, f(x_1, \dots, x_n, y - 1)) & \text{se } y > 0 \end{cases}$



- *Esempio:* $f(x, 4) = h(f(x, 3)) \implies h(x, 2, f(x, 2)) \implies h(x, 1, f(x, 1)) \implies h(x, 0, f(x, 0)) = g(x) = OK$, risolvo quindi riportando il risultato in maniera ricorsiva.

Di conseguenza Kleene-Robinson sostengono che le persone siano in grado di risolvere la ricorsione primitiva.

```
f(x,y)=F=g(x):
  for i = 0 to y{
    F = h(x, i, F)
  }
```

$f : \mathbb{N}^n \rightarrow \mathbb{N}$ è primitiva ricorsiva se si può ottenere con un numero finito di definizioni (1) e (2) a partire da funzioni di base.

Teorema: Se f è primitiva ricorsiva, allora è **TOTALE**

Somma: $\mathbb{N}^2 \rightarrow \mathbb{N}$

$$\begin{cases} +(x, 0) = x & = \Pi_1^1(x) \\ +(x, y+1) = S(+ (x, y)) & = h(x, y, +(x, y)) \end{cases}$$

Prodotto: $\mathbb{N}^2 \rightarrow \mathbb{N}$

$$\begin{cases} \cdot(x, 0) = 0 & = zero(\Pi_1^1(x)) \\ \cdot(x, y+1) = +(x \cdot y, x) \end{cases}$$

elevamento a potenza: x^2

$$\begin{cases} pot(x, 0) = 1 & = S(zero((x))) \\ pot(x, y+1) = \cdot(pot(x, y), x) \end{cases}$$

iper-potenza: x^y

$$\begin{cases} ipot(x, 0) = n \\ ipot(x, y+1) = pot(x, ipot(x, y)) \end{cases}$$

esempio:

$$ipot(3, 5) = pot(3, ipot(3, 4)) = pot(3, pot(3, ipot(3, 3))) = \dots = pot(3, pot(3, pot(3, pot(3, pot(3, ipot(3, 0)))))) = 3^{3^{3^{3^3}}} = \text{numero gr:}$$

predecessore:

$$pred(x) = \begin{cases} 0x - 1 & x > 0 \\ x & x = 0 \end{cases}$$

$$\begin{cases} pred(0) = 0 \\ pred(y+1) = y \end{cases}$$

$$p(x, 0) = zero(x)$$

$$p(x, y+1) = \Pi_1^3(x, y, p(x, y))$$

$$pred(x) = p(\Pi_1', (x), \Pi_1'(x))$$

$$\begin{cases} pred(0) = 0 \\ pred(y+1) = pred(-(x, y)) \end{cases}$$

segno:

$$sg(x) = \begin{cases} 1 & x > 0 \\ 0 & x = 0 \end{cases}$$

$$\begin{cases} sg(0) = 0 \\ sg(y+1) = 1 - S(zero(x)) \end{cases}$$

$$\begin{cases} p(x, 0) = zero(x) \\ p(x, y+1) = h(x, y, p(x, y)) \end{cases}$$

$$h(x, y, z) = S(zero(\Pi_1^3(x, y, z))) \implies sg(y) = h(y, y, y)$$

resto



$$\begin{cases} \text{mod}(x, 0) = 0 \\ \text{mod}(x, y + 1) = S(\text{mod}(y, x)) \cdot sg(-(x, S(\text{mod}(x, y)))) \end{cases} \quad h(y, x, \text{mod}(y, x))$$

produttoria

$$f(\vec{x}, y) = \prod_{i=0}^{y-1} x_i$$

$$\text{scritto anche con: } \prod_{i < y} g(\vec{x}, i)$$

$$\begin{cases} f(\vec{x}, 0) = 1 \\ f(\vec{x}, y + 1) = g(\vec{x}, y) \times f(\vec{x}, y) \end{cases}$$

sommatoria

$$f(\vec{x}, y) = \sum_{i=0}^{y-1} g(\vec{x}, i)$$

$$\text{scritto anche con: } \sum_{i < y} g(\vec{x}, i)$$

$$\begin{cases} f(\vec{x}, 0) = 0 \\ f(\vec{x}, y + 1) = g(\vec{x}, y) + (f(\vec{x}, y)) \end{cases}$$

μ operatore-limitato (di minimizzazione)

sia f primitiva ricorsiva $f : \mathbb{N}^{n+1} \rightarrow \mathbb{N}$, definisco $g(x_1, \dots, x_n, y) = \mu z < y$ (più piccolo z minore di y tale che). $f(x_1, \dots, x_n, z) = 0$ (se esiste)

$$\begin{cases} \text{il più piccolo } z \text{ minore di } y \text{ tale che } f(x_1, \dots, x_n, z) = 0 & \text{se esiste} \\ y & \text{altrimenti} \end{cases}$$

Esempio: (se chiedessi $\mu z < 4 : f(0, z = 0)$) (i numeri risultanti da $f(x, z)$ sono completamente casuali, ci interessa solo il primo risultato che soddisfa la condizione di essere pari a 0)

- $f(0, 0) = 0$ **V**
- $f(0, 1) = 7$ **V**
- $f(0, 2) = 3$ **V**
- $f(0, 3) = 6$ **V**
- $f(0, 4) = 0$ **F, STOP**
- $f(0, 5) = 8$
- $f(0, 6) = 0$

in pratica è un ciclo for che arriva fino a $y-1$, se trova un risultato prima allora esce, altrimenti li fa tutti

```
z = 0, found = false
while (z < y && !found){
  if (f(x,z) == 0){
    found = true
  }else{
    z ++
  }
}
return z
```

$$g = (x_1, \dots, x_n, y) = \sum_{\mu < y} \left(\prod_{u \leq v} sg(f(x_1, \dots, x_n, u)) \right)$$

$$\begin{cases} v = 0 & \rightarrow sg(f(0, 0)) = 1 \\ v = 1 & \rightarrow sg(f(0, 0)) \cdot sg(f(0, 1)) = 1 \cdot 1 \\ v = 2 & \rightarrow sg(f(0, 0)) \cdot sg(f(0, 1)) \cdot sg(f(0, 2)) = 1 \cdot 1 \cdot 1 \\ v = 3 & \rightarrow sg(f(0, 0)) \cdot sg(f(0, 1)) \cdot sg(f(0, 2)) \cdot sg(f(0, 3)) = 1 \cdot 1 \cdot 1 \cdot 1 \\ v = 4 & \rightarrow sg(f(0, 0)) \cdot \dots \cdot sg(f(0, 4)) = 1 \cdot 1 \cdot 1 \cdot 1 \cdot 0 \\ \text{STOP} & \rightarrow \text{sarà sempre 0} \end{cases}$$

Come detto in precedenza, tutte le funzioni che abbiamo definito fino a ora, sono primitiva ricorsive, e quindi SONO TUTTE TOTALI.



Quante sono le funzioni primitive totali?

- Solo "iterando" con +, che diventa ·, che diventa esponente, . . . e così via. Solo iterando la costruzione possiamo iterarla illimitatamente.
- Ogni f primitiva ricorsiva si ottiene applicando un numero finito di volte k , è un regolare (testo di lunghezza finita)

Sono quindi sono una infinità numerabili.

Definiamo

$g(x) = f_a(x) + 1$ e vado a calcolare $g(1) = f_1(1) + 1$ e così via. Possiamo notare che è g è TOTALE ed è diversa da una tutte le primitive ricorsive. Quindi le primitive ricorsive **non coprono tutte le funzioni totali**.

Ackermann

Definiamo:

$$\begin{cases} A(0, y) = y + 1 \\ A(x + 1, 0) = A(x, 1) \\ A(x + 1, y + 1) = A(x, A(x + 1, y)) \end{cases}$$

Se prendessimo $A(5,5)$ sarebbe un numero che farebbe sembrare il numero di atomi nell'universo osservabile tendente a 0 in confronto.

Se prendessimo una parola e volessimo inserirla nel dizionario, si ordinerebbe osservando la prima lettera, che ha un determinato numero di lettere che vengono prima di essa. Qualora avessimo due parole con la stessa prima lettera, si osserebbe la seconda lettera per mettere in confronto. Tuttavia, in questo caso ci sono un determinato numero di infinite parole (anche senza senso) che vengono prima di essa.

$$(1) < (2) < (3) < \dots < (7)$$

$$(1, 1) < (1, 2) < (1, 3) < \dots < (1, 7) < (1, 8) < \dots < (7, 7)$$

Allo stesso modo Ackermann, grazie alla sua definizione, una volta dato un $(x+1, y+1)$ inizia a inserire numeri che vengono prima di $(x+1, y+1)$, in maniera sempre più densa. Per esempio con $A(5,5)$:

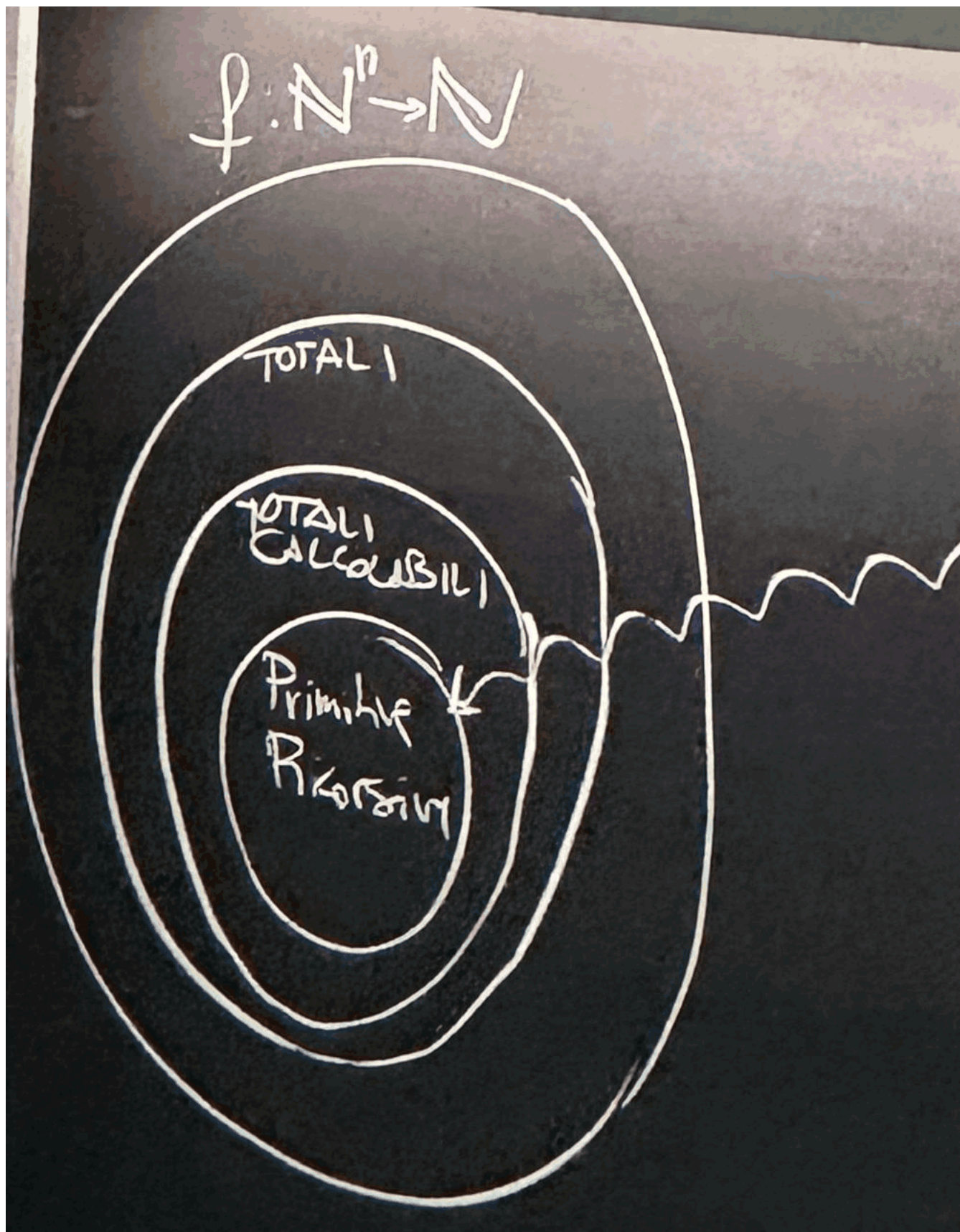
1. aggiungo il risultato $A(5,4)$
2. aggiungo il risultato $A(5,3)$
3. aggiungo il risultato $A(5,2)$
4. aggiungo il risultato $A(5,1)$
5. prendo il risultato di $A(5,1)$ e lo inserisco in $A(4, A(5,0))$, creando una colonna incredibilmente alta e densa

se volessimo partire dal basso:

- $A(0, 0) = 1$ $A(1, 0) = 2$
- $A(0, 1) = 2$ $A(1, 1) = A(0, A(1, 0)) = 3$
- $A(0, 2) = 3$ $A(1, 2) = A(0, A(1, 1)) = A(0, 3) = 4$
- $A(0, 3) = 4$ $A(1, 3) = A(0, A(1, 2)) = A(0, 4) = 5$
- $A(0, 4) = 5$ $A(1, 4) = \dots$
- $\vdots$
- $A(0, y) = y + 2 = 2 + (y + 3) - 3$
- $\vdots$
- $A(2, 0) = A(1, 1) = 5$
- $A(2, 1) = A(1, A(2, 0)) = A(1, 3) = 5$
- $A(2, 2) = A(1, A(2, 1)) = A(1, 5) = 7$
- $A(2, 3) = A(1, A(2, 2)) = A(1, 7) = 9$
- $\vdots$
- $A(2, y) = 2(y + 3) - 3$
- $\vdots$
- $A(3, 0) = A(2, 1) = 5$
- $A(3, 1) = A(2, A(3, 0)) = A(2, 5) = 2(5 + 3) - 3 = 13$
- $A(3, 2) = A(2, A(3, 1)) = A(2, 13) = 2(13 + 3) - 3 = 29$
- $A(3, 3) = A(2, A(3, 2)) = A(2, 29) = 2(29 + 3) - 3 = 61$
- $\vdots$
- $A(3, y) = 2^{y+3} - 3$
- $\vdots$
- $A(4, y) = 2^{2^{\text{inserisci 2 per 3y volte}}} - 3$



ricapitolando



- Ricorsive Primitive:
 - Regole:
 - composizione
 - ricorsione primitiva
 - Funzioni:
 - zero
 - successore



- proiezione
- somma
- prodotto
- elevamento a potenza
- iper-potenza
- predecessore
- segno
- resto
- produttoria
- sommatoria
- Totali calcolabili:
 - Ackermann (seppur un numero così grande, è comunque calcolabile, e quindi totale)
- Totali:

Def

$$f \triangleleft A \iff \exists m \text{ t.c. } f(\vec{x}) < A(m, \max f(\vec{x}))$$

Esempio

$$f(x, y) = x + y$$

$$m = 3 \quad A(3, y) = 2^{y+3} - 3$$

$$f(x, y) = x + y \stackrel{=}{=} 2^{\max x, y+3} - 3$$

Hackermann cresce quindi più di una retta.

Osservazione:

$$A \not\triangleleft A$$

Per ameno $A \triangleleft A$ Dunque esiste m t.c. $\forall x \forall y A(x, y) < A(M, \max A(x, y))$ Deve valere che ponendo $x = y = m$

$$A(m, m) < A(m, m): \text{ assurdo}$$

Teorema: Se f è primitiva ricorsiva, allora $f \triangleleft A$

Dimostrazione (cenno):

BASE:

- $zero \triangleleft A \rightarrow m=0$ è sufficiente
- $successore \triangleleft A$
- $\prod_{i=1}^n \triangleleft A$

PASSO INDUZIONE:

$$f(x_1, \dots, x_n) = h(g_1(x_1, \dots, x_n), \dots, g_k(x_1, \dots, x_n))$$

Dopo una serie infinita di passaggi (che neanche il prof ha voluto scrivere) si arriva a dimostrare che Hackermann **NON** è primitiva ricorsiva

Sia f totale definisco:

$$g(\vec{x}) = \mu z \left(f(\vec{x}, z) = 0 \right)$$

Ovvero una nuova regola per costruire una funzione a partire da altre, detto μ -operator (NON è limitato)

$$z = 0$$

$$\underline{\text{while}} (f(\vec{x}, z) \neq 0) \{$$

$$z++$$

$$\} \text{return } z$$

Pairing function

$$\begin{cases} f(0) = 1 \\ f(1) = 1 \\ f(x) = f(x-1) + f(x-2) & x > 1 \end{cases}$$

possiamo dimostrarla con:



$$\begin{aligned}
 par(x, y) &= \frac{1}{2}(x^2 + 2xy + y^2 + 3x + y) \\
 &= \frac{1}{2}(\underbrace{(x+y)^2 + (x+y)}_{\text{è pari}} + 2x)
 \end{aligned}$$

Notiamo che è pari (per il 2x) quindi è tutto pari

$$\begin{aligned}
 &= \frac{1}{2}(k^2 + k) + x \\
 par : \mathbb{N}^2 &\rightarrow \mathbb{N} \text{ è biettiva}
 \end{aligned}$$

Chiamiamo $(z)_1 = 1$ proiezione del primo elemento, e $(z)_2 = 2$ proiezione del secondo elemento, ovvero un input che torna se stesso. Quando trovo un valore sull'asse delle y, che trasformato è maggiore del dato in unput mi fermo.

Proiezione 1 e proiezione 2 sono tutte e due primitive ricorsive.

Per casa scrivere il codice

scrivimi

Idea:

$$\begin{aligned}
 F(n) &= \text{pair}(\text{fibonacci}(n), \text{fibonacci}(n+1)) \\
 F(0) &= \text{pair}(\underbrace{\text{fibonacci}(0)}_0, \underbrace{\text{fibonacci}(1)}_1) = 4 \\
 F(n+1) &= \text{pair}(\text{fibonacci}(n+1), \text{fibonacci}(n+2)) \\
 &= \text{pair}(\text{fibonacci}(n+1), \text{fibonacci}(n) + \text{fibonacci}(n+1)) = h(F(n))
 \end{aligned}$$

Quindi fibonacci non è altro che la proiezione del primo elemento di F, e quindi è primitiva ricorsiva.

$$\begin{aligned}
 \text{dupla}(x_1, x_2) &= \text{pair}(x_1, x_2) \\
 \text{dupla}(x_1, x_2, \dots, x_n) &= \text{pair}(x_1, \text{dupla}(x_2, \dots, x_n))
 \end{aligned}$$

Teorema di equivalenza delle funzioni calcolabili

$f : \mathbb{N} \rightarrow \mathbb{N}$ è parziale ricorsiva solo e solo se è Turing calcolabile

Dimostrazione:

Se φ è ricorsiva, allora esiste una MdT che la calcola che:

- Funziona anche se il nastro a destra dell'input e a sinistra della posizione iniziale della testina non è sequenza illimitata di £
- Quando termina, termina subito a destra dell'input, il quale non viene modificato dalla computazione
- La parte del nastro che sta a sinistra della posizione iniziale della testina non viene modificata

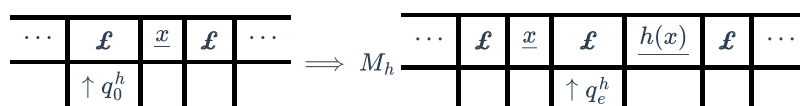
Come le 3 funzioni di base, queste tre ipotesi sono sufficienti a provare in maniera induttiva.

- Zero
- Successore
- Proiezione

Vogliamo definire una MdT che calcola $f(g(x), h(x))$ nell'ipotesi di possedere le macchine di turing M_f, M_g, M_h che calcolano funzioni:

- $f : \mathbb{N}^2 \rightarrow \mathbb{N}$
- $g, h : \mathbb{N} \rightarrow \mathbb{N}$

Composizione:



[illegible]

$\cdots$	$\mathcal{F}$	$\underline{x}$	$\mathcal{F}$	$\underline{h(x)}$	$\mathcal{F}$	$\underline{x}$	$\mathcal{F}$	$\underline{g(x)}$	$\mathcal{F}$	$\underline{h(x)}$	$\mathcal{F}$	$\cdots$
							$\uparrow q_0^f$					

$$\Rightarrow M_f \quad \begin{array}{c|cccccccccccccc} \hline \dots & \mathcal{L} & \underline{x} & \mathcal{L} & \underline{h(x)} & \mathcal{L} & \underline{x} & \mathcal{L} & \underline{g(x)} & \mathcal{L} & \underline{h(x)} & \mathcal{L} & \underline{f(g(x), h(x))} & \mathcal{L} & \dots \\ \hline & & & & & & & & & & \uparrow q_e^f & & & & \\ \hline \end{array}$$

$\dots$	$\mathcal{L}$	$\underline{x}$	$\mathcal{L}$	$\underline{f(g(x), h(x))}$	$\mathcal{L}$	$\dots$
			$\uparrow q_0$			

Definiamo la MdT che calcola la funzione $f : \mathbb{N}^2 \rightarrow \mathbb{N}$

$$\begin{cases} f(x, 0) &= g(x) \\ f(x, y+1) &= h(x, y, f(x, y)) \end{cases}$$

```
F = g(x)
for (i=0; i < y; i++){
    F = h(x, i, F)
}
return F
```

$\dots$	$\mathcal{L}$	$\underline{x}$	$\mathcal{L}$	$\underline{y}$	$\mathcal{L}$	$\dots$
	$\uparrow q_0$					

$\cdots$	$\mathcal{L}$	$\underline{x}$	$\mathcal{L}$	$\underline{y}$	$\mathcal{L}$	$\mathcal{L}$	$\underline{x}$	$\mathcal{L}$	$\underline{y-1}$	$\mathcal{L}$	$\cdots$
										$\uparrow q$	

$\dots$	$\mathcal{L}$	$\underline{x}$	$\mathcal{L}$	$\underline{y}$	$\mathcal{L}$	$\mathcal{L}$	$\underline{x}$	$\mathcal{L}$	$\underline{y-1}$	$\mathcal{L}$	$\underline{x}$	$\mathcal{L}$	$\dots$	$\mathcal{L}$	1	$\mathcal{L}$	$\underline{x}$	$\mathcal{L}$	0	$\underline{x}$	$\mathcal{L}$	$\mathcal{L}$	$\dots$
																						$\uparrow q'$	

$\cdots$	$\mathcal{L}$	$\underline{x}$	$\mathcal{L}$	$\underline{y}$	$\mathcal{L}$	$\mathcal{L}$	$\underline{x}$	$\mathcal{L}$	$\underline{y-1}$	$\mathcal{L}$	$\cdots$	$\mathcal{L}$	$\underline{0}$	$\mathcal{L}$	$\underline{x}$	$\mathcal{L}$	$\underline{g(x)}$	$\mathcal{L}$	$\cdots$
																$\uparrow q_e^g$			

$\dots$	$\mathcal{L}$	$\underline{x}$	$\mathcal{L}$	$\underline{y}$	$\mathcal{L}$	$\mathcal{L}$	$\underline{x}$	$\mathcal{L}$	$\underline{y-1}$	$\mathcal{L}$	$\dots$	$\mathcal{L}$	$\underline{0}$	$\mathcal{L}$	$\underline{x}$	$\mathcal{L}$	$\underline{g(x)}$	$\mathcal{L}$	$\underline{h(x, 0, f(x, 0))}$	$\mathcal{L}$	$\dots$
																		$\uparrow q_e^h$			



ricordando che $h(x, 0, f(x, 0)) = f(x, 1)$

Continuano a lanciare la macchina, si otterrà che $h(x, y - 1, f(x, y - 1)) = f(x, y)$, dove si copierà $f(x, y)$ e la macchina termina

Ma quando è che mi fermo?

Mi fermo quando trovo due £ consecutive, ovvero quando $y = 0$.

μ operatore:

...	£	<u>x</u>	£	...
	$\uparrow q_0$			

Si aggiunga uno 0 e un £ a dx di x

Si lanci $M_g(x, 0)$

...	£	<u>x</u>	£	0	£	...
					$\uparrow q_0^g$	

Quando termina, vedo se l'output è 0, in tal caso basta spostarsi a sx di un £ $\implies$

...	£	<u>x</u>	£	0	£	0	£	...
					$\uparrow q$			

Se l'output è > 0 si aggiunge uno 0 (...) e si rilancia M_g (e così via) $\implies$

...	£	<u>x</u>	£	0	0	£	...
	$\uparrow q_0^g$						

Verso opposto:

Mostriamo ora l'implicazione inversa e cioè se φ è Turing calcolabile, allora φ è parziale ricorsiva.

Sia φ una funzione calcolabile da una MdT Z . L'obiettivo è quello di codificare il comportamento della MdT Z come una funzione sui naturali.

Per semplicità assumiamo che la MdT Z sia definita con 2 simboli, che corrispondono ai numeri $0 = \text{£}$ e 1 . inoltre gli stati $q_0, \dots, q_k$ sono identificati dai numeri $0, \dots, k$.

$$\Sigma = 0, 1$$

$$Q = 0, \dots, k$$

$$\begin{cases} m = \sum_{i=0}^{\infty} b_i 2^i = \sum_{i=0}^{k_m} b_i 2^i \\ i = \sum_{i=0}^{\infty} c_i 2^i = \sum_{i=0}^{k_n} c_i 2^i \end{cases}$$

Dove k_m (k_n) è il massimo i per cui $b_i \neq 0$ (rispettivamente $c_i \neq 0$).

Se volessimo usare il binario nella macchina di turing, non potremmo sapere il numero equivalente, perchè sul lato destro ho infiniti zeri. ma possiamo invece leggerlo al contrario, partendo dal ultimo 1 a destra (usandolo come bit significativo) e leggiamo i bit da destra a sinistra, fino a quando non trovo un 0/1.

sulla parte sinistra invece, possiamo leggere come al solito, da sinistra a destra, fino a quando non troviamo un 0/1.

Definiamo le seguenti 3 funzioni:

$$\sigma_Q : Q \times \Sigma \rightarrow Q \cup k + 1, \sigma_\Sigma : Q \times \Sigma \rightarrow \Sigma, \sigma_X : Q \times \Sigma \rightarrow 0, 1$$

- $\sigma_Q(q, s)$ è lo stato che la MdT Z assume quando si trova nello stato q e legge il simbolo s
- $\sigma_\Sigma(q, s)$ è il simbolo che la MdT Z produce quando si trova nello stato q e legge il simbolo s
- $\sigma_X(q, s)$ è lo spostamento a destra (=1) o a sinistra (=0) compiuto dalla MdT Z esegue quando si trova nello stato q e legge il simbolo s

Per i valori non definiti si assegna il valore $k + 1$ a σ_Q e 0 e alle altre.

Sono definite per i casi della matrice di Z e dunque primitive ricorsive.

Possiamo quindi definire le trasformazioni compiute eseguendo un singolo passo della MdT Z sulle quadruple rappresentanti da una generica ID di Z .

$$g_Q(q, s, m, n) = \delta_Q(q, s)$$



$$g_{\Sigma}(q, s, m, n) = (m \bmod 2)(1 - \delta_X(q, s)) + (n \bmod 2)\delta_X(q, s)$$

$$g_M(q, s, m, n) = (2m + \delta_{\Sigma}(q, s))\delta_X(q, s) + (m \bmod 2)(1 - \delta_X(q, s))$$

$$g_N(q, s, m, n) = (2n + \delta_{\Sigma}(q, s))(1 - \delta_X(q, s)) + (n \bmod 2)\delta_X(q, s)$$

Queste funzioni sono chiaramente primitive ricorsive, essendo la composizione di funzioni primitive di funzioni ricorsive. Possiamo dunque rappresentare una trasmissione $\alpha - |\beta$ nel modo seguente:

Se $ar(\alpha) = (q, s, m, n)$, allora

$$ar(\beta) = (g_Q(q, s, m, n), g_{\Sigma}(q, s, m, n), g_M(q, s, m, n), g_N(q, s, m, n)).$$

Per rappresentere l'effetto di t-transizioni o passi di calcolo della MdT Z , definiamo le seguenti funzioni di 5 argomenti:

- $P_Q(t, q, s, m, n)$ è lo stato ($\in [0.k]$) ottenuto dopo t -passi partendo da una ID α tale che $ar(\alpha) = (q, m, s, n)$
- $P_{\Sigma}(t, q, s, m, n)$ è il simbolo ($\in \{0, 1\}$) ottenuto dopo t -passi partendo da una ID α tale che $ar(\alpha) = (q, m, s, n)$
- $P_M(t, q, s, m, n)$ è il valore ($\in \mathbb{N}$) rappresentante il nastro a sinistra della testina ottenuto dopo t -passi partendo da una ID α tale che $ar(\alpha) = (q, m, s, n)$
- $P_N(t, q, s, m, n)$ è il valore ($\in \mathbb{N}$) rappresentante il nastro a destra della testina ottenuto dopo t -passi partendo da una ID α tale che $ar(\alpha) = (q, m, s, n)$
- P_Q che può essere definita come segue:
 - $P_Q(0, q, s, m, n) = q$
 - $P_Q(t+1, q, s, m, n) = P_Q(g_Q(q, s, m, n), g_{\Sigma}(q, s, m, n), g_M(q, s, m, n), g_N(q, s, m, n))$

Come capisco la terminazione?

Se la mia macchina di turing che simulavo, se nella mia computazione arrivo ad un determinato numero che cercavo, ho finito.

La MdT Z termina il suo calcolo nel primo (minimo) t tale che:

$$P_Q(t, q_0, s_0, m_0, n_0) = k+1, \text{ essendo } \alpha_0 \text{ la configurazione iniziale tale che } ar(\alpha_0) = (q_0, s_0, m_0, n_0).$$

$$\mu t. (k+1 - P_Q(t, q_0, s_0, m_0, n_0) = 0)$$

FINE FORMALISMO

L'ultimo passo:

Sia $x \in \mathbb{N}$ un numero naturale di input. al solito, assumiamo che la MdT inizi con il suo calcolo avendo il numero x sul nastro alla destra della sua testina codificato come una sequenza di $x+1$ uni

...	0	0	$\underbrace{1 \dots 1}_{x+1}$	0	...
		$\uparrow q_0$			

Pertanto avremo che $q = 0, s = 0, m = 0, n = 2^{x+1} - 1$

La MdT si fermerà in uno stato:

...	0	0	$\underbrace{1 \dots 1}_{f(x)+1}$	0	...
		$\uparrow q$			

Per cui $n = (2^{f(x)+1}) + \dots$ dove in $\dots$ ci saranno esponenti di 2 grado superiore (ma lo 0 dopo l'output ci permette di determinare $f(x)$ da n).

Sia C la funzione primitiva corsiva tale che dato un n numero del tipo sopra, permette di calcolare $f(x)$. C si può definire nel seguente modo:

Prima si definisce f come $f(x, 0) = x$ e $f(x, y+1) = f(x, y)/2$ Dunque:

$$C(n) = \mu y \leq n (f(n, y) \bmod 2 = 0) - 1$$

La funzione φ calcolata da Z è esprimibile dalla seguente funzione parziale:

$$\varphi(x) = C(P_N(\mu t. (k+1 - P_Q(t, 0, 0, 0, 2^{x+1} - 1) = 0), 0, 0, 0, 2^{x+1} - 1))$$

COROLLARIO

Per ogni funzione di Turing calcolabile φ esistono f.g primitive ricorsivi tali che

$$\varphi(x) = f(\mu t. (g(t, x) = 0), x)$$

ovvero, in parole più semplici, ogni singolo programma che possiamo scrivere in un qualsiasi linguaggio di programmazione, potrebbe essere scritto con solo `while`



Tesi di Church-Turing:

La classe delle funzioni "intuitivamente calcolabili" coincide con la classe delle funzioni Turing calcolabili.

o, spiegato in italiano:

Tutto ciò che è intuitivamente calcolabile è calcolabile da una macchina di Turing.

f è Turing calcolabile $\implies f$ è ricorsiva ($x \rightarrow$ nozione (while) di funzione calcolabile $\implies$ Tesi di Church-Turing

Vorremmo arrivare ad un programma (o MdT) universale tale che per ogni "programma" P e per ogni "suo input" x :

$$U(P, x) = P(x)$$

Come input potremmo dare una stringa di zeri, ma come potremmo passargli la pagina?

Idea:

associare un numero naturale ad ogni MdT.

MdT con alfabeto $\{\mathcal{L}, 0\}$

1. stato:

	$\mathcal{L}$	0
q_0	$q_0 \mathcal{L} L$	$q_0 \mathcal{L} L$
	$q_0 \mathcal{L} R$	$q_0 \mathcal{L} R$
	$q_0 0 L$	$q_0 0 L$
	$q_0 0 R$	$q_0 0 R$

Sono presenti quindi un massimo di 25 combinazioni

Possiamo una gerarchia arbitraria, come ad esempio:

- niente $< q_0 \mathcal{L} L$
- $q_0 \mathcal{L} L < q_0 \mathcal{L} R$
- $q_0 \mathcal{L} R < q_0 0 L$
- $q_0 0 L < q_0 0 R$

possiamo scrivere tutte le possibili combinazioni di 25 elementi, fino a quando si arriva alla 6° combinazione, otteniamo:

$q_0 | q_0 \mathcal{L} L | q_0 \mathcal{L} L$ dove usa due variabili

continuando con le combinazioni vorremmo arrivare a:

$$U(P, x) = P(x) \implies U(n, x) = M_n(x)$$

ovvero una macchina universale.

Partendo dal caso più semplice, dove la macchina non si muove e l'input diventa output (viene calcolata la funzione identità) otteniamo la macchina 0.

In confronto la macchina 6 non può terminare, perchè per definizione per terminare deve trovare un simbolo definito, ma la macchina 6 non ha simboli definiti, solo $q_0 \mathcal{L} L$, che vuol dire che scrive blank e poi andare a sinistra (per sempre).

$$\forall n \left(M_6(x) \uparrow \right)$$

E quindi è sempre indefinita (con il simbolo $\uparrow$ si indica una funzione che va in loop infinito), di fatto neanche legge l'input.

Ma quindi, tutte le successive sono infinite?

NO

2. stati:

	$\mathcal{L}$	0
q_0		
q_1		

quindi abbiamo 9 possibilità per cella, con 4 celle, abbiamo 9^4 combinazioni, e così via.

ad esempio:

la **25**:



£	0
q_0	
q_1	

la **26**:

£	0
q_0	$\diagup$
q_1	$\diagup$
	q_0 £ L

la **30**:

£	0
q_0	$\diagup$
q_1	$\diagup$
	q_0 £ L

possiamo notare come la **26** fornisca la funzione identità

	1	2	3	...
combinazioni per cella	5	9	13	...
celle	2	4	6	...
combinazioni totali	5^2	9^4	13^6	...

scriviamo ora un programma in Java che:

1. Algoritmo dato un numero n, ci restituisce la matrice
2. Simula la MdT e lo esegue sui input x

Ma se lo scrivo tramite programma, posso usare anche una macchina di turing per scrivere il programma.

Dato M_0 posso calcolare φ_0 e così per qualsiasi M_n posso calcolare φ_n .

NOTAZIONE:

$M_n(x)$ = Esguo in MdT n.esponenti nell'input n

$M_n(x) \downarrow =$ l'esecuzione termina

$M_n(x) \uparrow =$ l'esecuzione non termina

$M_n(x) \downarrow y =$ l'esecuzione termina e restituisce y

$M_n(x) \neq y =$ l'esecuzione termina ma restituisce un valore diverso da y OPPURE $M_n(x) \uparrow$

$\varphi_n(x) \downarrow =$ la funzione φ_n calcolata dalla macchina n-esima è definita nell'input x

$\varphi_n(x) \uparrow =$ la funzione φ_n calcolata dalla macchina n-esima non è definita nell'input x

$\varphi_n(x) = y =$ la funzione φ_n tale che esiste y

$\forall n(\varphi_0(x) = x) = \varphi_0$ l'identità

$\forall n(\varphi_n(x) \uparrow) =$ la funzione è definita ma non termina in x

Lemma

Non esiste una funzione calcolabile g tale che

$$g(n) = \begin{cases} 1 & \varphi_x(x) \downarrow [M_x(x) \downarrow] \\ 0 & \varphi_x(x) \uparrow [M_x(x) \uparrow] \end{cases}$$

Non esiste un programma, che dato un input in un altro programma ci sappia dire se terminerà o meno.

Dimostrazione:

Per assurdo, sia g una funzione calcolabile



$\implies$ esiste $i_0 \in \mathbb{N}$ tale che $g = \varphi_{i_0}$ (cioè g è calcolabile dalla MdT)

Definisco:

$$g'(x) = \begin{cases} \uparrow & \text{se } g(x) = 1 \\ 0 & \text{se } g(x) = 0 \end{cases}$$

Quindi: se esiste una macchina in m_{i_1} , aggiungo uno stato alla macchina che mi calcola g'

- Se $\varphi_{i_1} = 0$ significa che $g(x) = 0$ dove $\varphi_{i_1}(i_1) \uparrow$
- Se $\varphi_{i_1} \uparrow$ significa che $g(x) = 1$ dove $\varphi_{i_1}(i_1) \downarrow$

ASSURDO

Teorema di indecidibilità di Halting

Non esiste h calcolabile tale che:

$$\forall x \forall y \quad h(x, y) = \begin{cases} 1 & \text{se } \varphi_x(y) \downarrow \\ 0 & \text{se } \varphi_x(y) \uparrow \end{cases}$$

Teorema (a prova di ingegnere):

Siamo in un linguaggio di programmazione, Non esiste un programma Halt t.c.:

$$Halt(P, x) = \begin{cases} 1 & \text{se } P(x) \downarrow \\ 0 & \text{se } P(x) \uparrow \end{cases}$$

Dimostrazione:

Per assurdo, supponiamo che HALT esista.

Avrà la forma:

```
Halt(P,x){
  codice di blocco fantachilometrico
  // Suppongo che l'output y ritorni y
  return y
}
```

ha due output:

```
Pippo(A){
  P = A
  x = A?
  BLOCCO DI Halt
  if (y == 0){
    return 1
  }else {
    while (true) {
      y = y
    }
  }
}
```

Se ora eseguiassi Pippo(Pippo) - *Pippo è un file, visto che lo ho scritto* - avrei due casi:

- $Pippo(Pippo) = 1 \implies$ la esecuzione di $Halt(Pippo, Pippo)$ restituisce 0 $\implies Pippo(Pippo) \uparrow$
- $Pippo(Pippo) \uparrow \implies$ la esecuzione di $Halt(Pippo, Pippo)$ restituisce 1 $\implies Pippo(Pippo) \downarrow$

ASSURDO. Non esiste Halt

Teorema S-M-N

Per ogni coppia di numeri interi positivi $m, n \geq 1$ esiste una funzione totale calcolabile di $m + 1$ argomenti s_n^m tale che:

$$\forall y_1 \forall y_2 \dots \forall y_n \in \mathbb{N} \quad \text{tale che}$$

$$\forall z_1 \forall z_2 \dots \forall z_m (\varphi_x(y_1, \dots, y_n, z_1, \dots, z_m) = \varphi_{s_n^m(x, y_1, \dots, y_n)}(z_1, \dots, z_m))$$

**Idea:**

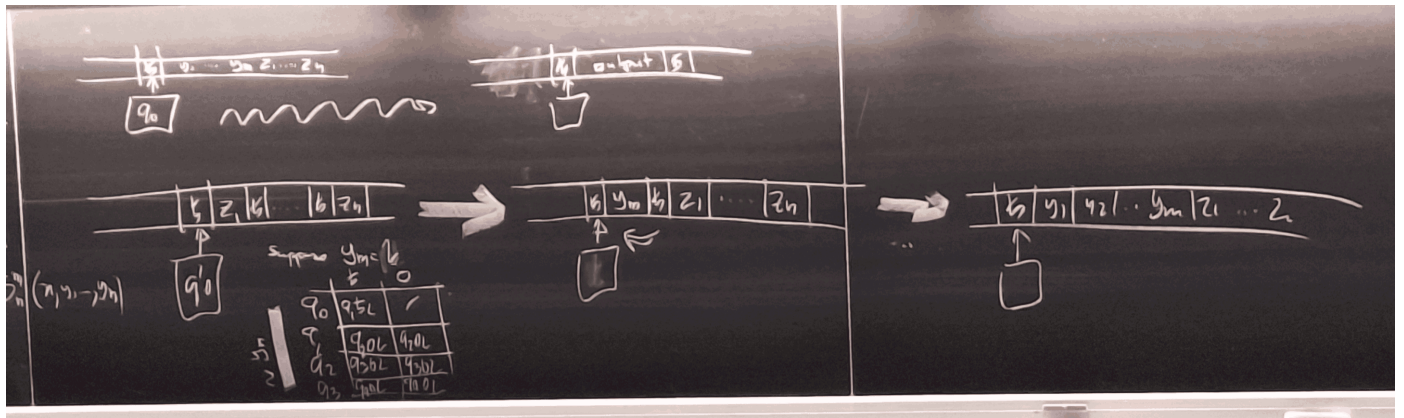
Scrivo un programma con $m + n$ argomenti.
ad esempio:

	Nome
	Indirizzo
m	Partita iva
	⋮
	Numero dipendenti
	Liste Nomi dipendenti
n	introito mensile
	uscita mensile
	⋮

Abbiamo quindi un programma $P(y_1, y_2, y_3(m), z_1, z_2, z_3, z_4(n))$ Però questo tipo di programma ad alcune aziende non è il massimo, perchè non serve sapere per ogni dipendenti il nome della azienda, per ogni dipendente. Possiamo quindi specializzare il software:

$Q(z_1, z_2, z_3, z_4)\{y_1 = \text{Pippo}, y_2 = \text{Paperino}, y_3 = 1234\}$

Riscrivere un programma generico ad uno specifico, se il programma generico è scritto bene, è un'operazione meccanica e banale, realizzabile anche da una persona che non conosce il linguaggio di programmazione, basta che conosca la logica del programma.



E alla fine eseguo M_x e dopo aver sostituito i vari pezzi di codice, calcolo il nuovo numero di MdT che dipende solo da $n, y_1, \dots, y_n$
 $S_n^m(x, y_1, \dots, y_n)$

Versione "semplificata" con $m = n = 1$:

Dato $\psi(a, b)$ calcolabile

Esiste g ricorsiva totale tale che:

$$\forall a \forall b (\psi(a, b) = \varphi_{g(a)}(b))$$

E quindi $\exists n + c. M_x$ calcolabile $\in \psi$ per ψ $SMN \exists S_1^1$ t.c. $\forall a \forall b (\psi(a, b) = \varphi_x(a, b) = \varphi_{S_1^1(x, a)}(b))$

Classificazione delle funzioni calcolabili

Una funzione primitia ricorsiva è calcolabile. Quando si usa il temrine ricorsivo, si intende calcolabile.

- **decidibile**: se esiste un programma che termina e restituisce o 1 o 0. ricorsivo
- **indecidibile**: se non è decidibile. non ricorsivo
- **non calcolabile**: se esiste un programma che quando è vera restituisce 1, altrimenti non termina. ricorsivamente calcolabile

Un **insieme** A è ricorsivo se esiste una funzione calcolabile totale tale che:

$$f(x) = \begin{cases} 1 & \text{se } x \in A \\ 0 & \text{se } x \notin A \end{cases}$$

Notiamo che A è ricorsivo se e solo " $x \in A$ " è decidibile.



Un insieme A non è ricorsivo se la funzione NON esiste:

$$K = \{x \in \mathbb{N} : \varphi_x(x) \downarrow\} = \{x \in \mathbb{N} : M_x(x) \downarrow\}$$

Versione 1:

Un insieme A è ricorsivamente numerabile (RE) se esiste f calcolabile tale che:

$$f(x) = \begin{cases} 1 & \text{se } x \in A \\ \uparrow & \text{else} \end{cases}$$

Dato x costruisco la MdT M_x e simulo $M_x(x)$, se termina, *return 1* (else loop)

Quindi K non è ricorsivo, ma è ricorsivamente numerabile.

Teorema:

A è ricorsivo se e solo se A e $\bar{A}$ sono entrambi ricorsivamente numerabili.

Dimostrazione:

$(\Rightarrow) A$ è ricorsivo $\implies \exists f$ t.c RT t.c.

$$\forall f(x) = \begin{cases} 1 & \text{se } x \in A \\ 0 & \text{se } x \notin A \end{cases}$$

$$\text{definisco } f_1(x) = \begin{cases} 1 & f(x) = 1 \\ \uparrow & f(x) = 0 \end{cases}$$

$$\text{definisco } f_2(x) = \begin{cases} 1 & f(x) = 0 \\ \uparrow & \text{else} \end{cases}$$

```

eseguo:
  y = f(x):
  if (y == 0):
    return 1
  else:
    while(h2){y = y}

```

$A k \implies \exists f$ tale che $f(n) = 1$ se $n \in A$ e $f(n) \uparrow$ altrimenti.

$\bar{A} k \implies \exists g$ tale che $g(n) = 1$ se $n \notin A$ e $g(n) \uparrow$ altrimenti.

poniamo :

- $y_1 = f(x) \quad M_f$
- $y_2 = g(x) \quad M_g$

Eseguo M_f per 10 passi, se termina restituisco 1, altrimenti eseguo M_g per 10 passi, se termina restituisco 0, altrimenti eseguo M_f per altri 100 passi, e così via.

corollario:

$$\bar{K} = \{x \in \mathbb{N} : \varphi_x(x) \uparrow\}$$

NON è rolo, se lo fosse, allo poichè $\bar{K}$ è RN, allora sarebbe k ricorsiva ASSURDO

$$\text{dom}(f) = \{x \in \mathbb{N} : f(x) \downarrow\}$$

$$\text{range}(f) = \{y \in \mathbb{N} : (\exists x \in \mathbb{N})(f(x) = y)\}$$

$$W_n = \text{domain}(\varphi_n) = \{y : \varphi_n(y) \downarrow\}$$

$$E_n = \text{range}(\varphi_n) = \{y : (\exists z \in \mathbb{N})(\varphi_n(z) = y)\}$$

definizione 1:

supponiamo che f calcolabile t.c:

$$f(u) = \begin{cases} 1 & u \in A \\ \uparrow & u \notin A \end{cases} \implies \begin{array}{l} \text{lista MdT che} \\ \text{la calcola} \\ \text{sia } (n) \\ \text{il mio indice} \end{array}$$



Chi è il dominio di n W_n ?

$$A = \{y : f(y) = 1\} = \{y : M_x(y) \downarrow\} = W_n$$

Supponiamo che esista d.c.:

$$A = W_n \implies \varphi_x(x) = \begin{cases} a \in \mathbb{N} & x \in A \\ \uparrow & \text{else} \end{cases}$$

Definisco:

$$f(n) = \begin{cases} 1 & \varphi_n \downarrow \\ \uparrow & \text{else} \end{cases}$$

è calcolabile? Eseguo $M_n(n)$, se termina restituisco 1, altrimenti loop

definizione 2:

A.r.e se esiste $n \in \mathbb{N}$ tale che $A = W_n$

definizione 3:

A ricorsivamente enumerabile (v.e) se

1. $A = \emptyset$
oppure
2. $A = \text{range}(f)$, f ricorsiva

f ricorsiva totale

$y \in A$?

Eseguo e controllo se $f(0) = y$, se sì, restituisco 1, altrimenti controllo se $f(1) = y$, se sì, restituisco 1, altrimenti controllo se $f(2) = y$, e così via.

TEOREMA

I tre enunciati sono equivalenti, ovvero:

1. A è r.e.
2. esiste φ calcolabile t.c. $A = \text{range}(\varphi)$
3. $A = \emptyset$ oppure esiste f ricorsiva totale t.c. $A = \text{range}(f)$

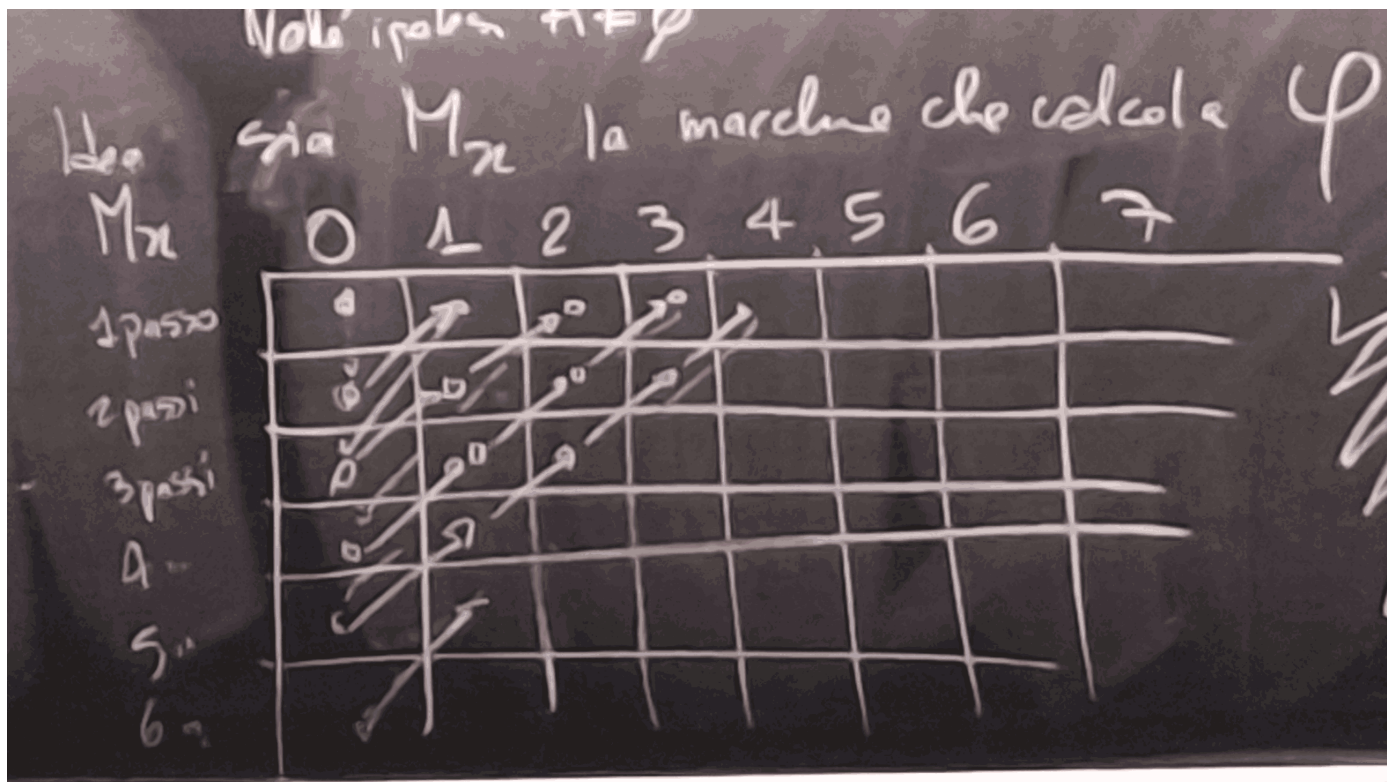
1) $\Rightarrow$ 2)

Ipotesi: A è r.e.

$$\exists f \text{ t.c. } f(n) = \begin{cases} 1 & n \in A \\ \uparrow & \text{else} \end{cases} \quad \varphi(u) = \begin{cases} u_4 & f(u) = 1 \\ \uparrow & \text{else} \end{cases}$$

ipotesi: c'è φ t.c. $A = \text{range}(\varphi)$

idea: sia M_x la macchina che calcola φ



Questo processo si chiama **dog tail**, dove si scorre tutta la tabella ricorsivamente.

L'ipotesi $A \neq \emptyset$ mi garantisce che prima o poi arriverò ad una terminazione

Scopro che:

$M_X(n_0)$ termina : k passi

$$\begin{cases} f(0) = 20 \\ f(i+1) = \text{uso la tabella} \end{cases}$$

3) $\Rightarrow$ 1)

$$A = \emptyset \rightsquigarrow A = \text{dom}(\varphi_6) = W_6 \rightarrow A. r. e.$$

Teorema:

A è ricorsivo se e solo se $A = \emptyset$ oppure $A = \text{range}(f)$ con f calcolabile totale non decrescente

$\rightarrow$) A per ipotesi ricorsivo, ci sono due casi:

- $A = \emptyset$
- $A \neq \emptyset$

Sia g la funzione di A

$$g(x) = \begin{cases} 1 & x \in A \\ 0 & x \notin A \end{cases}$$

Noi lanciamo $g(0)$, $g(1)$ e così via, come faccio a definire una range di f ?

troviamo il più piccolo valore di $g(n)$ che restituisce 1, e lo chiamiamo n_0 tale che $f(0) = n_0$

Siamo sicuri che termini per il fatto che $A \neq \emptyset$

Continuo a procedere..

$$\begin{cases} f(0) = n_0 \dots \\ f(n+1) = \begin{cases} f(n) & \text{se } g(n+1) = 0 \\ 0 & \text{se } g(n+1) = 1 \end{cases} \end{cases}$$

$\leftarrow A = \emptyset$ n.e.

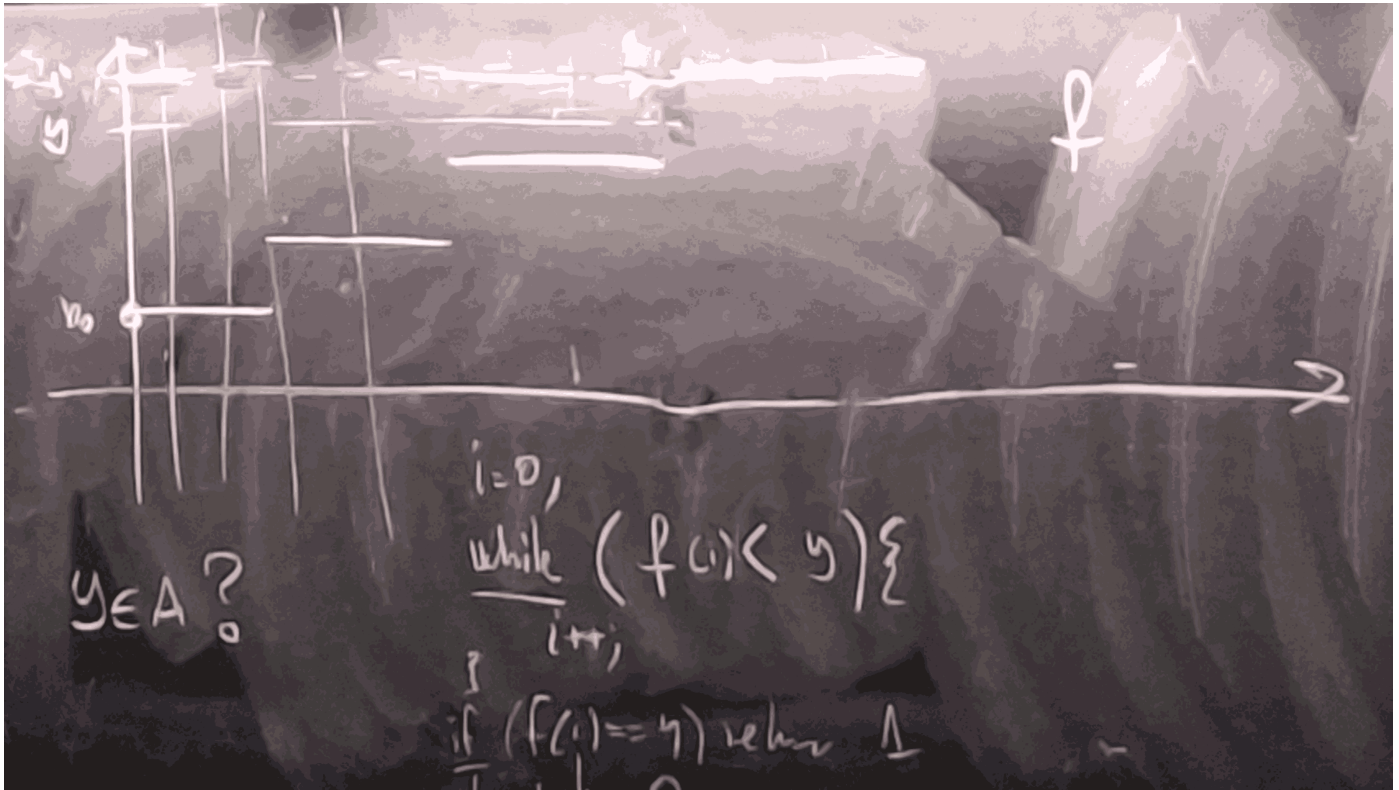
$A \neq \emptyset$:

- A finito

$$A = \{n_0, n_1, \dots, n_k\} \implies f(n) = \begin{cases} 1 & \text{n se} \\ 0 & \text{else} \end{cases}$$



- A infinito



```

i = 0,
while (f(i) < y){
    i++;
}
if (f(i) == y){
    return 1
}else {
    return 0
}

```

$\emptyset$ è ricorsivo $f(n) = 0 \forall n$ è la sua funzione caratteristica

$\mathbb{N}$ è ricorsivo $f(n) = 1 \forall n$ è la sua funzione caratteristica

A è cofinito ($\mathbb{N} \setminus A$ è finito) $f(n) = \begin{cases} 0 & x = n_1 \text{ or } n_2 \text{ or } \dots \\ 1 & \text{else} \end{cases}$

con un esempio:

$$A = \{x : M_x(3) = 5\}$$

Questo insieme ha una differenza sostanziale rispetto a quelli visti prima: la proprietà non ha più a che fare con il numero, ma con il comportamento (o semantica operativa)

Eseguiamo M_x sull'input 3, se termina controllo l'output se è 5 return 1, altrimenti faccio il loop.

Questa funzione è esattamente la **definizione 1**.

$$B = \{x : M_x(4) = 6\}$$

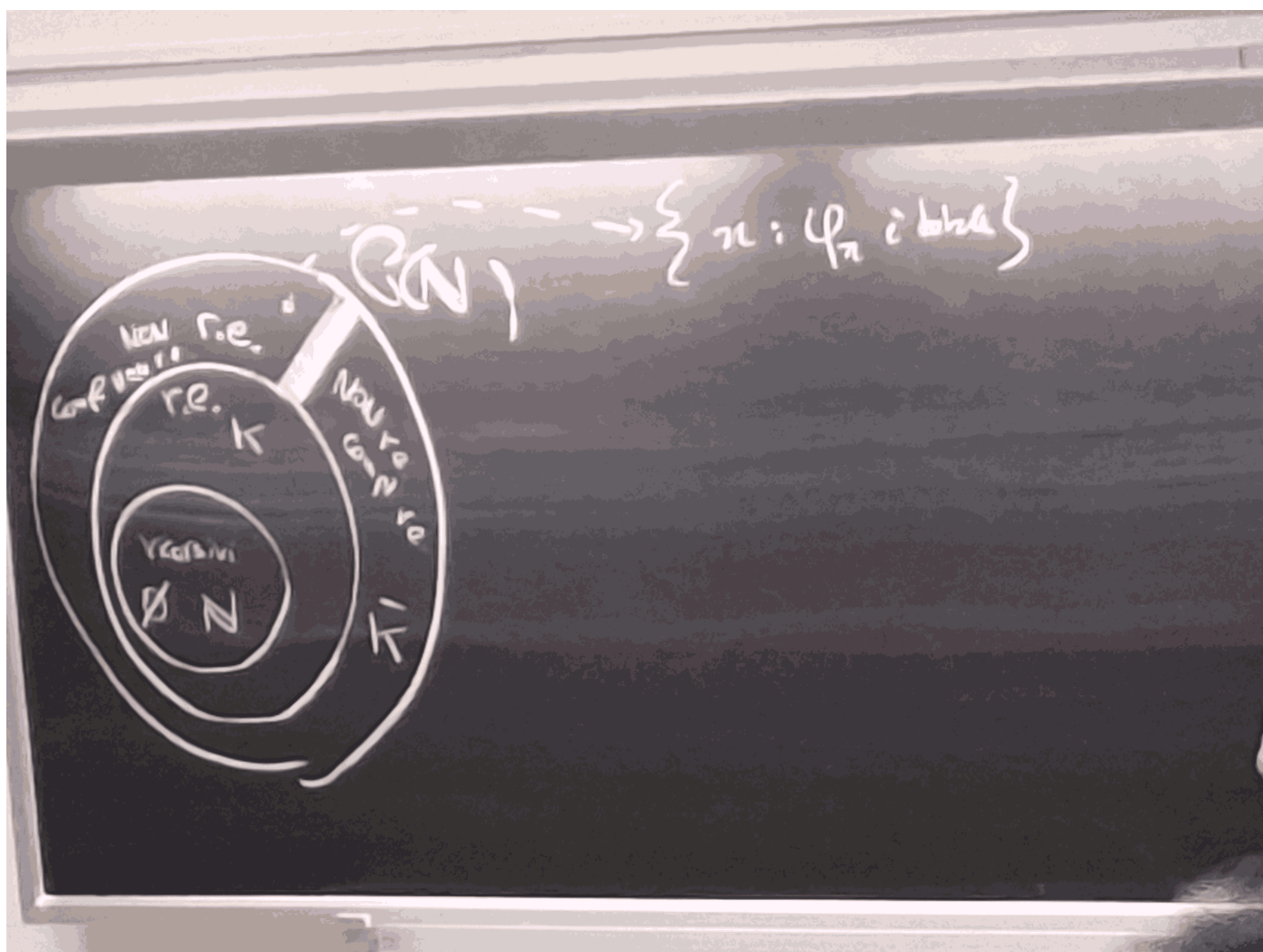
come prima, ma con un input diverso.

Questi due insiemi, sono uguali? NO

ma ci sono infinite macchine di Turing che se terminano con 5 con 3 di input e che terminano con 6 con 4 di input, tuttavia non tutte le macchine di Turing che terminano con 5 con 3 di input, terminano con 6 con 4 di input e viceversa.



Tipologia di insiemi ricorsivi



Dati insiemi $A, B \subseteq \mathbb{N}$ con $A \leq B$ (A si riduce a B , A è riducibile a B) se esiste f ricorsiva totale tale che $\forall x$:

1. $x \in A \iff f(x) \in B$
2. $x \notin A \iff f(x) \notin B$

o, in modo compatto:

$$x \in A \iff f(x) \in B$$

Supponiamo esiste $f: \mathbb{N} \rightarrow \mathbb{N}$ t.c. $A \leq B$ usando f

Supponiamo B ricorsivo, ovvero esiste g t.c. $g(x) = \begin{cases} 1 & x \in B \\ 0 & x \notin B \end{cases}$

Consideriamo h definita con $h(x) = g(f(x))$

$$x \in A \rightarrow f(x) \in B \rightarrow g(f(x)) = 1$$

$$y \notin A \rightarrow f(y) \notin B \rightarrow g(f(y)) = 0$$

Per la riduzione, f è calcolabile, g è calcolabile, quindi $g(f(\cdot))$ è calcolabile, quindi A è ricorsivo.

$$A \leq B, \quad B \text{ è ricorsivo} \rightarrow A \text{ è ricorsivo}$$

riduco il problema della appartenenza ad A a quello di B

$$A \leq B \wedge A \text{ non è ricorsivo} \rightarrow B \text{ non è ricorsivo}$$

$$A \leq B \wedge A \text{ non r.e.} \rightarrow B \text{ non r.e.}$$

$A \leq A$?

Sia f l'identità id:



- $x \in A \rightarrow id(x) = x \in A$
- $x \notin A \rightarrow id(x) = x \notin A$

$$A \leq B \text{ via } f \quad B \leq C \text{ via } g$$

Cosa succede componendole?

$$x \in A \rightarrow f(x) \in B \rightarrow g(f(x)) \in C$$

$$x \notin A \rightarrow f(x) \notin B \rightarrow g(f(x)) \notin C$$

Quindi vale la proprietà transitiva

$$A \leq B \wedge B \leq A ?$$

Consideriamo $A = \{x : \varphi_x(0) = 10\}$ $B = \{x : \varphi_x(3) = 0\}$

$x \in A$?

Dato x , calcolo $M_x(0)$ se termina guardo l'output se è 10 return 1, altrimenti loop

$$\text{Definisco } \psi(a, b) = \begin{cases} 0 & a \in A \text{ cioè se } \varphi_a(0) = 0 \text{ test decidibile} \\ \text{loop} & \text{else} \end{cases}$$

ψ è calcolabile e ha due argomenti, e grazie al [Teorema S-M-N](#), possiamo specializzarla:

$$\exists g \text{ ric. totale t.c. } \forall a \forall b (\Psi(a, b) = \varphi_{g(a)}(b))$$

$$a \in A \rightarrow \psi(a, b) = 0 \forall b \rightarrow \varphi_{g(a)}(b) = 0 \forall b \rightarrow \varphi_{g(a)}(3) = 0 \rightarrow g(a) \in B$$

$$a \notin A \rightarrow \psi(a, b) = \uparrow \forall b \rightarrow \varphi_{g(a)}(b) = \uparrow \forall b \rightarrow \varphi_{g(a)}(3) = \uparrow \rightarrow g(a) \notin B$$

$$\text{Definisco } \psi(a, b) = \begin{cases} 0 & a \in A \text{ cioè se } \varphi_a(0) = 10 \text{ test decidibile} \\ \text{loop} & \text{else} \end{cases}$$

$$a \in B \rightarrow \psi(a, b) = 10 \forall b \rightarrow \varphi_{g(a)}(b) = 10 \forall b \rightarrow \varphi_{g(a)}(0) = 10 \rightarrow g(a) \in A$$

$$a \notin B \rightarrow \psi(a, b) = \uparrow \forall b \rightarrow \varphi_{g(a)}(b) = \uparrow \forall b \rightarrow \varphi_{g(a)}(0) = \uparrow \rightarrow g(a) \notin A$$

quindi se:

$$A \leq B \text{ e } B \leq A \rightarrow A \equiv B$$

$\emptyset \leq A$? per ogni A ?

l'insieme vuoto è ricorsivo, considero $f(x) = 0$ per ogni x , supponiamo che $a \in A$:

Supponiamo che $a \neq \mathbb{N}$ e che $a \in \mathbb{N} \setminus A$

$$x \in \emptyset \rightarrow \text{assurdo perchè: } f(n) = a \notin A$$

$$x \notin \emptyset \rightarrow f(x) = a \in A$$

$$\forall x \in \mathbb{N} \quad x \notin \emptyset$$

ma quindi, $\mathbb{N} \leq A$?

supponiamo che $A \neq \emptyset$ e che $a \in A$

$$x \in \mathbb{N} \rightarrow f(x) = a \in A$$

$$x \notin \mathbb{N} \rightarrow \text{assurdo perchè: } f(n) = a \in A$$

ma quindi, $\emptyset \leq \mathbb{N}$?

Se lo fosse, allora esisterebbe f t.c.

$$\underbrace{\forall x \in \emptyset \rightarrow f(x) \in \mathbb{N}}_{\text{assurdo}} \quad \underbrace{\hspace{10em}}_{\text{TRUE}}$$

$$\underbrace{x \notin \emptyset}_{\text{TRUE}} \rightarrow \underbrace{f(x) \notin \mathbb{N}}_{\text{FALSE}} \implies \text{FALSO}$$

Sia A ricorsivo, sia $B \neq 0$ e $\neq \mathbb{N}$, allora $A \leq B$

$$A \text{ ric} \implies \exists g \quad g(g) = \begin{cases} 0 & y \in A \\ \uparrow & x \notin A \end{cases}$$



$$\text{Definisco } f(x) = \begin{cases} u & \text{se } g(x) = 1 \quad g \text{ è calcolabile} \\ v & \text{se } g(x) = 0 \quad f(n) = g(n) \cdot u + v(n - g(n)) \end{cases}$$

Sia A ricorsivamente numerabile, allora $A \leq K$

Se A è reale, allora esiste f t.c. $f(n) = \begin{cases} 1 & n \in A \\ \uparrow & \text{else} \end{cases}$

Definiamo $\varphi(a, b) = \begin{cases} 67 & f(a) = 1 \\ \uparrow & \text{else } (a \in A) \end{cases}$

Per il teorema S-M-N: $\exists g$ ric. totale t.c. $\forall a \forall b \Psi(a, b) = \varphi_{g(a)}(b)$

$$a \in A \rightarrow \psi(a, b) = 0 \forall b \rightarrow \varphi_{g(a)}(b) = 0 \forall b \rightarrow \varphi_{g(a)}(g(a)) = 0 \rightarrow \varphi_{g(a)}(g(a)) = \downarrow \rightarrow g(a) \in K$$

$$a \notin A \rightarrow \psi(a, b) = \uparrow \forall b \rightarrow \varphi_{g(a)}(b) = \uparrow \forall b \rightarrow \varphi_{g(a)}(g(a)) = \uparrow \rightarrow \varphi_{g(a)}(g(a)) = \uparrow \rightarrow g(a) \notin K$$

$$k = \{x : \varphi_x(x) = \downarrow\}$$

$$K \leq \overline{K} \quad \overline{k} \not\leq K$$

A si dice **completo** se:

- A è r.e.
- $\forall B$ r.r vale che $B \leq A$

Teorema (già visto): K è completo.



Una terminazione in un punto

Sia A completo visto il principio di prima, $K \leq A$

Se A fosse ricorsivo, allora K sarebbe ricorsivo, **ASSURDO**.

Quindi se un insieme è completo, allora non è ricorsivo.

Oppure, A completo $\implies A$ non è ricorsivo.

A completo $\implies \overline{A}$ non è r.e.

Dato A , calcolando che è r.e. completo:

1. devo dire che è r.e.

Mi chiedo, come stabilisco che $x \in A$? *programmino*

2. se riesco a mostrare che

$K \leq A \rightarrow$ Per la proprietà transitiva di $\leq$

Visto che K è completo, e dunque $\forall B$ r.e. vale che $B \leq K$

$$A = \{x : \varphi_x(4) = (4) \downarrow\} \quad \text{Terminazione in un punto}$$

A r.e.: Dato x , (costruisco M_x) simulo M_x sull'input $M_x(4)$, Se termina, return 1, altrimenti loop

$\implies$ calcola la funzione caratteristica di A

Dimostro che $K \leq A$

Definisco:

$$\psi(a, b) = \begin{cases} 0 & \text{(qualunque numero va bene)} \\ \uparrow & \text{else} \end{cases} \quad a \in K$$

è calcolabile (simulo M_a), se ..

$$a \in K \rightarrow M_x(a) \downarrow \rightarrow \psi(a, b) = 0$$

$$\forall b \rightarrow \varphi_{g(a)}(b) = 0 \quad \forall b \rightarrow$$

$$\varphi_{g(a)}(g(4)) \downarrow \rightarrow g(a) \in A$$

e quindi:

$$a \notin K \rightarrow M_x(a) \uparrow \rightarrow \psi(a, b) = \uparrow \quad \forall b$$

$$\rightarrow \varphi_{g(a)}(b) = \uparrow \quad \forall b \rightarrow \text{in particolare } \varphi_{g(a)}(g(4)) \uparrow \rightarrow g(a) \notin A$$

$$\text{Terminazione in un punto} \begin{cases} K = \{x : \varphi_x(x) = \downarrow\} \\ A = \{x : \varphi_x(3) = 6\} \end{cases}$$

B r.e., dato x (costruisco M_x) simulo M_x sull'input $M_x(3)$, Se termina, controllo se l'output è 6, se sì return 1, altrimenti loop

questo programma calcola la funzione $f(n) \begin{cases} 1 & n \in B \\ \uparrow & \text{else} \end{cases}$

Definisco:

$$\psi(a, b) = \begin{cases} 6 & \text{(qualunque numero va bene)} \\ \uparrow & \text{else} \end{cases} \quad a \in K$$

è calcolabile (simulo M_a), se se termina return 6, altrimenti loop

Dunque per Th S-M-N $\exists g$ M.t.c. $\forall a \forall b (\psi(a, b) = \varphi_{g(a)}(b))$

$$a \in K \rightarrow \forall b : \varphi_{g(a)}(b) = b \rightarrow \varphi_{g(a)}(g(3)) = 6 \rightarrow g(a) \in B$$

$$a \notin K \rightarrow \forall b : \varphi_{g(a)}(b) = \uparrow \rightarrow \varphi_{g(a)}(g(3)) = \uparrow \rightarrow g(a) \notin B$$

$$k \leq B$$



Due terminazioni

$$C = \{x : \varphi_x(10) = 5 \wedge \varphi_x(20) = 10\}$$

C r.e.:

Dato x (costruisco M_x) simulo M_x sull'input 10, se termina, controllo se l'output è 5, se sì, simulo M_x sull'input 20, se termina, controllo se l'output è 10, se sì return 1, altrimenti loop (posso inventire l'ordine di controllo, devono essere entrambi soddisfatti)

$$\psi(a, b) = \begin{cases} \frac{b}{2} & \text{(qualunque funzione va bene)} \\ \uparrow & \text{else} \end{cases} \quad a \in K$$

$$a \in k \rightarrow \forall b : \varphi_{g(a)}(b) = \frac{b}{2} \rightarrow \varphi_{g(a)}(g(10)) = 5 \wedge \varphi_{g(a)}(g(20)) = 10 \rightarrow g(a) \in C$$

$$a \notin K \rightarrow \forall b : \varphi_{g(a)}(b) = \uparrow \rightarrow \varphi_{g(a)}(g(10)) = \uparrow \wedge \varphi_{g(a)}(g(20)) = \uparrow \rightarrow g(a) \notin C$$

$$k \leq C$$

qualora avessimo numeri più complessi, potremmo fare "i fubri" riscrivendo:

$$\psi(a, b) = \begin{cases} 5 & b < 15 \wedge a \in K \\ 10 & b \geq 15 \wedge a \in K \\ \uparrow & \text{else} \end{cases}$$

e ripete il processo di prima, cambiando la regola di controllo, ma sempre con la stessa logica.

ma se usassimo anzichè del "and" usassimo l' "or"?

$$D = \{x : \varphi_x(10) = 5 \vee \varphi_x(20) = 10\}$$

La logica di prima va riscritta, perchè entrerebbe in loop se il primo caso non fosse soddisfatto, e invece potrebbe essere soddisfatto il secondo caso.

Quindi riscriviamo con:

Mando due thread in parallelo, uno che controlla se $M_x(10)$ termina con 5, e l'altro che controlla se $M_x(20)$ termina con 10, se uno dei due thread restituisce 1, allora return 1, altrimenti aspetto che l'altro finisca. Se anche l'altro restituisce 1, allora return 1, altrimenti loop.

Usando una generalizzazione dell'insieme C , potremmo definire:

$$E = \{x : (\forall y \leq x)(\varphi_x(y) = \uparrow)\}$$

Dato x eseguo $M_x(0)$, $M_x(1)$, $M_x(2)$, e così via fino a $M_x(x)$ se terminano tutte, return 1

```
while (i <= x){
  eseguo M_x(i)
  i ++
}
return 1
```

Di conseguenza ha un numero finito di determinazioni (in AND), e dunque è r.e.

La dimostrazione con il teorema S-M-N è **FONDAMENTALE** da scrivere nell'esame almeno una volta.

$$F = \{x : (\exists y \leq x)(\varphi_x(y) \downarrow)\}$$

Il metodo di prima non funzionerebbe, perchè se $M_x(0)$ non termina, allora non posso controllare se $M_x(1)$ termina, e così via, e potrei perdere la terminazione di $M_x(2)$ che invece potrebbe terminare.

Dato x eseguo $M_x(0)$, $M_x(1)$, $M_x(2)$, e così via fino a $M_x(x)$ se termina almeno uno, return 1

```
while (i <= x){
  eseguo M_x(i)
  if (termina){
    return 1
  }
  i ++
}
return 0
```



$$G = \{(x, y) : \varphi_x(y) \downarrow \wedge \varphi_y(x) \downarrow\}$$

Dato x, y eseguo $M_x(y)$ e $M_y(x)$ in parallelo, se entrambi terminano return 1, altrimenti loop

```
while (true){  
    eseguo M_x(y) e M_y(x) in parallelo  
    if (M_x(y) termina && M_y(x) termina){  
        return 1  
    }  
}
```

Definisco:

$$\psi(a, b) = \begin{cases} 0 & a \in K \quad a \in K \rightarrow \forall b \\ \uparrow & \text{else} \end{cases}$$

$$\varphi_{g(a)}(b) = 0 \rightarrow \varphi_{g(a)}(b) = 0 \rightarrow \text{pair}(g(a), g(b)) \in G$$



I Teorema di Ricorsione

Sia t funzione ricorsiva totale, allora esiste $n \in \mathbb{N}$ tale che

$$\varphi_n = \varphi_{t(n)}$$

Dimostrazione:

Definisco:

$$\psi(u, x) = \begin{cases} \varphi_{\varphi_u(u)}^{(x)} & u \in K \text{ Se } \varphi_n(u) \downarrow \\ \uparrow & \text{else} \end{cases}$$

è calcolabile?

Dati u e x siano $M_u(u)$

Se termina vedo il risultato (chiamo y), siano $M_y(x)$ definisce ricorsivo totale (else loop)

$$\implies \exists g \text{ ric totale tale che } \forall n \forall x (\varphi_{g(n)}(x) = \psi(u, x))$$

Dall'enunciato sono partito da $\underline{t}$ ricorsiva totale.

Ora ho anche $\underline{g}$ ric totale

Sono in grado di calcolare $t(g(n))$? Certo che si

Sia v indice di tale macchina di turing:

$$\forall x (\varphi_v(x) = t(g(x)))$$

in particolare $\varphi_v(n) \downarrow$

Mi chiedo:

$$\underbrace{\psi(v, x)}_{\stackrel{*}{=} \varphi_{g(v)}} \stackrel{\text{def}}{=} \begin{cases} \varphi_{\varphi_v(v)}^{(n)} & \text{se } \varphi_v(v) \downarrow \\ \uparrow & \text{else} \end{cases}$$

quindi:

$$\varphi_{g(v)}^{(x)} = \varphi_{t(g(v))}^{(x)} \forall x$$

conseguenza:

Esiste n.t.c. $W_n = \{n\}$

Definisco:

$$\psi(a, b) = \begin{cases} 0 & b = a \\ \uparrow & \text{else} \end{cases}$$

è calcolabile, e dunque per il teorema S-M-N $\exists t$ M.t.c. t.c. $\forall a \forall b (\psi(a, b) = \varphi_{t(a)}(b))$

e quindi esiste anche $W_{t(a)} = \{a\}$, e per il teorema della ricorsione $\varphi_n = \varphi_{t(n)}$ e quindi $W_{t(n)} = \{n\}$

Esercizietti:

Dimostrare n.t.c

1. $\forall x \quad \varphi_n(x) = n$
2. $\exists n = \{0, 1, 2, \dots, n\}$
3. $W_n = \{0, 2, 4, 6, \dots, 2n\}$



II teorema di ricorsione

Sia $f : \mathbb{N}^2 \rightarrow \mathbb{N}$ una funzione ricorsiva totale, allora esiste $y : \mathbb{N} \rightarrow \mathbb{N}$ r.t. t.c.

$$\forall y \quad \varphi_{\nu}(y) = \varphi_{f(\nu(y), y)}$$

è una generalizzazione del primo teorema, con infiniti punti fissi.

Una proprietà dei programmi è Π è un insieme di Macchine di Turing, Visto che le MdT sono indicizzate, usiamo la notazione dell'indice y

$$\Pi \supseteq \Pi \text{ insieme di MdT di MdT}$$

$$\Pi = \emptyset \text{ nessun}$$

$$\Pi = \mathbb{N} \text{ tutti i}$$

$$\Pi = \{0\} \text{ il programma } M_0 \text{ che calcola l'identità}$$

Una proprietà di programma Π si dice **ESTENSIONALE** se vale che:

$$\forall x \forall y \left((x \in \Pi \wedge \varphi_x = \varphi_y) \rightarrow y \in \Pi \right)$$

che vuol dire: Se due programmi "fanno la stessa cosa" o entrambi stanno dentro la proprietà o entrambi stanno fuori

$$B = \{x : W_x = \{3\}\} \quad A\{x : W_x = \{x\}\}$$

$$\varphi_x \rightsquigarrow W_x = \{3\} \quad \varphi_x \text{ t. c. } W_x = \{x\}$$

$$\varphi_y \rightsquigarrow W_y = \{3\} \quad \begin{array}{l} y \neq x \\ \varphi_x = \varphi_y \implies W_y = \{x\} \end{array}$$

Tecnica del **Padding**:

Dato x , so scrivere infinite M.d.T equivalenti a M_x

Come fare?

Creo la matrice della nostra MdT iniziale, a cui posso aggiungere uno stato in più, che potrebbe fare qualsiasi cosa, ma non verrebbe mai letto, perchè la macchina termina prima, di conseguenza posso ripetere tale operazione infinite volte, creando così infinite macchine di turing equivalenti.

Di conseguenza, le proprietà estensionali, sono infinite **TRANNE** per $\emptyset$



Teorema di Rice

Sia Π una proprietà estensionale. Π è ricorsiva, $\iff$ è banale, ovvero $\Pi = (\emptyset \vee \mathbb{N})$

$$\longleftarrow \emptyset \rightsquigarrow f(n) = 0 \quad \mathbb{N} \rightsquigarrow f(n) = 1$$

$\longrightarrow$ per assurdo Π estensionale, ricorsivo, non banale

$\implies \exists \text{gr.t. tale che}$

$$g(x) = \begin{cases} 1 & x \in \Pi \\ 0 & x \notin \Pi \end{cases}$$

$$\text{definiamo } h(x) = \begin{cases} b & xg(n) = 1 \\ a & xg(n) = 0 \end{cases}$$

$n \in \Pi \rightsquigarrow h(n) = b$ ma $b \notin \Pi$. Perchè $\varphi_n = \varphi_{h(n)}$ determino che $h(n) \in \Pi$

$n \notin \Pi \rightsquigarrow h(n) = a$ ma $a \in \Pi$. Perchè $\varphi_n = \varphi_{h(n)}$ determino che $h(n) \notin \Pi$

$$n \in \Pi \iff h(n) \notin \Pi$$

E quindi se non è estensionale, allora non è ricorsivo.

Se due funzioni sono entrambi totali, allora entrambi stanno dentro o entrambi stanno fuori, se invece una è totale e l'altra no, allora stanno in due insiemi diversi.

Per noi umani è naturale decidere se un programma è ricorsivo o meno, ma per dimostrarlo dovremmo fare una dimostrazione "su misura", esempio:

$$C = \{n : (\forall y \leq 1000)(\varphi_n(y) > y)\}$$

Esiste? **SI**

Vuoto? **NO**

$\mathbb{N}$? **NO**: "zero" $\notin C$

Quindi C non è ricorsivo.

Proprietà di Programmi

$$\Pi \in \mathbb{N}$$

Estensione di

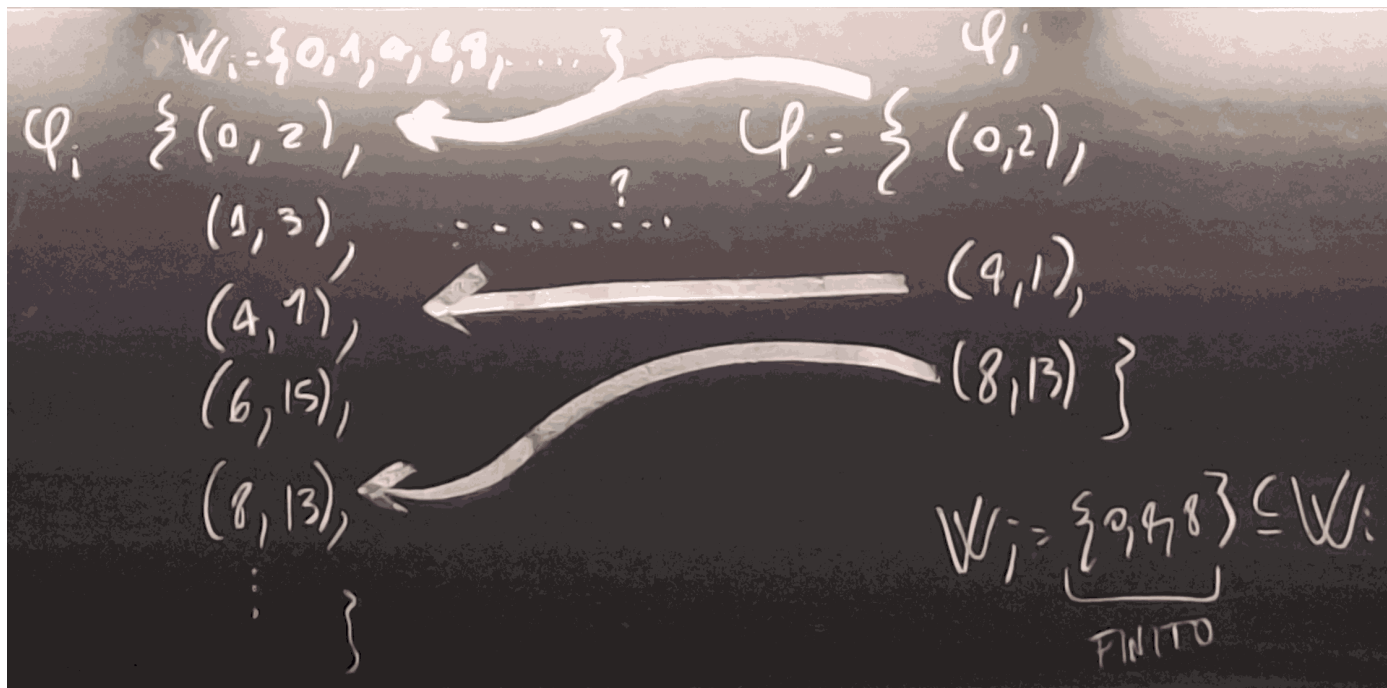
$$\forall i \forall j, (i \in \Pi \wedge \varphi_i = \varphi_j \Rightarrow j \in \Pi)$$

RICE-SHAPIO

Se Π è calcolabile, allora, Per ogni $i \in \mathbb{N}$

$$i \in \Pi \iff \exists j \in \Pi (\varphi_j \subseteq \varphi_i \wedge W_j \text{ è finito})$$

$$\forall i (i \in \Pi \iff (\exists j \in \Pi)(\varphi_j \subseteq \varphi_i \wedge W_j \text{ è definito}))$$



$$A = \{x : W_x = \{0\}\}$$

Supponiamo che i sia indice di MdT t.c. $M(j) \downarrow$ se e solo se $j=0$

Sia ora i macchina di MdT che decide sullo $\mathbb{N}$ e tale che $M_i(0) = M_j(0)$

$$i \in A? \leftarrow \begin{cases} W_i \text{ è finito} & i \in A \\ W_j \subseteq W_i \end{cases}$$

NO $\rightarrow W_i = \mathbb{N} \neq \{0\}$

Si può riprendere ogni visita che la caratterizzazione riduce SOLO funzioni con W FINITO

$$B = \{x : \varphi_x \text{ è totale} (W_x = \mathbb{N})\}$$

Sia $i \in B$ (ad esempio $i = 0$)

Sia j t.c. $M_i(j) = j \forall y \leq 100$ (else loop).

Osservo che i numeri da 1 a 100 appartengono (quindi finito), e osservo anche che $j \notin B$

NON r.e

La caratterizzazione dell'insieme richiede SOLO funzioni con dominio INFINITO

Dimostrazione

Definiamo Π estensionale r.e.

Definiamo il $\longleftrightarrow$

$\leftarrow$ Supponiamo (per assurdo) che

1. $i \in \mathbb{N}$
2. $\forall j \quad \varphi_j \subseteq \varphi_i$
3. W_i è finito
4. $j \in \Pi$
5. $i \notin \Pi$

definisco $M_j^{(b \downarrow)(\varphi_i(b) \downarrow)}$
 $(M_i(a) \downarrow)$

$$\psi(a, b) \begin{cases} \varphi_j(b) & (b \in W_j) \text{ oppure } (a \in k) \\ \uparrow & \text{else} \end{cases}$$

Quindi è calcolabile per Th SMN:

$\exists g$ tale che



$$\psi(a, b) = \varphi_{g(a)}(b)$$

$$a \in K \rightarrow \forall b \varphi_{g(a)}(b) = \varphi_i(b) \implies \varphi_{g(a)} = \varphi_i \xrightarrow{(5 \text{ estensibilità})} g(a) \notin \Pi$$

$$a \notin K \rightarrow \forall b \varphi_{g(a)}(b) = \varphi_i(b) \text{ solo passando } b \subseteq W_j \implies \varphi_{g(a)} = \varphi_i \xrightarrow{(5 \text{ estensibilità})} g(a) \in \Pi$$

→ Sia $i \in \Pi$

1. W_j è finito → Prendo $j = i$ e stop

2. W_j è infinito, e *per assurdo* $j \notin \Pi$ ogni e
 $\varphi_j \subseteq \varphi_i \wedge W_j$ è finito

$$\psi(a, b) = \begin{cases} \text{Se } M_a(a) \text{ non è terminato simulò b passi} & \text{calcolabile e decidibile} \\ \uparrow & \text{else} \end{cases}$$

$$a \in \bar{K} \rightarrow M_a(a) \text{ non termina mai per un qualsiasi numero di passi } b \rightarrow \forall b \varphi_{g(a)}(b) = \varphi_i(b) \rightarrow g(a) \in \Pi$$

$$a \in K \rightarrow M_a(a) \text{ termina per un certo numero di passi} \implies \begin{cases} b < h & \varphi_{g(a)}(b) = \varphi_i(b) \\ b \geq h & \varphi_{g(a)}(b) \uparrow \end{cases}$$

e quindi $\bar{k} \leq \Pi \implies$ assurdo, perchè $\bar{k}$ non è r.e.

Un esempio:

$$A = \{x : (\forall y \geq 100)(\varphi_x(y) = 2y - 1)\}$$

Cosa ci dice rice?

ci chiediamo:

- $A = \emptyset$? Certo che no, basta osservare la funzione, che è primitiva ricorsiva, e quindi esiste un x che la calcola
- $A = \mathbb{N}$ NO, $f(y) = 0$ e quindi esiste un x che la calcola

Quindi A non è ricorsivo.

Ma è RS?

$$x \in A \rightarrow W_x \supseteq \mathbb{N} \setminus \{0, -, 99\} \implies A \text{ non è RS.}$$

$$\psi(a, b) = \begin{cases} 2b - 1 & a \in K \\ \uparrow & \text{else} \end{cases}$$

(assumendo che $\forall b \geq 100$)

$$a \in K \rightarrow \forall b \varphi_{g(a)}(b) = \varphi_i(b) = 2b - 1 \rightarrow g(a) \in A$$

$$a \notin K \rightarrow \forall b \varphi_{g(a)}(b) = \uparrow \rightarrow g(a) \notin A$$

$$\bar{K} \leq \bar{A} \implies A \text{ non è r.e.}$$

Un altro esercizio:

Definisco:

$$\psi(ab) = \begin{cases} 2b - 1 & \text{Se } M_a(a) \not\downarrow \text{ in } b \text{ passi da calcolare} \\ 0 & \text{else} \end{cases} \quad \text{per SMN } \varphi_{g(a)}(b) = \psi(a, b)$$

$$\begin{cases} a \notin K \\ a \in \bar{K} \end{cases} \rightarrow M_a(a) \text{ termina in } b \text{ passi qualunque } b \rightarrow \forall b \varphi_{g(a)}(b) = 2b - 1 \text{ in particolare con } b > 100 \rightarrow g(a) \in A$$

$$\begin{cases} a \in K \\ a \notin K \end{cases} \rightarrow \text{Esistono } b \text{ per cui } M_a(a) \downarrow \text{ in } b \text{ passi} \rightarrow \begin{cases} b < h & \varphi_{g(a)}(b) = 2b - 1 \\ b \geq h & \varphi_{g(a)}(b) = 0 \end{cases} \text{ Solo che non è vero che } \forall b > 100 \varphi_{g(a)}(b) = 2b - 1 \rightarrow g(a)$$

Ipotesi:

Sia $x \in \mathbb{N}$ t.c. $W_x \subseteq \bar{K}$

Mi chiedo:

- Dove sta n

ipotesi:

$$1. x \in W_x = \{y : \varphi_x(y) \downarrow\} \implies \varphi_x(x) \downarrow \rightarrow x \in K$$

$$2. x \in K \rightarrow \varphi_x(x) \downarrow \rightarrow x \in W_x$$



Se provo a "dal basso" a catturare $\overline{K}$ con il dominio di una funzione calcolabile so produrre un elemento che sia il dominio $\overline{K} \implies \overline{K}$ non può essere un W_x per nessun x , e dunque $\overline{K}$ non è r.e.

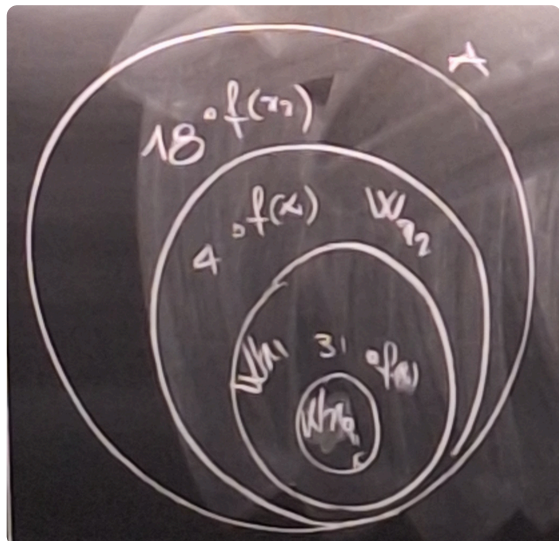
Un insieme A si dice **produttivo** se esiste f ricorsiva totale t.c.:

$$\forall n \left(W_n \subseteq A \rightarrow f(n) \in A \setminus W_n \right)$$

$\overline{K}$ è produttivo ($f = id$)

A è produttivo $\implies A$ non è r.e.

(se prendo A r.e. $\implies A = W_n$ ma allora $f(n) \in A \setminus A = \emptyset$ assurdo)



Sia A produttivo (con f)

Sia x t.c. $W_x \subseteq A$

Rendo $x = 6$ con $W_6 = \emptyset$

$$\implies f(6) \in A \setminus \emptyset = A$$

$i_0 = f(x)$ Per esempio: 31

Definisco quindi una MdT che termina solo all'input 31

Calcolo il suo indice x_1 (per esempio 123456)

$W_n = \{i_0\}$ esiste $= \{31\} \subseteq A$

$\rightarrow f(n) = i_1$ esempio: 4

Questa cosa è molto importante, perché dal punto di vista pratico, tutti i programmi che non terminano mai richiedono una verifica del software tramite insiemi produttivi.

Definisco una MdT che termina solo in i_0 e i_1 (4 e 31)

Calcolo il suo indice x_2 (per esempio 7654321)

$$W_{x_2} = \left\{ \underbrace{i_0}_{31 \in A}, \underbrace{i_1}_{4 \in A} \right\}$$

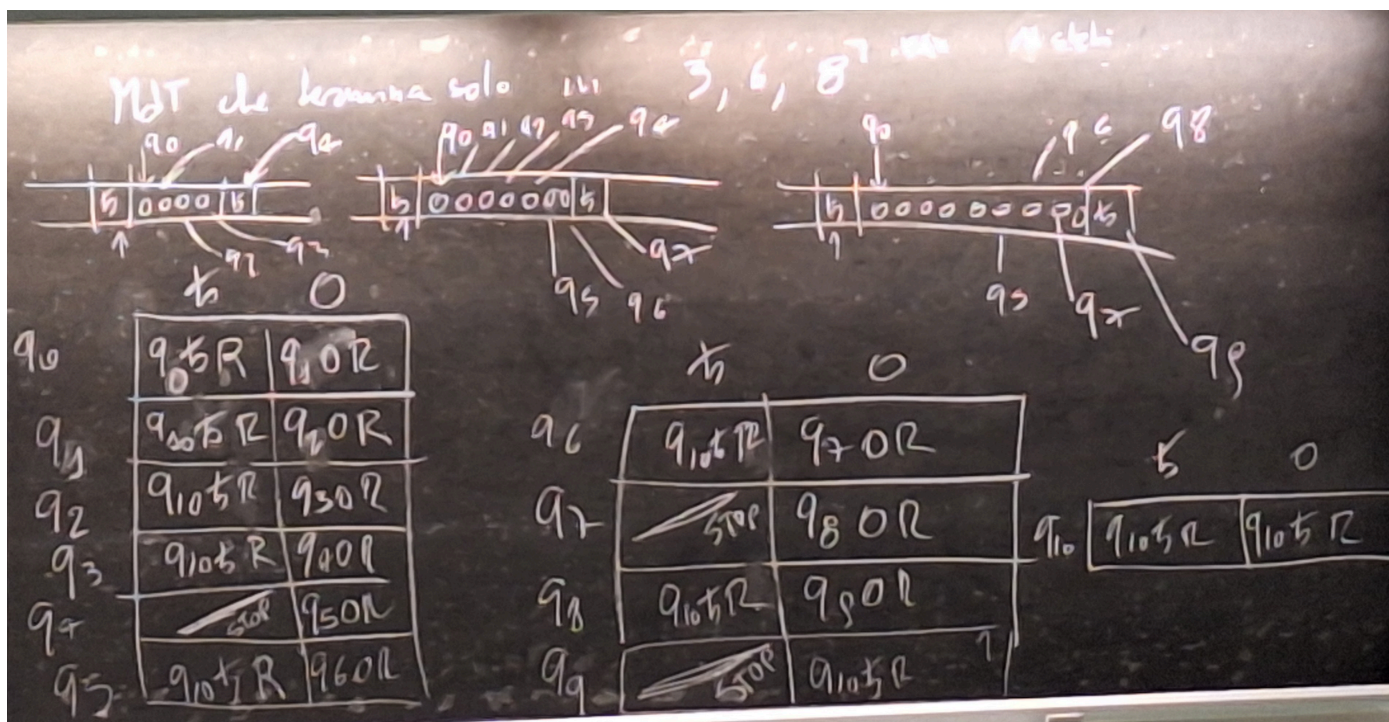
$$\implies W_{x_2} \subseteq A$$

$$\rightarrow f(x_2) \in A \setminus W_{x_2}$$

Teorema:

Se A è produttivo A ammette una soluzione r.e. è infinita

MdT che termina solo in 3,6,8



Teorema(esercizio):

Se A r.e. infinito, ammette un $B \subseteq A$ ricorsivo infinito

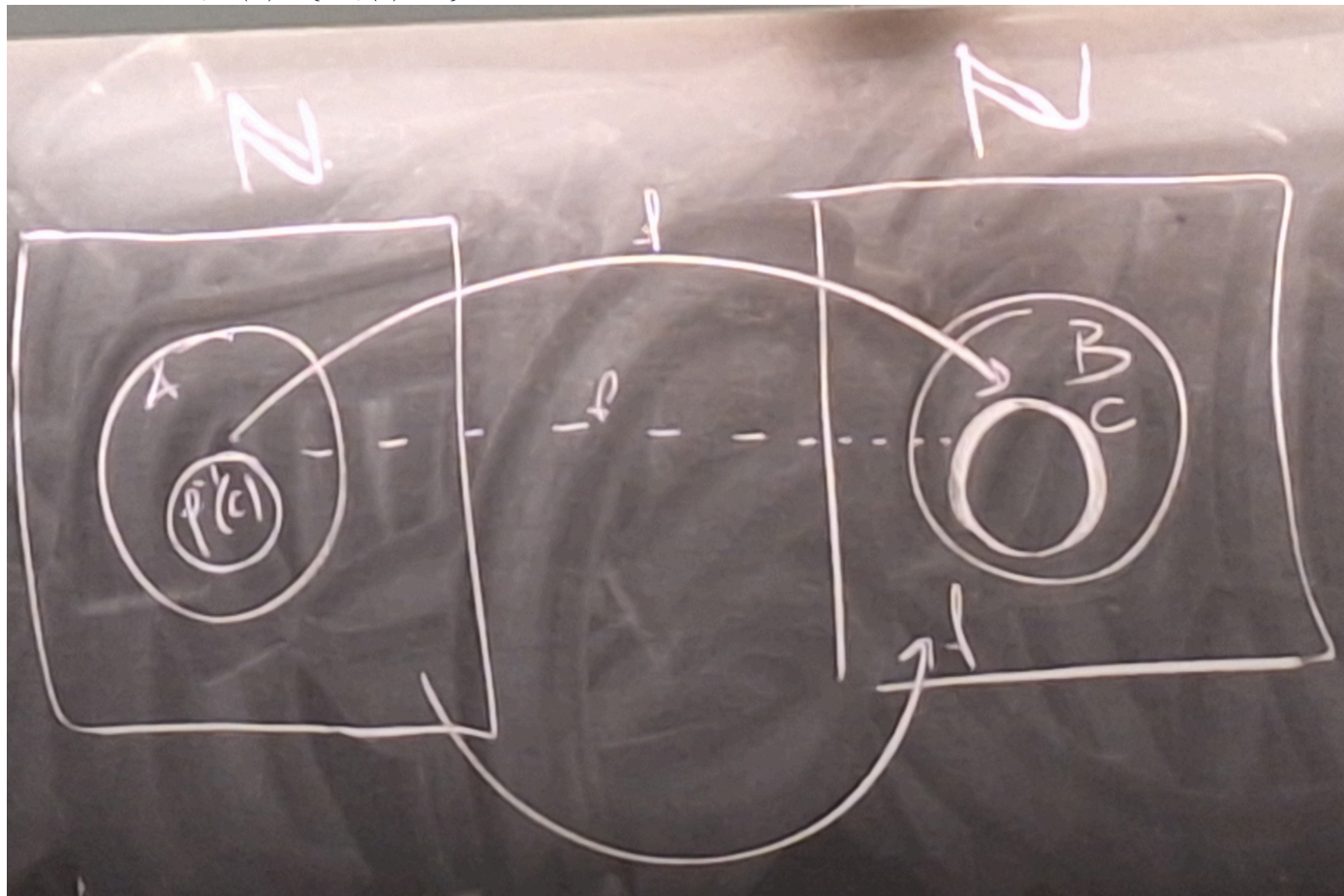
**Teorema:**

Se A è produttivo e $A \leq B$, allora B è produttivo

Supponiamo che $A \leq B$ usando f

A produttivo con g

Usiamo la notazione $f^{-1}(C) = \{x : f(x) \in C\}$



(magari mettila a destra)

Poerche $A \subseteq B$ se $C \subseteq B \rightarrow f^{-1}(C) \subseteq A$

$$\psi(a, b) = \begin{cases} 1 & \text{Se } f(b) \subseteq W_a \\ \uparrow & \text{else} \end{cases}$$

$W_{h(a)}?W_a$

Sia a tale che $W_a \subseteq B$

$W_{h(a)} = f^{-1}(W_a)$

$W_{h(a)} \subseteq A$, A prodotto via $G \rightarrow \begin{cases} g(h(a)) \in A \\ g(h(a)) \notin W_{h(a)} \end{cases}$

$F(g(h(a))) \in B$ perchè c'è la riduzione e $g(G(a)) \in W_a$ perchè $f(g(h(a))) \notin W_a$

$$B = \{x : Ex \subset \{0, 1, 2\}\}$$

$$\psi(a, b) = \begin{cases} 0 & M_a(a) \not\leq b \text{ passi} \\ 3 \text{ (qualsiasi numero che non andrebbe bene)} & \text{else} \end{cases}$$

Questo è calcolabile perchè:

$$\psi(a, b) = \varphi_{g(a)}(b)$$

$a \in \overline{K} \rightarrow M_a(a) \not\leq b \rightarrow \forall b \varphi_{g(a)}(b) = 0$

$\rightarrow Eg(a) = \{0\} \subseteq \{0, 1, 2\} \rightarrow g(a) \in B$

Quindi B è produttivo.

Ma chi è il complemento di B?

$$\overline{B} = \{x : Ex \not\subseteq \{0, 1, 2\}\}$$

Significa che: $\exists y \in En, y > 2$

Significa che: $\exists z$ tale che $\varphi_n(z) \downarrow$ e $\varphi_n(z) > 2$, e possiamo trovarlo con **dog-tail**



$$\overline{B} \text{ è completo } \begin{cases} \overline{B} \text{ è r.e.} \\ \overline{B} = B \text{ è produttivo} \end{cases}$$

per sfizio: (nno serve in un compito)

Provo a $K \subseteq \overline{B}$

$$\psi(a, b) = \begin{cases} 3 & a \in K \\ \uparrow & \text{else} \end{cases}$$

$$a \in K \rightarrow Eg(a) = \{3\} \not\subseteq \{0, 1, 2\} \rightarrow g(n) \in \overline{B}$$

$$a \notin K \rightarrow Eg(n) = \emptyset \subseteq \{0, 1, 2\} \rightarrow g(n) \in B$$



Teorema di Myhill

Definizione:

A è creativo se:

1. A è r.e.
2. $\bar{A}$ è produttivo

Se A è completo:

1. A è r.e.
 2. $\forall B$ r.e. $B \leq A$
- Ma so che K è r.e. $\implies K \leq A$
- $\downarrow$
- $\bar{K} \leq \bar{A} \implies \bar{A}$ è produttivo
- A completo $\rightarrow A$ è creativo

Dimostrazione:

Inizialmente quando vennero scoperti si pensava fossero diversi, ma poi si è scoperto che sono equivalenti.



- A creativo
- A r.e.
- $\bar{A}$ produttivo con f

Sia B r.e.

Definiamo

$$\psi(x, y, z) = \begin{cases} 0 & \left(\begin{array}{l} \text{vuol dire che} \\ \text{non sappiamo} \\ \text{nulla dell'output} \end{array} \right) \text{ se } \overbrace{y \in B}^{\text{semidecidibile}} \text{ e } \overbrace{z = f(x)}^{\text{decidibile}} \\ \uparrow & \text{else} \end{cases}$$

ψ è calcolabile per Th S-M-N $\exists g$ tale che:

$$\forall x \forall y \forall z \left(\psi(x, y, z) = \varphi_{g(x, y)}(z) \right)$$

$$W_{g(x, y)} = \begin{cases} \{f(n)\} & y \in B \\ \emptyset & \text{else} \end{cases}$$

Per il II teorema di ricorsione, esiste $\nu : \mathbb{N} \rightarrow \mathbb{N}$ tale che:

$$\forall y \quad \varphi_{g(\nu(y), y)} = \varphi_{\nu(y)}$$

$$W_{\nu(y)} = \begin{cases} \{f(\nu(y))\} & y \in B \\ \emptyset & \text{else} \end{cases}$$

$$\text{Se } y \notin B \rightarrow W_{\nu(y)} = \emptyset \subseteq \bar{A} \rightarrow f(\nu(y)) \in \bar{A} \setminus \overbrace{W_{\nu(y)}}^{\emptyset} \rightarrow f(\nu(y)) \notin \bar{A}$$

$$\text{Se } y \in B \rightarrow W_{\nu(y)} = \{f(\nu(y))\} \text{ se } f(\nu(y)) \in \bar{A} \text{ allora } W_{\nu(y)} \subseteq \bar{A} \rightarrow f(\nu(y)) \in \bar{A} \setminus W_{\nu(y)} \text{ ASSURDO}$$

$$\text{quindi } f(\nu(y)) \in A$$

Insiemi semplici

A è semplice se:

1. A è r.e.
2. $\bar{A}$ è infinito
3. $\forall x (W_x \text{ è infinito} \rightarrow A \cap W_x \neq \emptyset)$



Esiste un insieme semplice? **NON SI SA**

Ma se esistesse:

teorema: Se A è semplice, allora:

1. A non è ricorsivo
2. A non è completo

DIM (*chiede spesso all'orale*)

1. Sia A semplice:

Per assurdo, se A fosse ricorsivo:

- A è r.e.
- $\bar{A}$ è r.e. (*teorema di Post*) $\rightarrow \exists x$ t.c. $\bar{A} = W_x$
 - $W_n \cap \bar{A} \neq \emptyset$
 - per $A \cap \bar{A} = \emptyset$

2. A semplice:

Per assurdo, se A fosse completo:

- $\rightarrow \bar{A}$ è produttivo
 - ammette un sottinsieme r.e. infinito, ma ciò sarebbe assurdo perchè $W_1 \subseteq \bar{A}$

!foto?

Molti sospettano che ci siano due insiemi diversi, N e N_p , ma non è ancora stato dimostrato (*chi lo dimostrerà vincerà un premio di un milione di dollari*)



Teorema di Selberg

Teorema: Esiste S , l'insieme semplice

Dimostrazione:

Definisco $A = \{(x, y) : \overbrace{t \in W_x}^{M_x(y) \downarrow} \wedge y > 2x\}$

A è r.e., Dato z suppongo che esiste $z \in A$

$x=(a)1 \quad y=(z)2$

ric. forte $y > 2x$ se e solo $M_x(y)$ se termino ritorno 1, else loop

$$\psi(a, b) = \begin{cases} 0 & a \in K \\ \uparrow & \text{altrimenti} \end{cases}$$

$$\psi(a, b) = \varphi_{g(a)}(b)$$

$$a \in K \rightarrow \forall b \varphi_{g(a)}(b) = 0 \rightarrow \text{pair}(g(a), 2g(a) + 1) \in A$$

$$a \notin K \rightarrow \exists b \varphi_{g(a)}(b) \uparrow \rightarrow \text{pair}(g(a), 2g(a) + 1) \in A$$

A r.e. $A = \emptyset$ ad esempio $(0, 1) \in A$

$\rightarrow A = \text{range}(f) \quad f$ ricorsiva totale

Esempio:

$f(0) =$	$(61, 140)$
$f(1) =$	$(30, 70)$
$f(2) =$	$(30, 61)$
$f(3) =$	$(0, 3)$
$f(4) =$	$(0, 5)$
$f(5) =$	$(1, 8)$
$f(6) =$	$(2, 70)$
$f(7) =$	$(2, 5)$
$f(8) =$	$(30, 61)$

$(x, y) \triangleleft (x', y')$ se "esce" prima nella enumerazione

Idea:

per ogni x prendo $(x, y) \in A$, scelgo un unico y :

$$B = \{(61, 140), (30, 70), (0, 3), \dots\}$$

$$B = \{(x, y) \in A : \forall z \neq y \quad ((x, y) \in A \rightarrow (x, y) \triangleleft (x, z))\}$$

Br.e.

$$S = \{y : \exists x \quad (x, y) \in B\}$$

Quindi uno vede y , e va a vanti fino a quando non trova una macchina che lo supporta.

1. S è r.e. (analizzo l'output di f)

2. $\overline{S}$ è infinito

$$\circ xW_x 0 \rightarrow \begin{array}{c|c|c|c|c} 0 & 1 & 2 & 3 & 4 \\ 0 & 2 & 1 & & \end{array}$$

$$\circ xW_x 1 \rightarrow \begin{array}{c|c|c|c|c|c} 0 & 1 & 2 & 3 & 4 & 5 \\ 1 & 2 & 3 & 4 & & \end{array}$$

$\circ \forall n$ esiste 1 a $2n$ che non sarà inserito in S , quindi $\overline{S}$ è infinito

3. $\forall x (W_x \text{ è infinito, allora } S \cap W_x \neq \emptyset)$

$\circ \rightarrow$ ci saranno infiniti valori dove $y > 2x$ per cui $\varphi_x(y) \downarrow$

$\circ$ tra questi ci sarà uno che sempre compare per primo nella enumerazione



esercizio:

$$x \in A \rightarrow |X| \text{ è definito } \xrightarrow{R.C} \text{ NON r.e.}$$

Completare?

$$A = \{x : (\forall y \in \mathbb{N})(\varphi_x(y) \downarrow \wedge \varphi_x(y) = 5)\}$$

$$\overline{A} = \{x : (\exists y \in \mathbb{N})(\varphi_x(y) \uparrow \vee (\varphi_x(y) \downarrow \wedge \varphi_x(y) \neq 5))\}$$

$$\psi(a, b) = \begin{cases} 5 & a \in K \\ \uparrow & \text{else } \forall a \forall b \end{cases}$$

è calcolabile per Th S-M-N $\exists g$ tale che:

$$a \in K \rightarrow \forall b \quad (\varphi_{g(a)}(b) = 5) \rightarrow g(a) \in A$$

$$a \notin K \rightarrow \forall b \quad \varphi_{g(a)}(b) \uparrow \rightarrow g(a) \notin A$$

$$\implies \overline{A} \text{ è prodotto } K \leq A$$

$$\psi(a, b) = \begin{cases} 5 & Ma(a) \not\leq a \subseteq b \\ 0 & \text{else} \end{cases}$$

$$a \in \overline{K} \rightarrow \forall b \quad (\varphi_{g(a)}(b) = 5) \rightarrow g(a) \in A$$

$$a \in K \rightarrow \text{eseguire analisi} \rightarrow g(a) \notin A$$

$$\overline{K} \leq A \implies A \text{ non è r.e.}$$

$$A = \{x : (x, 2x+1) \in B\}(\forall y \in \mathbb{N})(\varphi_x(y)5)\}$$

$$B = \{x : (\forall y \in \mathbb{N})(\varphi_x(y)x)\}$$

Per B non possiamo usare il teorema di Rice, perchè non è estensionale, ma per A sì, perchè è estensionale.

$$\psi(a, b) = \begin{cases} a & \\ \text{textprovo} & a \in K \\ \uparrow & \text{else} \end{cases}$$

$$a \in \overline{K} \rightarrow \forall b \varphi_{g(a)}(b) = X \downarrow \rightarrow g(a) \in A \implies \text{textValeAncora}$$

$$a \notin K \rightarrow \forall b \varphi_{g(a)}(b) \uparrow \rightarrow g(a) \notin A \implies \text{textValeAncora} \implies \text{NON FUNZIONA}$$

Sarebbe un GRAN errore, provare a sostituire a con $g(a)$, perchè se $a \in K$ allora $g(a) \in A$, ma se $a \notin K$ allora $g(a) \notin A$, e quindi non è vero che $\forall b \varphi_{g(a)}(b) \uparrow$

e andando poi a elaborare, troverei che $\varphi_{g(a)}(b) = a$, il che sarebbe errato.

$$x \in B$$

$$\psi(a, b, c) = \begin{cases} b & a \in K \\ \uparrow & \text{else} \end{cases}$$

Per teorema S-M-N $\exists g$ ric. totale tale che:

$$\forall a, b, c \quad \varphi_{g(a,b)}(c) = \psi(a, b, c)$$

Per il II teorema di ricorsione $\exists \nu$ tale che:

$$\forall a \quad \varphi_{a, \nu(a)} = \varphi_{\nu(a)}$$

$$a \in K \rightarrow \varphi_{g(a, \nu(a))}(c) = \varphi_{\nu(a)}(c) = \nu(a) \rightarrow \nu(a) \in B$$

$$a \notin K \rightarrow \varphi_{g(a, \nu(a))}(c) = \varphi_{\nu(a)}(c) \uparrow \rightarrow \nu(a) \notin B$$

ho dovuto usare un parentro in più, perchè con solo due al cambiare di uno cambiava anche l'altro.

Potremmo prender anche una strada più furba:

$$C = \{x : \varphi_x(x) = x\}$$

questo intuitivamente è r.e.:



C è r.e.e Dato x eseguo $M_x(x)$ e se termina controllo se l'output è x, in caso affermativo accetto, altrimenti loop.

$$\psi(a, b) = \begin{cases} a \text{ non funziona} & a \in K \\ \uparrow & \text{else} \end{cases}$$

è calcolabile per S-M-N $\exists g$ tale che:

$$\exists g \text{ ricorsiva totale } \varphi_{g(a)}(b) = \psi(a, b)$$

controllando i due casi visti prima, otteniamo che: nel primo caso otteniamo che $\dots \rightarrow g(a) \notin C$

nel secondo caso otteniamo che $\dots \rightarrow \forall b \varphi_{g(a)}(b) = a$

e quindi non funziona.

funzionerebbe usando questo principio con dei passaggi più complessi, ma non è necessario perchè grazie all'identità, c'è un modo più semplice:

$$\varphi_{g(a)}(b) = a$$

$$\varphi_{f(a)}(b) = g(a)$$

esercizio:

$$A = \{x : W_x \subseteq \{0, 1\}\}$$

$$B = \{x : W_x \supseteq \{0, 1\}\}$$

$$C = \{x : W_x = \{0, 1\}\}$$

Tutti e tre sono estensionali, tutti e 3 sono $\neq \emptyset$ e $\neq \mathbb{N}$

In A e C Rice-Shapiro ci dice che non è r.e.

possiamo riscrivere B anche con:

$$B = \{x : \varphi_x(a) \downarrow \wedge \varphi_x(b) \downarrow\}$$

B r.e. Dato x eseguo $M_x(0)$ e $M_x(1)$ e se entrambe terminano, return 1, else loop.

$$\psi(a, b) = \begin{cases} 0 & a \in K \\ \uparrow & \text{else} \end{cases}$$

Per S-M-N $\exists g$ ricorsiva totale tale che:

$$a \in K \rightarrow \varphi_{g(a)}(b) = 0, \quad \varphi_{g(x)}(x) = 1 \rightarrow g(x) \in B$$

$$a \notin K \rightarrow \varphi_{g(a)}(a) \uparrow \rightarrow g(a) \notin B$$

$$A = \{x : W_x = \emptyset \text{ or } W_x = \{0\} \text{ or } W_x = \{1\} \text{ or } W_x = \{0, 1\}\}$$

$$\overline{A} = \{x : (\exists y > 1)(\varphi_x(y) \downarrow)\}$$

A è r.e. con dog-tail: eseguo $M_x(0), M_x(1), M_x(2), \dots$ se trovo un $y > 1$ che termina, allora loop, else return 1

$$\psi(a, b) = \begin{cases} 0 & a \in K \\ \uparrow & \text{else} \end{cases}$$

$$a \in K \rightarrow \forall b \quad \varphi_{g(a)}(b) \downarrow \text{ dunque ponendo } b = 2 \rightarrow g(b) \in \overline{A}$$

$$a \notin K \rightarrow \forall b \varphi_{g(a)}(b) \uparrow \rightarrow W_{g(a)} = \emptyset \rightarrow g(a) \in A$$

"Studio" C: Termina in 0 (r.e.), termina in 1 (r.e.), non termina in 2,3,4,...

$$\overline{C} = \{x : W_x \neq \{0, 1\}\}$$

in questo caso è il contrario di prima, dove termina ovunque, tranne che in 0 e 1.

proviamo con:

$$\psi(a, b) = \begin{cases} 0 & b \neq 1 \wedge a \in K \\ \uparrow & \text{else} \end{cases}$$

$$a \in K \rightarrow W_{g(a)} = \{0, 1\} \rightarrow g(a) \in C$$

$$a \notin K \rightarrow W_{g(a)} = \emptyset \rightarrow g(a) \in \overline{C}$$



con questo abbiamo dimostrato che è produttivo, NON che è completo

A volte si pu provare anzichè usare un and, usare un or:

$$\psi(a, b,) = \begin{cases} 0 & b \neq 1 \text{ or } a \in K \\ \uparrow & \text{else} \end{cases}$$

a volte funziona, a volte non succede nulla

$$a \notin K \rightarrow W_{g(a)} = \{0, 1\} \rightarrow g(a) \in C$$

$$a \in K \rightarrow W_{g(a)} = \mathbb{N} \rightarrow g(a) \notin C$$

Quindi C prodiuttivo, e $\overline{C}$ produttivo, ma non r.e.

Se provassimo qualcosa di più complesso:

$$A' = \{x : W_x \subseteq \{0, x\}\}$$

$$B' = \{x : W_x \supseteq \{0, x\}\}$$

$$C' = \{x : W_x = \{0, x\}\}$$

vale tutto come prima, TRANNE il fatto che sono estensionali.

Dal punto di vista intuitivo, che il codice guardi i valori in 0, o che guardi i valori in x, non cambia nulla.

B' è r.e. Dato x eseguo $M_x(0)$ e $M_x(x)$ e se entrambe terminano, return 1, else loop.

A' è r.e. con dog-tail: eseguo $M_x(0), M_x(x), M_x(1), M_x(2), \dots$ se trovo un y > 1 che termina, allora loop, else return 1

Per C', devo provare sia con AND e con OR, dopo le dovute riduzioni, trovo che

$$C = \{x : W_x = \{0, 1\}\}$$

$$\psi(a, b) = \begin{cases} 0 & b \leq 1 \\ \uparrow & b > 1 \wedge M_x(a) \not\leq in \leq \text{passi} \\ 0 & \text{else} \end{cases}$$

facciamo poi le riduzioni riducendolo a K



Complessità computazionale

vogliamo mettere in piedi una infrastruttura che ci permetta di studiare tutti i problemi.

Classificare "problemi" in base alla complessità computazionale "necessaria" a risolvere le loro istanze.

Cerchiamo quindi una funzione (il più possibile semplice - soprattutto l'andamento sintotico, non le costanti) che "approssima" il numero di passi necessario IN FUNZIONE dell'input.

Ad esempio la programmazione degli orari delle lezioni, sarebbe una matrice booleana, per vedere quali professori sono disponibili in quali orari, per provare a creare un programma delle lezioni.

Ma come **misuro l'input**?

- lunghezza di una stringa necessaria a descriverlo

Ma perchè? le misurazioni con θ e O non sempre sono precise, perchè un numero che viene elaborato con una determinata complessità, se fosse molto elevato, aumenterebbe comunque il tempo di esecuzione, rispetto ad un numero più piccolo, con la stessa complessità.

```
(almeno n > 2)
for i = 2 do n - 1
  if n % i == 0
    return composto
return primo
```

questo algoritmo *appaerentemente* ha complessità $O(n)$, ovvero lineare.

se prendessimo $n = 10,000,001$, nel caso peggiore non sto 8 passi, ma 10m di passi. questo algoritmo quindi in proporzione alle dimensioni dell'input è esponenziale, il che è fondamentale, perchè su questo algoritmo si basa la crittografia mondiale.

La vera complessità sarebbe $O(10^{|x|})$

Nella teoria della complessità **MAI** si usa base 1, perchè il tempo si sarebbe lineare, ma lo spazio sarebbe esponenziale.

$$\text{Numeri naturali} \rightarrow n \begin{cases} n & \text{Base 1} \\ \log_2(n) & \text{Base 2} \\ \log_{10}(n) & \text{Base 10} \end{cases}$$

Problema è l'insieme delle sue istanze a cui non c'è risposta "SI"

- Roblee = insieme di stringhe
- Roblee = linguaggio = insieme istanze a cui rispondo "SI"

Decimali vs Minimizzazione

1. Trova l'algoritmo minimo da A a B (nel grafo) (prb. funzionale)
2. esiste un cammino da A a B di distanza $\leq h$ (prb. decisionale)

Complessità computazionale: Macchina di Turing

è un po diversa dalla macchina di turing "classica": ha un nuovo simbolo $\triangleright$, che rimane per sempre sul disco, che non può essere sovrascritto, e una volta raggiunto la macchina torna indietro.

da ora in poi, accetteremo MdT con **K** nastri (che può essere anche 1) e con una sola testina di lettura/scrittura, che può leggere tutti i nastri, dove i movimenti per ogni nastro possono essere **L**, **R** o **F** (per fermarsi).

$$\delta(q, s_1, \dots, s_r) = (\overset{\text{nuovo stato b}}{\underbrace{q'}}, \overset{1}{\underbrace{s'_1, M_i}}, \dots, \overset{k}{\underbrace{s'_r, M_k}})$$

Notare come i nastri cambiano di posizione e sono **K**, ma lo stato rimane sempre uno

Una macchina di Turing deterministica, a **k** nastri (K.MdT) è una quadrupla $M = (Q, \sigma, p_0, P)$ dove:

- Q è un insieme finito di stati
- $q_0 \in Q$ è lo stato iniziale
- σ è un alfabeto finito che contiene $\triangleright$ e un simbolo di blank (spazio vuoto)



- P è un insieme di quintuple $(k+1) + (2k+1)$ tuple descritto da una funzione deterministica e totale $\delta : (Q \times \sigma^k) \rightarrow ((Q \cup \{\text{yes, no, halt}\}) \times (\sigma^k \times M)^k)$

$$\delta(q, \dots, \triangleright, \dots) = (q', \dots, \triangleright, R, \dots)$$

- Una K-MdT è di tipo decimale se ogni volta che termina, termina con yes o con no.
- Una K-MdT è di tipo funzionale se ogni volta che termina, termina con halt (h)

Si assume che la MdT iniziale sia così:

$\triangleright$	x	$\mathcal{L}$	
q_0			
$\triangleright$	$\mathcal{L}$		
q_0			
$\vdots$			
$\triangleright$	$\mathcal{L}$		
q_0			

Data una K-MdT, e un input x , il tempo necessario ad M per x è il numero di passi per terminare la computazione (con yes, no o halt)

Sia $f : \mathbb{N} \rightarrow \mathbb{N}$ totale, Si dice che M opera in tempo $f(n)$ se per ogni possibile input di x il tempo necessario ad M per x è $\leq f(|x|)$

caso particolare: Sia σ alabeto tale che $\triangleright \notin \sigma$ e $\mathcal{L} \notin \sigma$.

Un linguaggio $L \subseteq \sigma^*$ è **decidibile** da una k-MdT M Se per ogni $x \in \sigma^*$

1. se $x \in L$ M con input x termina sullo stato yes ($M(x) = \text{yes}$)
2. se $x \notin L$ M con input x termina sullo stato no ($M(x) = \text{no}$)

Se $L \subseteq \sigma^*$ è decidibile da una k-MdT che opera su tempo $f(n)$, allora:

$$L \in \text{TIME}(f(n))$$



$$P = \bigcup_{h \geq 0} \text{TIME}(n^h)$$

$$\text{PAL} = \{x \in \{0, 1\}^* : x \text{ è palindroma}\}$$

ritornando a come lo facevamo con la MdT ad un nastro come idea base:



- andavo a leggere il primo simbolo, e lo memorizzavo nello stato
- andavo a leggere l'ultimo simbolo, e lo confrontavo con quello memorizzato nello stato
- se erano diversi, terminavo con no, altrimenti cancellavo entrambi i simboli, e ricominciavo da capo
- se alla fine rimaneva solo il simbolo di blank, terminavo con yes

complessità: al primo passaggio fa n passi, al secondo passaggio fa $n - 2$ passi, al terzo passaggio fa $n - 4$ passi, e così via, fino a quando non rimane solo il simbolo di blank. quindi $n + (n - 2) + (n - 4) + \dots = \frac{n^2}{2} \rightarrow O(n^2)$

Portando in questo caso i concetti visti prima, potremmo affermare che in questo caso $PAL \in TIME(n^2)$

se provassimo con 2 nastri?

$\triangleright$	1	0	1	0	1	$\mathcal{L}$
q_0	q_2					
$\triangleright$	$\mathcal{L}$					

$$\delta(q_2, 1, \mathcal{L}) = (q_2, 1, R, 1, R)$$

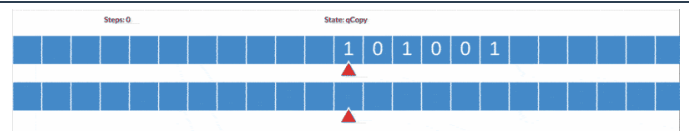
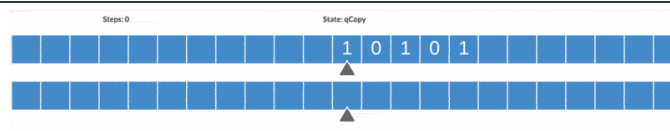
$$\delta(q_2, 0, \mathcal{L}) = (q_2, 0, R, 0, R)$$

posso copiare l'intero nastro 1 sul nastro 2 (al contrario, o in alternativa li copio uguali e uno lo leggo dalla fine), confrontarli con le due testine contemporaneamente, e cancellare entrambi i simboli se sono uguali, terminando con no se sono diversi, o spostandosi alla destra se sono uguali, e così via fino a quando non rimane solo il simbolo di blank.

questa semplice modifica, ci permette di portare la complessità da $O(n^2)$ a $O(n)$, e quindi $PAL \in TIME(n)$

accetta

non accetta



Sia M una K-MdT che opera in tempo $f(n)$, Allora possiamo costruire 1-MdT "equivalente" che opera in tempo $O(n \cdot f(n) + (f(n))^2)$

Sia $L \in TIME(f(n))$, allora:

$$(\forall \epsilon > 0)(L \in TIME(\epsilon f(n) + n + 2\frac{\epsilon}{6}n)) \implies L \in TIME(3n^2) \implies L \in TIME(\frac{1}{3} \cdot n^3 + \dots)$$

e quindi ogni volta che ho una costante (o una funzione di grado inferiore), posso trascurarla in comparazione a una funzione di grado superiore.

Sia $L \in TIME(f(n))$, allora esiste un programma RAM (detto **assembler**, una sorta di pseudo-codice in assembly) "equivalente" che opera in tempo $O(n + f(n))$

Sia P un programma RAM che opera in tempo $f(n)$, allora esiste una 7-MdT "equivalente" che opera in tempo $O(f(n)^3)$

si simula un formalismo con accesso diretto, **SENZA** avere accesso diretto (perchè devo scorrere il nastro). se c'è un algoritmo polinomiale in un qualsiasi linguaggio di programmazione, ci sarà sempre un algoritmo polinomiale equivalente in una macchina di turing